

# IMPLEMENTACE VÝKONNÉHO PÁROVACÍHO JÁDRA BURZOVNÍHO SYSTÉMU

**Jan Markes**

Bakalářská práce
Fakulta informačních technologií
České vysoké učení technické v Praze
Katedra softwarového inženýrství
Studijní program: Informatika
Specializace: Softwarové inženýrství
Vedoucí: Ing. Jan Matoušek
15. května 2026

FAKULTA INFORMAČNÍCH TECHNOLOGIÍ ČVUT V PRAZE

# Zadání bakalářské práce

| | |
|---|---|
| **Název:** | Implementace výkonného párovacího jádra burzovního systému |
| **Student:** | Jan Markes |
| **Vedoucí:** | Ing. Jan Matoušek |
| **Studijní program:** | Informatika |
| **Obor / specializace:** | Softwarové inženýrství 2021 |
| **Katedra:** | Katedra softwarového inženýrství |
| **Platnost zadání:** | do konce letního semestru 2027/2028 |

## Pokyny pro vypracování

Cílem práce je implementovat softwarové párovací jádro burzovních objednávek (Order Matching Engine) a jeho klíčovou datovou strukturu, limitní knihu objednávek (Limit Order Book). Práce se zaměří na identifikaci bottlenecků ve standardních implementacích a jejich odstranění s využitím moderního standardu C++ a optimalizací pro OS Linux.

Výzvy a optimalizace:
Hlavní výzvou práce je dosažení deterministické latence a maximalizace propustnosti systému, což vyžaduje překonání limitů běžných softwarových implementací. Práce se proto zaměří na identifikaci a řešení bottlenecků v následujících klíčových oblastech:
- správa paměti a eliminace alokací,
- optimalizace datových struktur,
- paralelizace a synchronizace,
- nízkoúrovňové optimalizace kódu,
- interakce s operačním systémem,
- měření a analýza výkonu.

Pokyny k vypracování:

1) Proveďte rešerši problematiky párování objednávek a fungování limitní knihy. Analyzujte faktory ovlivňující latenci v C++ a možnosti pro optimalizaci výkonu.
2) Navrhněte vhodnou výchozí architekturu pro párovací jádro s důrazem na efektivní

---

*Elektronicky schválil/a Ing. Michal Valenta, Ph.D. dne 19. února 2026 v Praze.*

správu paměti a minimalizaci zamykání.
3) Implementujte výchozí verzi a následně optimalizované verze párovacího jádra v C++.
4) Vhodným způsobem otestujte funkčnost řešení a změřte výkon pomocí profilovacích nástrojů v porovnání s referenčními implementacemi.
5) Analyzujte výsledky, popište přínos použitých optimalizací, limity řešení a navrhněte další možnosti vylepšení.



*Elektronicky schválil/a Ing. Michal Valenta, Ph.D. dne 19. února 2026 v Praze.*

České vysoké učení technické v Praze
Fakulta informačních technologií
© 2026 Jan Markes. Všechna práva vyhrazena.
*Tato práce vznikla jako školní dílo na Českém vysokém učení technickém v Praze, Fakultě informačních technologií. Práce je chráněna právními předpisy a mezinárodními úmluvami o právu autorském a právech souvisejících s právem autorským. K jejímu užití, s výjimkou bezúplatných zákonných licencí a nad rámec oprávnění uvedených v Prohlášení, je nezbytný souhlas autora.*

Odkaz na tuto práci: Markes Jan. *Implementace výkonného párovacího jádra burzovního systému*. Bakalářská práce. České vysoké učení technické v Praze, Fakulta informačních technologií, 2026.

*Rád bych poděkoval svému vedoucímu bakalářské práce Ing. Janu Matouškovi za cenné rady a odborné vedení při zpracování této práce.*

# Prohlášení

Prohlašuji, že jsem předloženou práci vypracoval samostatně a že jsem uvedl veškeré použité informační zdroje v souladu s Metodickým pokynem o dodržování etických principů při přípravě vysokoškolských závěrečných prací. Beru na vědomí, že se na moji práci vztahují práva a povinnosti vyplývající ze zákona č. 121/2000 Sb., autorského zákona, ve znění pozdějších předpisů. V souladu s ust. § 2373 odst. 2 zákona č. 89/2012 Sb., občanský zákoník, ve znění pozdějších předpisů, tímto uděluji nevýhradní oprávnění (licenci) k užití této mojí práce, a to včetně všech počítačových programů, jež jsou její součástí či přílohou a veškeré jejich dokumentace (dále souhrnně jen „Dílo“), a to všem osobám, které si přejí Dílo užít. Tyto osoby jsou oprávněny Dílo užít jakýmkoli způsobem, který nesnižuje hodnotu Díla a za jakýmkoli účelem (včetně užití k výdělečným účelům). Toto oprávnění je časově, teritoriálně i množstevně neomezené.

Prohlašuji, že jsem v průběhu příprav a psaní závěrečné práce použil nástroje umělé inteligence. Vygenerovaný obsah jsem ověřil. Stvrzuji, že jsem si vědom, že za obsah závěrečné práce plně zodpovídám.

V Praze dne 15. května 2026

# Abstrakt

Tato práce se zabývá návrhem a implementací vysoce výkonného párovacího jádra objednávek pro elektronické burzovní systémy, včetně jeho stěžejní datové struktury – limitní knihy objednávek. Text popisuje iterativní vývoj párovacího jádra a aplikaci optimalizačních technik za využití moderního standardu jazyka C++, které zajišťují jeho maximální propustnost a co nejnižší deterministickou latenci. Mezi tyto techniky patří volba vhodných datových struktur a algoritmů, eliminace dynamických alokací, optimalizace pro procesorovou mezipaměť a ladění na úrovni operačního systému Linux. Jednotlivé optimalizační kroky jsou měřeny pomocí profilovacích nástrojů na reálných tržních datech. Vliv těchto úprav na celkový výkon párovacího jádra je analyzován a porovnán s jeho verzemi z předchozích iterací a s jinými volně dostupnými řešeními. Výsledné párovací jádro dosahuje na testovaném prostředí propustnosti v řádu desítek milionů objednávek za sekundu při submikrosekundové latenci, čímž referenční řešení řádově překonává.

**Klíčová slova** vysokofrekvenční obchodování, burzovní systém, párovací jádro objednávek, limitní kniha objednávek, optimalizace výkonu, měření výkonu, nízká latence párování, vysoká propustnost zpráv, procesorová mezipaměť, C++, Linux

# Abstract

This thesis focuses on the design and implementation of a high-performance order matching engine for electronic exchange systems, including its core data structure – the limit order book. The text describes the iterative development of the matching engine and the application of optimization techniques utilizing modern C++ standard, which ensure its maximum throughput and the lowest possible deterministic latency. These techniques include the selection of appropriate data structures and algorithms, the elimination of dynamic memory allocations, processor cache optimization, and tuning at the Linux operating system level. Individual optimization steps are measured using profiling tools on real-world market data. The impact of these modifications on the overall performance of the matching engine is analyzed and compared with its previous iterations as well as other freely available solutions. In the testing environment, the resulting matching engine achieves a throughput of tens of millions of orders per second with sub-microsecond latency, outperforming reference solutions by orders of magnitude.

**Keywords** high-frequency trading, exchange system, order matching engine, limit order book, performance optimization, performance profiling, low-latency matching, high-throughput messaging, CPU cache, C++, Linux

# Obsah

**Úvod** 1

**1 Cíle práce** 2

**2 Burzovní systémy a párovací jádra** 3
2.1 Vysokofrekvenční obchodování . . . 3
2.2 Architektura burzovního systému . . . 4
2.2.1 Kritická a nekritická cesta . . . 4
2.2.2 Hlavní komponenty systému . . . 5
2.2.3 Provozní a technické aspekty . . . 5
2.3 Párovací jádro objednávek . . . 6
2.4 Typy a životní cyklus objednávek . . . 7
2.4.1 Životní cyklus a stavy objednávky . . . 8
2.4.2 Modifikátory doby platnosti . . . 9
2.5 Párovací algoritmy a pravidla priority . . . 10
2.5.1 Cenově-časová priorita (*price-time*) . . . 10
2.5.2 Proporcionální párování (*pro-rata*) . . . 11
2.5.3 Technologické dopady na párovací jádro . . . 11
2.6 Existující a referenční řešení . . . 11
2.6.1 Akademická referenční implementace . . . 12
2.6.2 Volně dostupná implementace . . . 12

**3 Faktory ovlivňující výkon párovacího jádra** 13
3.1 Reprezentace tržních dat a aritmetika . . . 13
3.1.1 Omezení typů s pohyblivou řádovou čárkou . . . 14
3.1.2 Aritmetika s pevnou řádovou čárkou . . . 14
3.2 Paměťová hierarchie a mezipaměť (*cache*) . . . 14
3.2.1 Prostorová lokalita a přenos dat . . . 15
3.2.2 Asymptotická složitost vs. reálný výkon . . . 15
3.2.3 Datově orientovaný návrh . . . 16
3.3 Dynamická alokace a správa paměti . . . 16
3.3.1 Prealokace a paměťové fondy . . . 16
3.3.2 Minimalizace režie C++ abstrakcí . . . 17
3.4 Architektura procesoru a predikce skoků . . . 17
3.4.1 Bezskokové programování a volba algoritmů . . . 17
3.4.2 Instrukční mezipaměť a rozložení kódu . . . 18

3.4.3 Zahřívání mezipaměti (*cache warming*) . . . . . 18
3.5 Interakce s OS Linux . . . . . 19
3.5.1 Minimalizace systémových volání . . . . . 19
3.5.2 Afinita a izolace vláken . . . . . 19
3.5.3 Správa virtuální paměti . . . . . 19

**4 Metodika měření a testování párovacího jádra 21**
4.1 Testovací prostředí . . . . . 21
4.1.1 Hardwarové prostředí . . . . . 21
4.1.2 Softwarové prostředí . . . . . 22
4.2 Charakteristika testovacích dat . . . . . 23
4.2.1 Reálná tržní data . . . . . 24
4.2.2 Syntetická data . . . . . 26
4.3 Nástroje pro testování a měření výkonu . . . . . 27
4.3.1 GoogleTest . . . . . 27
4.3.2 Google Benchmark . . . . . 28
4.3.3 Linux `perf` . . . . . 28
4.4 Zajištění determinismu měření . . . . . 29
4.4.1 Uzamykání fyzické paměti . . . . . 29
4.4.2 Deaktivace randomizace paměti . . . . . 29
4.4.3 Stabilizace frekvence a konfigurace CPU . . . . . 29
4.4.4 Izolace vlákna na úrovni OS . . . . . 30
4.4.5 Exaktní hardwarové měření času . . . . . 31
4.5 Metodika srovnání s referenčními řešeními . . . . . 31

**5 Návrh řešení 33**
5.1 Vymezení rozsahu a požadavky na systém . . . . . 33
5.1.1 Funkční požadavky . . . . . 33
5.1.2 Nefunkční požadavky . . . . . 34
5.2 Základní architektura párovacího jádra . . . . . 36
5.2.1 Konfigurace a řízení běhu . . . . . 38
5.2.2 Příprava, generování a export dat . . . . . 38
5.2.3 Burzovní doména a párovací jádro . . . . . 38
5.3 Scénáře testování a měření výkonu . . . . . 39
5.3.1 Scénáře testování funkčnosti . . . . . 39
5.3.2 Benchmarky a měření výkonu . . . . . 41

**6 Iterativní vývoj párovacího jádra 43**
6.1 Výchozí implementace . . . . . 43
6.1.1 Úprava integračních testů . . . . . 44
6.1.2 Měření výkonu pomocí TSC . . . . . 45
6.1.3 Výsledky měření . . . . . 46
6.2 Optimalizace datových struktur a algoritmů . . . . . 50
6.2.1 Přímé indexování objednávek . . . . . 50

6.2.2 Eliminace dynamických alokací . . . 52
6.2.3 Linearizace stromových struktur . . . 54
6.2.4 Bitové masky pro hledání cenových hladin . . . 55
6.2.5 Výsledky měření . . . 56
6.3 Izolace běhového prostředí . . . 58
6.3.1 Uzamčení fyzické paměti . . . 59
6.3.2 Afinita vláken a izolace jader . . . 63
6.3.3 Výsledky měření . . . 63
6.4 Nízkoúrovňové optimalizace . . . 65
6.4.1 Zarovnání pro CPU mezipaměť . . . 66
6.4.2 Eliminace instrukčních skoků . . . 67
6.4.3 Optimalizace správy paměťového fondu . . . 68
6.4.4 Optimalizační příznaky kompilace . . . 70
6.4.5 Výsledky měření . . . 72

**7 Srovnání a zhodnocení výsledků měření 75**
7.1 Vyhodnocení propustnosti a latence . . . 75
7.1.1 Srovnání propustnosti na LOBSTER datech . . . 75
7.1.2 Srovnání propustnosti na syntetických datech . . . 77
7.1.3 Srovnání distribuce latence a její stability . . . 78
7.2 Vliv volby překladače na výsledný výkon . . . 79
7.3 Srovnání s referenčními řešeními . . . 81
7.4 Návrhy na budoucí rozvoj . . . 82

**8 Závěr 84**

**A Uživatelská příručka 86**
A.1 Požadavky na systém . . . 86
A.2 Sestavení projektu . . . 86
A.3 Spuštění aplikace . . . 87
A.4 Testování a benchmarky . . . 87

**Bibliografie 88**

**Obsah příloh 93**

# Seznam obrázků


5.1 Schéma architektury výchozího řešení . . . . . 37

6.1 Plamenný graf zachycující výpadky stránek na kritické cestě . . 59
6.2 Plamenný graf po aplikaci uzamčení paměti pomocí `mlockall` . 60

7.1 Propustnost verzí jádra na LOBSTER datech . . . . . 76
7.2 Propustnost verzí jádra na syntetických datech . . . . . 77
7.3 Distribuce latence zpracování objednávek . . . . . 78
7.4 Srovnání propustnosti podle kompilátoru (s PGO) . . . . . 80
7.5 Srovnání latence podle kompilátoru (s PGO) . . . . . 80
7.6 Srovnání propustnosti vůči referenčním řešením (bez PGO) . . 81
7.7 Srovnání latence vůči referenčním řešením (bez PGO) . . . . . 82


# Seznam tabulek


4.1 Hardwarová specifikace testovacího prostředí . . . . . 22
4.2 Softwarová specifikace a testovací nástroje . . . . . 22
4.3 Struktura souboru zpráv ve formátu LOBSTER . . . . . 25
4.4 Struktura souboru stavů limitní knihy ve formátu LOBSTER . 26

6.1 Výsledky měření operací výchozí implementace . . . . . 47
6.2 Propustnost a latence výchozí implementace (LOBSTER data) 48
6.3 Propustnost a latence výchozí implementace (syntetická data) . 49
6.4 Výsledky měření operací architektury ploché limitní knihy . . . 56
6.5 Propustnost a latence ploché limitní knihy (LOBSTER data) . 57
6.6 Propustnost a latence ploché limitní knihy (syntetická data) . . 58
6.7 Výsledky mikrobenchmarků s aktivní izolací prostředí . . . . . 64
6.8 Propustnost a latence po izolaci prostředí (LOBSTER data) . . 64
6.9 Propustnost a latence po izolaci prostředí (syntetická data) . . 65
6.10 Výsledky mikrobenchmarků po paměťovém zarovnání . . . . . 67
6.11 Výsledky mikrobenchmarků po nasazení vzoru *free list* . . . . . 70
6.12 Vliv kompilátorových optimalizací na konstantní operace . . . . 72

6.13 Výsledky měření operací finální implementace . . . . . 72
6.14 Propustnost a latence finální implementace (LOBSTER data) . 73
6.15 Propustnost a latence finální implementace (syntetická data) . 74

# Seznam výpisů kódu

6.1 Definice třídy `OrderBook` pro výchozí implementaci . . . . . 44
6.2 Zamknutí frekvence a vypnutí úsporných stavů CPU . . . . . 46
6.3 Obnovení výchozího nastavení procesoru . . . . . 46
6.4 Spuštění párovacího jádra s LOBSTER daty . . . . . 48
6.5 Spuštění párovacího jádra s generováním dat . . . . . 49
6.6 Definice třídy `OrderBook` s využitím přímého indexování . . . . 51
6.7 Využití `monotonic_buffer_resource` pro eliminaci volání OS . 52
6.8 Struktura `OrderBook` s využitím vlastního paměťového fondu . 53
6.9 Architektura ploché limitní knihy s indexováním hladin . . . . 55
6.10 Výsledky běhu před aplikací `mlockall` . . . . . 61
6.11 Výsledky běhu po aplikaci `mlockall` . . . . . 62
6.12 Konfigurace zavaděče GRUB pro izolaci procesorových jader . . 63
6.13 Explicitní zarovnání struktury `Order` na hranici 32 B . . . . . 66
6.14 Správa volných bloků pro objednávky pomocí vzoru *free list* . . 69
6.15 Proces generování, tréninku a využití PGO profilu . . . . . 71

# Seznam zkratek

| | |
|---|---|
| AON | Platnost objednávky – vše, nebo nic (All-Or-None) |
| AoS | Pole struktur (Array of Structures) |
| ASLR | Randomizace rozvržení adresního prostoru (Address Space Layout Randomization) |
| AVX2 | Pokročilé vektorové rozšíření 2 (Advanced Vector Extensions 2) |
| CFS | Zcela spravedlivý plánovač (Completely Fair Scheduler) |
| CME | Chicago Mercantile Exchange |
| CPU | Centrální procesorová jednotka (Central Processing Unit) |
| CSV | Hodnoty oddělené čárkou (Comma-Separated Values) |
| D-cache | Datová mezipaměť (Data Cache) |
| DCE | Odstranění mrtvého kódu (Dead Code Elimination) |
| DOD | Datově orientovaný návrh (Data-Oriented Design) |
| EEVDF | Plánovač s nejbližším možným virtuálním termínem (Earliest Eligible Virtual Deadline First) |
| EHP | Explicitní obrovské stránky (Explicit Huge Pages) |
| FIFO | Párovací algoritmus první dovnitř, první ven (First-In, First-Out) |
| FIX | Protokol pro výměnu finančních informací (Financial Information eXchange) |
| FOK | Platnost objednávky – vyplnit, nebo zrušit (Fill-Or-Kill) |
| FPU | Jednotka pro pohyblivou řádovou čárku (Floating-Point Unit) |
| GCC | GNU Compiler Collection |
| GRUB | Grand Unified Bootloader |
| GTC | Platnost objednávky do zrušení (Good-Till-Canceled) |
| GTD | Platnost objednávky do určitého data (Good-Till-Date) |
| HFT | Vysokofrekvenční obchodování (High-Frequency Trading) |
| I-cache | Instrukční mezipaměť (Instruction Cache) |
| I/O | Vstupně-výstupní operace (Input/Output) |
| ID | Identifikátor (Identifier) |
| IEEE | Institut pro elektrotechnické a elektronické inženýrství (Institute of Electrical and Electronics Engineers) |
| IOC | Platnost objednávky – ihned provést, nebo zrušit (Immediate-Or-Cancel) |
| ITCH | Binární protokol burzy Nasdaq pro distribuci tržních dat |

| | |
|---|---|
| L1d | Datová mezipaměť úrovně 1 (Level 1 Data Cache) |
| L1i | Instrukční mezipaměť úrovně 1 (Level 1 Instruction Cache) |
| L2 | Mezipaměť úrovně 2 (Level 2 Cache) |
| L3 | Mezipaměť úrovně 3 (Level 3 Cache) |
| LMAX | London Multi-Asset Exchange |
| LOB | Limitní kniha objednávek (Limit Order Book) |
| LOBSTER | Systém pro efektivní rekonstrukci limitní knihy objednávek (Limit Order Book System – The Efficient Reconstructor) |
| LTO | Optimalizace ve fázi linkování (Link-Time Optimization) |
| MiFID | Směrnice o trzích finančních nástrojů (Markets in Financial Instruments Directive) |
| OMS | Systém pro správu objednávek (Order Management System) |
| OME | Párovací jádro objednávek (Order Matching Engine) |
| OOP | Objektově orientované programování (Object-Oriented Programming) |
| OS | Operační systém (Operating System) |
| OTR | Poměr odeslaných objednávek a uskutečněných obchodů (Order-to-Trade Ratio) |
| OUCH | Binární protokol burzy Nasdaq pro zadávání objednávek |
| PGO | Optimalizace řízená profilem (Profile-Guided Optimization) |
| RAM | Operační paměť s přímým přístupem (Random Access Memory) |
| SEC | Komise pro cenné papíry a burzy (Securities and Exchange Commission) |
| SIMD | Jedna instrukce, více dat (Single Instruction, Multiple Data) |
| SMT | Souběžné zpracování vláken (Simultaneous Multithreading) |
| SoA | Struktura polí (Structure of Arrays) |
| SRP | Princip jedné odpovědnosti (Single Responsibility Principle) |
| THP | Transparentní obrovské stránky (Transparent Huge Pages) |
| TIF | Doba platnosti objednávky (Time-In-Force) |
| TLB | Vyrovnávací paměť překladu adres (Translation Lookaside Buffer) |
| TSC | Čítač časových značek (Time Stamp Counter) |
| UDP | Uživatelský datagramový protokol (User Datagram Protocol) |
| UEFI | Jednotné rozšiřitelné rozhraní firmwaru (Unified Extensible Firmware Interface) |
| vDSO | Virtuální dynamický sdílený objekt (Virtual Dynamic Shared Object) |

# Úvod

S nástupem elektronického a vysokofrekvenčního obchodování se požadavky na rychlost burzovních systémů posunuly z milisekund do řádu nanosekund. Srdcem každé elektronické burzy je párovací jádro objednávek a jeho centrální datová struktura, limitní kniha objednávek. Při zátěži milionů zpráv za sekundu již nerozhoduje pouze asymptotická složitost algoritmů, ale především fyzikální limity hardwaru, jako jsou latence procesorové mezipaměti, propustnost paměťové hierarchie a režie operačního systému.

Standardní programovací přístupy v tomto prostředí narážejí na své limity. Tradiční objektově orientovaný návrh, dynamická alokace paměti za běhu či využití uzlových datových struktur způsobují fragmentaci a nutí procesor neustále čekat na data, což způsobuje nepředvídatelná zpoždění. Cílem této práce je navrhnout, implementovat a optimalizovat párovací jádro s využitím moderního standardu jazyka C++. Práce se zaměřuje na překonání limitů běžných softwarových implementací pomocí nízkoúrovňových technik, jako je eliminace dynamických alokací, bezskokové programování a striktní izolace vláken na úrovni operačního systému Linux. Pomocí profilovacích nástrojů a zátěžových testů na reálných tržních datech práce demonstruje, jak lze dosáhnout propustnosti a deterministické latence blížící se teoretickému hardwarovému minimu.

Text práce je logicky rozdělen do tří hlavních částí. Úvodní teoretické kapitoly definují principy párování objednávek a popisují faktory ovlivňující výkon párovacího jádra. Praktická část detailně popisuje iterativní vývoj, postupnou optimalizaci a měření výkonu vytvořeného jádra. Závěr následně analyzuje naměřené výsledky a porovnává je s referenčními implementacemi.

# Kapitola 1

# Cíle práce

Hlavním cílem této práce je návrh a implementace vysoce výkonného párovacího jádra burzovního systému, včetně jeho centrální datové struktury, limitní knihy objednávek, s důrazem na maximální propustnost a minimální deterministickou latenci.

Teoretická část práce definuje fungování burzovních systémů, identifikuje faktory ovlivňující výkon párovacího jádra implementovaného v moderním standardu jazyka C++ a stanovuje exaktní metodiku měření.

Praktická část se věnuje iterativnímu vývoji. Po návrhu výchozí architektury jsou pomocí profilovacích nástrojů identifikovány kritické úseky kódu, které degradují celkový výkon jádra. Následně dochází k aplikaci algoritmických optimalizací a k nízkoúrovňovému ladění v prostředí operačního systému Linux.

Závěr práce je věnován analýze naměřených výsledků na reálných tržních datech. Text hodnotí přínos jednotlivých optimalizací, porovnává finální výkon s předchozími iteracemi i existujícími referenčními implementacemi a navrhuje možnosti dalšího rozvoje systému.

Kapitola 2

# Burzovní systémy a párovací jádra

*Tato kapitola uvádí teoretický kontext a doménu, kterou se předkládaná práce zabývá. Rostoucí vliv vysokofrekvenčního obchodování klade na dnešní elektronické burzy vysoké nároky z hlediska rychlosti a spolehlivosti, což neustále motivuje k optimalizaci burzovních technologií. Ačkoliv je celková architektura burzovních systémů vysoce komplexní, její stručné zmapování je nezbytné pro pochopení role párovacího jádra, které je kritickým prvkem celého systému a hlavním zaměřením této práce. Klíčovou datovou strukturou tohoto jádra je limitní kniha objednávek. V souvislosti s ní text rozebírá základní typy objednávek a jejich životní cyklus. Závěr kapitoly se následně věnuje algoritmům a prioritním pravidlům, kterými se proces párování a vytváření obchodů exaktně řídí.*

## 2.1 Vysokofrekvenční obchodování

Vysokofrekvenční obchodování (HFT) je specifickou formou algoritmického obchodování. Vzhledem k jeho globálnímu charakteru se definice tohoto oboru opírá o přístupy předních světových regulátorů finančních trhů, jako je americká Komise pro cenné papíry a burzy (SEC) a evropská směrnice MiFID II. Z pohledu SEC je pro HFT charakteristické využití velmi rychlých a sofistikovaných počítačových programů pro generování, směrování a exekuci objednávek. Dále se vyznačuje využíváním kolokačních služeb pro minimalizaci síťové latence, velmi krátkými časovými úseky pro držení pozic, vysokou mírou rušení zadaných objednávek a ukončením obchodního dne s minimem otevřených pozic (*flat position*) [1]. Evropská legislativa tuto charakteristiku dále rozšiřuje o požadavek na absolutní absenci lidského zásahu do analýzy a exekuce obchodů [2].

Z technologického hlediska je doména HFT neustálým závodem o minimalizaci síťové i procesní latence. Na elektronických trzích totiž platí pravidlo absolutní časové priority. Jakmile se objeví nová obchodní příležitost, profituje z ní pouze ten účastník, jehož požadavek dorazí do burzovního systému jako první. Tento nekompromisní konkurenční boj mezi algoritmickými obchodníky vyvíjí značný tlak na infrastrukturu burzy, od které se vyžaduje maximální propustnost zpráv a co nejnižší latence jejich zpracování.

Důsledkem snahy o minimalizaci rizika a dosažení optimální pozice je neustálé sondování trhu. HFT algoritmy generují obrovské množství objednávek, z nichž většina je krátce po svém odeslání zrušena nebo modifikována. Algoritmy tak v reálném čase reagují na nepatrné změny v tržní mikrostruktuře. Tento jev se kvantifikuje pomocí metriky Order-to-Trade Ratio (OTR), jež vyjadřuje poměr mezi odeslanými zprávami a uskutečněnými obchody. Na moderních burzách nabývá OTR vysokých hodnot, kdy na jednu spárovanou transakci připadají stovky stavových zpráv [3]. Architektura párovacího jádra proto musí disponovat vysokou propustností, aby dokázala bez zpoždění odbavit miliony zpráv za sekundu bez ohledu na skutečný počet realizovaných obchodů.

Posledním, avšak z hlediska softwarového inženýrství nejkritičtějším faktorem, je požadavek na striktní determinismus. Pro účastníky trhu není určující pouze nízká průměrná latence burzy, ale především eliminace jejích výkyvů (*jitter*). Pokud burzovní systém standardně odpovídá za 10 mikrosekund, avšak vlivem kontextového přepnutí operačního systému, výpadku v procesorové mezipaměti (*cache miss*) nebo dynamické alokace paměti dojde k nečekanému zpoždění, stává se pro vysokofrekvenční obchodování nespolehlivým. Návrh takové infrastruktury proto vyžaduje nízkoúrovňový přístup a minimalizaci interakcí se standardními mechanismy operačního systému za účelem dosažení absolutně predikovatelného výkonu.

## 2.2 Architektura burzovního systému

Moderní burzovní systém (neboli elektronická burza) tvoří komplexní platformu navrženou tak, aby s velmi nízkou latencí a vysokou propustností plynule párovala nákupní a prodejní příkazy tržních účastníků [4]. Architektura se primárně rozděluje na operace na kritické cestě a operace mimo ni, aby bylo dosaženo co nejvyššího možného výkonu.

### 2.2.1 Kritická a nekritická cesta

Základním architektonickým principem burzovního systému je minimalizace počtu úloh na kritické cestě. Kritickou cestou se nazývá sekvence úloh, která musí proběhnout v řádu mikrosekund (případně i nanosekund), od přijetí tržního příkazu až po generování zprávy o jeho provedení. Zahrnuje přijetí příkazu, kontrolu obchodních rizik, spárování v knize objednávek a odeslání

potvrzení. Všechny ostatní procesy, jako je odesílání tržních dat širší veřejnosti, auditní logování, zúčtování a reporting kvůli regulacím, jsou přesunuty mimo kritickou cestu a provádějí se s nižší prioritou (v řádu milisekund nebo sekund) [5].

### 2.2.2 Hlavní komponenty systému

Architektura burzovního systému se obvykle skládá z následujících klíčových komponent, které si navzájem asynchronně předávají události.

**Klientská brána (*client gateway*)**
Klientská brána tvoří vstupní bod systému. Přijímá příkazy přes síťové protokoly (např. textový FIX nebo binární OUCH). Má za úkol příkazy normalizovat do interního formátu burzy, provést autentizaci účastníka a zabránit přetížení pomocí omezování rychlosti požadavků (*rate limiting*) [5].

**Správce příkazů a řízení rizik (*order and risk manager*)**
Předtím, než se tržní příkaz dostane do párovacího jádra, je nutné provést předobchodní (*pre-trade*) kontrolu rizik. Systém ověřuje cenové limity, maximální povolenou velikost objednávky a dostatek prostředků k její realizaci [6].

**Párovací jádro objednávek (*order matching engine*)**
Tato klíčová komponenta burzy udržuje knihu objednávek (*order book*) kompletně v operační paměti, aby se vyhnula zpožděním diskových operací [6]. Pro efektivitu využívá datové struktury optimalizované pro časovou složitost ideálně $\mathcal{O}(1)$ nebo nejhůře $\mathcal{O}(\log n)$ [5]. Párovací jádro je hlavním tématem této práce, a proto je podrobněji rozebráno v následující sekci.

**Brána tržních dat (*market data gateway*)**
Komponenta dedikovaná k publikování anonymizovaných dat o stavu knihy a proběhlých obchodech. Z důvodu minimalizace latence a zajištění férovosti jsou data rozesílána prostřednictvím protokolu UDP multicast (např. pomocí binárního protokolu ITCH), často v podobě redundantních datových toků pro zajištění spolehlivosti [7].

### 2.2.3 Provozní a technické aspekty

Kromě softwarové architektury definují výkonnost a spolehlivost burzovního systému také specifické návrhové principy a fyzické umístění infrastruktury.

#### Optimalizace pro nízkou latenci a vysokou propustnost

Aby mohlo jádro zpracovat miliony objednávek za sekundu, musí architektura obcházet běžné limity operačních systémů. Klíčové komponenty proto často

využívají jednovláknovou architekturu (*single-threading*) a uzamykání vlákna na dedikované fyzické jádro procesoru (*CPU pinning*). Tím se eliminuje režie spojená s přepínáním kontextu a zamykáním sdílených zdrojů. Komunikace mezi vlákny probíhá přes fronty bez využití zamykání (*lock-free ring buffers*), známé například z obchodovací platformy LMAX a její komponenty Disruptor [8]. Síťová vrstva pak využívá techniky obcházení jádra OS (*kernel bypass*), které umožňují číst síťové pakety přímo do uživatelského paměťového prostoru.

### Odolnost vůči výpadkům a obnova po havárii

Vzhledem k tomu, že je stav párovacího jádra udržován výhradně v operační paměti, hrozí při výpadku napájení či chybě systému jeho nevratná ztráta. Tento problém se architektonicky řeší pomocí návrhového vzoru záznamu událostí (*event sourcing*). Každá příchozí zpráva je ihned asynchronně a sekvenčně zapsána do neměnného transakčního deníku (*append-only log*). Z tohoto logu lze kdykoliv deterministicky zrekonstruovat přesný stav systému. Pro zajištění nepřetržitého provozu navíc běží paralelně několik identických záložních systémů, které zpracovávají stejný tok dat. V případě selhání jednoho jádra okamžitě přebírá jeho roli jádro záložní (*failover*) [8].

### Fyzická infrastruktura

Na úrovni fyzické infrastruktury hraje v HFT roli každý metr kabelu. Obchodníci proto často umisťují své servery přímo do stejného datového centra, kde fyzicky sídlí burzovní systém (*colocation*). Aby burza zajistila všem účastníkům naprosto spravedlivé a rovné podmínky, využívá se technika fyzického vyrovnávání latence (*latency equalization*). Ta spočívá v umělém navinutí a naměření přesně stejné délky optického vlákna pro všechny účastníky tak, aby žádný z nich nebyl zvýhodněn kratší fyzickou vzdáleností k hraničním síťovým přepínačům [7].

## 2.3 Párovací jádro objednávek

Párovací jádro objednávek (*order matching engine*) tvoří ze softwarového hlediska centrální komponentu burzovního systému, která přijímá již normalizované a zvalidované zprávy z klientské brány. Toto jádro se řídí naprosto deterministickými pravidly, což znamená, že stejná posloupnost vstupních událostí vždy vygeneruje identický a předvídatelný výsledek [5]. Aby bylo dosaženo velmi nízké latence v řádu mikrosekund, udržuje jádro svůj kompletní stav výhradně v operační paměti (*in-memory*). Jakýkoliv synchronní zápis na tradiční pevný disk nebo do databáze by celý systém neúměrně zdržel, a proto jsou veškeré diskové vstupně-výstupní (I/O) operace z této kritické cesty striktně vyloučeny [5, 6].

Vnitřní stav tohoto jádra je pro každý obchodovaný instrument reprezentován specializovanou datovou strukturou zvanou limitní kniha objednávek (LOB). Tato kniha v reálném čase sdružuje všechny aktivní nabídky a poptávky tržních účastníků [4]. Z logického hlediska je kniha striktně rozdělena na dvě protichůdné strany: stranu poptávky, tvořenou nákupními objednávkami (*bid*), a stranu nabídky, tvořenou prodejními objednávkami (*ask*).

Dynamika řazení uvnitř limitní knihy odráží základní ekonomické principy chování účastníků trhu. Kupující mají přirozenou snahu nakoupit aktivum co nejlevněji, nicméně párovací algoritmus dává absolutní přednost tomu, kdo je ochoten zaplatit nejvíce. Nákupní strana je proto řazena sestupně od nejvyšší nabízené ceny. Naopak prodávající chtějí prodat co nejdráže, ale přednost získává ten, kdo trhu nabídne nejnižší cenu, a proto je prodejní strana řazena vzestupně od nejnižší hodnoty [5]. Pokud se na stejné cenové hladině sejde více objednávek, rozhodujícím faktorem pro určení priority exekuce se stává čas jejich příchodu, uplatňuje se tedy pravidlo *first in, first out* (FIFO) [4].

Vrchol knihy objednávek (*top of book*) je tvořen nejvyšší dostupnou poptávkou (*best bid*) a nejnižší dostupnou nabídkou (*best ask*). Cenový rozdíl mezi těmito dvěma hodnotami se označuje jako rozpětí neboli *spread* [4]. K momentu exekuce obchodu dochází ve chvíli, kdy do jádra dorazí nová objednávka, jejíž cenový limit tento *spread* překoná, čímž se otevírá prostor pro protnutí nabídky s poptávkou. Jádro v takovém okamžiku kompatibilní objednávky neprodleně spáruje, vygeneruje zprávy o provedení obchodu pro obě strany a realizované objemy z limitní knihy trvale vymaže [4, 5].

Aby mohl celý tento proces probíhat v mikrosekundách a systém bez zaváhání zvládal miliony zpráv za sekundu, jsou na návrh limitní knihy kladeny vysoké algoritmické nároky. Je naprosto klíčové, aby operace vkládání nové objednávky, její mazání i vyhledávání nejlepší protistrany pro párování probíhaly s co nejnižší asymptotickou složitostí, ideálně v konstantním čase $\mathcal{O}(1)$ nebo nejhůře v čase logaritmickém $\mathcal{O}(\log n)$ [5] vzhledem k počtu cenových hladin. V literatuře se jako standardní přístup často uvádí kombinace hashovacích tabulek pro mapování cenových hladin a spojových seznamů pro udržení časové priority [5]. Jak však bude detailně rozebráno v implementační části této práce, moderní postupy v jazyce C++ nabízí pro tyto účely ještě efektivnější datové struktury a paměťové optimalizace, které dokáží latenci dále minimalizovat.

## 2.4 Typy a životní cyklus objednávek

Párovací jádro musí při zpracování objednávek zohlednit zvolenou exekuční strategii obchodníka. Z hlediska způsobu realizace se rozlišují tři základní typy objednávek.

### Tržní objednávka (*market order*)

Účastník trhu požaduje okamžitý nákup nebo prodej aktiva, přičemž specifikuje pouze požadované množství, nikoliv cenu. Primární výhodou tohoto

příkazu je jistota provedení obchodu, pakliže je na trhu dostupná dostatečná likvidita. Zásadní nevýhodou je riziko cenového skluzu (*slippage*), jelikož tržní objednávka postupně konzumuje nejlepší dostupné nabídky v knize objednávek, dokud není plně vyřízena. Z povahy své definice se tržní objednávky do limitní knihy neukládají, nýbrž z ní přímo odčerpávají likviditu [9, 10].

**Limitní objednávka (*limit order*)**
Účastník trhu si přesně určí limitní cenu, za kterou je ochoten nakoupit (maximální cena) nebo prodat (minimální cena). Definuje tedy jak množství, tak i požadovanou cenovou hladinu. Výhodou je garance realizační ceny. Nevýhodou je naopak nejistota provedení obchodu – pokud tržní cena k zadanému limitu nedojde, obchod se neprovede. Tyto čekající limitní objednávky se po přijetí ukládají do párovacího jádra a tvoří obsah limitní knihy objednávek [9, 10].

**Stop objednávka (*stop-loss order*)**
Tento příkaz slouží primárně k omezení případné ztráty nebo k ochraně již dosaženého zisku. Účastník trhu u něj definuje tzv. spouštěcí cenu (*stop price*). Z architektonického hlediska je klíčové, že stop objednávky zpravidla neleží přímo v aktivní limitní knize párovacího jádra, ale jsou systémem pouze pasivně sledovány. Jakmile tržní cena dosáhne spouštěcí hranice, objednávka se automaticky transformuje a vstupuje do párovacího jádra jako standardní tržní objednávka. V případě specifické varianty tzv. *stop-limit* objednávky se příkaz po aktivaci mění na objednávku limitní. Pro párovací algoritmus na kritické cestě tedy stop objednávka jako taková neexistuje, systém zpracovává až její spuštěnou podobu [9, 10].

### 2.4.1 Životní cyklus a stavy objednávky

Z pohledu softwarového inženýrství přistupuje systém k objednávce jako ke stavovému automatu (*state machine*). Každý požadavek prochází během svého životního cyklu striktně definovanými stavy, které jsou standardizovány například v rámci protokolu FIX pro elektronické obchodování [5]. Změny stavů zpracovává párovací jádro, nicméně jejich kompletní evidenci a sledování (zda byla objednávka např. zrealizována, částečně vyřízena či zamítnuta) zajišťuje primárně systém pro správu objednávek (OMS) [6]. K těm nejzásadnějším stavům patří:

**Nová / přijatá (*new / accepted*)**
Objednávka úspěšně prošla klientskou bránou a kontrolou rizik, byla párovacím jádrem přijata a zařazena do limitní knihy objednávek [4, 5].

**Částečně vyřízená (*partially filled*)**
V knize se našla protistrana pouze pro část požadovaného množství. Tato odpovídající část byla úspěšně spárována, zatímco zbývající nezobchodovaný objem objednávky nadále čeká v limitní knize [4, 6].

**Kompletně zrealizovaná (*filled*)**
Celé požadované množství objednávky bylo úspěšně spárováno s tržní protistranou. Jádro vygeneruje zprávu o provedení obchodu a objednávka je z limitní knihy odstraněna [5, 6].

**Modifikovaná / nahrazená (*modified / replaced*)**
Účastník trhu může změnit parametry již podané a čekající objednávky (např. upravit limitní cenu nebo požadovanou velikost), aniž by ji musel zcela rušit a zadávat znovu [4, 5]. Z algoritmického hlediska je však zásadní, že změna ceny nebo navýšení objemu zpravidla znamená ztrátu časové priority a přesun objednávky na konec fronty.

**Zrušená / zamítnutá (*canceled / rejected*)**
Objednávka byla zrušena buď zásahem uživatele, nebo automaticky systémem (např. vlivem modifikátorů platnosti) a byla z knihy odstraněna, aniž by došlo k jejímu úplnému vyřízení [5, 6].

### 2.4.2 Modifikátory doby platnosti

V prostředí elektronického obchodování se objednávky často doplňují o modifikátory doby platnosti (TIF). Tyto parametry exaktně určují, jak dlouho smí objednávka v systému existovat před svým případným zrušením. K těm nejpoužívanějším patří:

**Good-Till-Canceled (GTC)**
Standardní příkaz s neomezenou platností. Objednávka leží v limitní knize a čeká na spárování tak dlouho, dokud nedojde k jejímu úplnému vyřízení, nebo dokud ji účastník explicitně nezruší [9, 10].

**Immediate-Or-Cancel (IOC)**
Agresivní typ příkazu vyžadující okamžitou reakci. Párovací jádro se pokusí okamžitě spárovat co největší možné množství. Pokud po tomto vyhodnocení zbyde nezobchodovaný objem, neukládá se do limitní knihy, ale je automaticky a bezodkladně zrušen [9, 10].

**Fill-Or-Kill (FOK)**
Striktní varianta příkazu, u níž se vyžaduje okamžité a kompletní vyřízení. Systém v tomto režimu nedovoluje částečnou realizaci (stav *partially filled*). Pokud v daném okamžiku neexistuje na trhu dostatečná likvidita k provedení celého objemu, jádro neprovede žádný obchod a celou objednávku zruší [9, 10].

**Good-Till-Date (GTD)**
Umožňuje obchodníkovi přesně definovat datum a čas expirace. Objednávka zůstává v systému aktivní, dokud není spárována, uživatelem zrušena, nebo dokud nevyprší její stanovená platnost [9].

**All-Or-None (AON)**
Příkaz, který podobně jako FOK vyžaduje kompletní uspokojení celého požadovaného množství. Na rozdíl od FOK však nevyžaduje okamžitou realizaci. Může tak ležet aktivní v limitní knize a čekat, dokud se v ní nenakumuluje dostatečná protinabídka k uspokojení celého objemu najednou [9, 10].

## 2.5 Párovací algoritmy a pravidla priority

Pokud do párovacího jádra dorazí nová tržní objednávka, může běžně nastat situace, kdy v limitní knize na nejlepší dostupné ceně čeká více nabídek od různých účastníků trhu, jejichž celkový objem tuto poptávku převyšuje. Vzhledem k tomu, že nelze plně zrealizovat všechny zúčastněné strany, musí systém využívat deterministický a spravedlivý algoritmus párování. Tento algoritmus stanovuje exaktní pravidla pro určení pořadí a poměru, v jakém budou dostupné prostředky přiděleny [4, 11]. Volba tohoto algoritmu zásadně ovlivňuje nejen dynamiku trhu, ale i technologický návrh párovacího jádra.

### 2.5.1 Cenově-časová priorita (*price-time*)

Nejrozšířenějším standardem, který je využíván většinou elektronických burz, je algoritmus *price-time*, často označovaný jako FIFO [4, 7]. Toto pravidlo přiděluje prioritu ve dvou na sebe navazujících krocích:

1. **Cena (*price*)**
   Primárním kritériem v hierarchii limitní knihy je vždy cena. Objednávka s pro trh výhodnější cenou (nejvyšší u poptávky, nejnižší u nabídky) má vždy přednost. Objednávka s lepší cenou se spáruje dříve i v případě, že do systému dorazila později [5, 12].

2. **Čas (*time*)**
   Pokud se na stejné cenové hladině sejde více čekajících objednávek, nastupuje jako sekundární kritérium časová priorita. Rozhodujícím faktorem se stává absolutní čas příchodu (*timestamp*) do párovacího jádra. Účastník, jehož objednávka byla do knihy zapsána nejdříve, získá přednost při realizaci obchodu před ostatními na stejné cenové hladině [11, 12].

Právě tento mechanismus, založený na mikrosekundových rozdílech v čase příchodu, je hlavním důvodem pro výraznou minimalizaci latence v odvětví HFT [4].

### 2.5.2 Proporcionální párování (*pro-rata*)

Alternativním přístupem k řešení shod na stejné cenové hladině je proporcionální párování. Tento algoritmus se často využívá na trzích s opcemi, úrokovými sazbami a u vybraných futures kontraktů (například na americké burze Chicago Mercantile Exchange (CME)) [4, 13].

Algoritmus *pro-rata* sice nadále dodržuje cenovou prioritu, ale při shodě ceny nezohledňuje čas příchodu objednávky. Namísto toho exekuci rozdělí poměrově vůči velikosti jednotlivých nabídek čekajících na dané hladině [11, 13].

Matematicky to znamená, že pokud objednávka tvoří určitý procentuální podíl celkové dostupné likvidity na dané cenové hladině, získá identický procentuální podíl z objemu příchozí tržní objednávky [12]. Případný zbytek po zaokrouhlení (*fractional shares*) se následně obvykle rozdělí doplňkovým pravidlem FIFO [13]. Důvodem pro zavedení tohoto mechanismu je snaha motivovat účastníky trhu, aby nesoutěžili primárně v rychlosti, ale naopak poskytovali trhu co největší objemy likvidity.

### 2.5.3 Technologické dopady na párovací jádro

Z pohledu softwarového inženýrství je klíčové, že volba párovacího algoritmu přímo determinuje datové struktury použité při implementaci jádra. Jak bylo zmíněno dříve, operace nad limitní knihou musí pro dosažení minimální latence probíhat s asymptotickou složitostí blížící se $\mathcal{O}(1)$ [5].

Při použití algoritmu *price-time* je nezbytné udržovat striktní chronologické pořadí. Architektura tak pod každou cenovou hladinou typicky využívá struktury podobné obousměrným spojovým seznamům (*doubly linked list*), do kterých přidává nové objednávky na konec a při párování je odebírá výlučně ze začátku [5].

Oproti tomu architektura postavená na algoritmu *pro-rata* nevyžaduje udržování přesného pořadí, nýbrž efektivní agregaci a udržování celkového objemu napříč všemi objekty na dané cenové hladině. Jádro navíc musí disponovat datovou strukturou, která algoritmu umožní rychle iterovat přes všechny zúčastněné objednávky k výpočtu jejich poměrného podílu na exekuci. K tomu se typicky využívají struktury s lokální paměťovou návazností (např. dynamická pole či prealokované bloky paměti), čímž se maximalizuje prostorová lokalita dat a efektivně využívá mezipaměť procesoru (minimalizace *cache misses*).

## 2.6 Existující a referenční řešení

Při návrhu a vývoji softwarových systémů s požadavkem na vysoký výkon je klíčové nevytvářet řešení izolovaně, ale zasadit jej do kontextu aktuálního stavu vývoje (*state of the art*). Pro objektivní zhodnocení výkonnosti a architektonické kvality vlastního párovacího jádra jsou pro účely této práce vybrány dvě

referenční implementace. Tyto projekty slouží při finálním srovnání výkonu v kapitole 7.

### 2.6.1 Akademická referenční implementace

Jako první referenční projekt je zvolena bakalářská práce [14], jejímž autorem je Jakub Filipovský. Text se nezaměřuje pouze na implementaci párovacího jádra, ale řeší architekturu celého burzovního systému s důrazem na jeho celkový výkon. Součástí projektu jsou mimo jiné i připravené výkonnostní testy pro izolované párování, a to v jazycích Python, Java a C++. Jak autor dokládá, implementace v jazyce C++ dosahuje v těchto testech nejlepších výsledků, což z ní dělá ideální výchozí podklad pro srovnání s touto prací.

Z architektonického hlediska je párovací jádro referenční implementace postaveno na struktuře binární haldy, konkrétně využívá standardní kontejner `std::priority_queue` pro obě strany limitní knihy. Vzhledem k algoritmickým omezením dané struktury (nemožnost efektivně odstraňovat prvky mimo vrchol haldy) systém spoléhá na tzv. líné mazání (*lazy cancellation*). Zrušené objednávky tak zůstávají v paměti až do chvíle, kdy probublají na vrchol, což sice zajišťuje složitost zrušení $\mathcal{O}(1)$, ale zvyšuje paměťovou režii. K vyhledávání instancí objednávek je využíváno dynamické pole chytrých ukazatelů (*smart pointers*). Popsaný přístup tak poskytuje ideální příležitost k porovnání tradičního návrhu s vysoce optimalizovaným datově orientovaným návrhem vlastního jádra.

### 2.6.2 Volně dostupná implementace

Druhou zvolenou implementací je *open-source* projekt od Piyushe Kumara [15]. Jedná se o vysoce optimalizované párovací jádro napsané v moderním standardu C++20, které se zaměřuje na maximalizaci propustnosti, efektivní využití mezipaměti a dosažení deterministické submikrosekundové latence.

Na rozdíl od první referenční implementace zvolený projekt opouští tradiční stromové a uzlové struktury a plně využívá datově orientovaný návrh (DOD). Jádro je postaveno na takzvané ploché limitní knize (*flat orderbook*), kde jsou cenové hladiny reprezentovány ve struktuře `std::vector` zaručující maximální prostorovou lokalitu a kontinuální přístup do paměti. Pro okamžité přeskakování prázdných cenových hladin systém využívá bitové masky ve spojení se specializovanými intrinsickými funkcemi. Další zásadní vlastností je absolutní eliminace dynamické alokace paměti na kritické cestě (*hot path*) využitím prealokovaných paměťových fondů z knihovny `std::pmr`.

Popsaný systém tak představuje klíčové srovnávací měřítko pro předkládanou práci. Jelikož obě řešení staví na pokročilých principech datově orientovaného návrhu, o výsledcích finálního srovnání rozhodují především specifické nízkoúrovňové optimalizace a implementační detaily, kterým se věnují následující kapitoly.

Kapitola 3

# Faktory ovlivňující výkon párovacího jádra

*Základními měřítky kvality každého párovacího jádra jsou latence a propustnost. Ve světě HFT, kde o zisku či ztrátě rozhodují nanosekundy, nelze spoléhat pouze na optimální asymptotickou složitost algoritmů. Aby systém dosahoval maximálního výkonu, musí být implementován s velkým důrazem na to, jak napsaný kód interaguje s hardwarem. Z těchto důvodů představuje jazyk C++ preferovanou volbu pro vývoj HFT systémů. Jeho výhodou je podpora abstrakcí s nulovou režií (zero-cost abstractions) a možnost kontroly nad alokací a správou paměti.*
*Tato kapitola analyzuje klíčové nízkoúrovňové a hardwarové faktory, které ovlivňují výkon párovacího jádra. Postupně rozebírá problematiku reprezentace tržních dat, vliv paměťové hierarchie a mezipaměti procesoru, režie spojené s dynamickou alokací paměti, vliv architektury procesoru na predikci skoků a v neposlední řadě také způsoby, jak minimalizovat režii OS Linux. Závěry z této kapitoly následně poslouží jako podklady pro implementaci jádra v praktické části této práce.*

## 3.1 Reprezentace tržních dat a aritmetika

Jedním z nejdůležitějších architektonických rozhodnutí při návrhu párovacího jádra je volba datových typů pro reprezentaci jednotlivých položek tržních zpráv a dat, jako jsou cena a objem. Ačkoliv se pro ukládání cen (například 10,50 $) intuitivně nabízí využití standardních datových typů s pohyblivou řádovou čárkou, jako jsou v jazyce C++ typy `float` nebo `double`, v profesionálním vývoji burzovních systémů a finančních aplikací je tento přístup zcela nepřípustný [16, 17].

### 3.1.1 Omezení typů s pohyblivou řádovou čárkou

Hlavní problém spočívá ve způsobu, jakým procesory s těmito datovými typy pracují. Většina systémů implementuje typy s pohyblivou řádovou čárkou podle standardu IEEE 754 [17, 18]. Z matematické podstaty tohoto binárního formátu nelze většinu běžných desítkových zlomků, jako je například hodnota 0,1, zapsat do paměti naprosto přesně. Ukládají se pouze jako velmi blízké aproximace (např. s nekonečným periodickým rozvojem v binární soustavě) [17, 18]. Při iterativním sčítání a odčítání se tyto drobné nepřesnosti nevyhnutelně akumulují, což vede ke známému jevu, kdy se například součet $0{,}1 + 0{,}2$ v počítači nerovná přesně 0,3 [16, 17]. Tyto odchylky navíc zcela znemožňují exaktní porovnávání hodnot, např. u hledání cenové hladiny. V burzovním prostředí, které podléhá přísnému dohledu regulačních úřadů, je i kumulovaná účetní chyba v řádech zlomků centů neakceptovatelná [17].

### 3.1.2 Aritmetika s pevnou řádovou čárkou

Výše uvedený nedostatek se v praxi řeší přechodem na aritmetiku s pevnou řádovou čárkou (*fixed-point arithmetic*). Párovací jádro ve své paměti neudržuje desetinná čísla, ale veškeré hodnoty jako ceny a objemy ukládá jako celočíselné hodnoty, pro které se nejčastěji využívají 64bitové datové typy `int64_t` nebo `uint64_t` [16, 17]. Pokud burza definuje přesnost instrumentu například na čtyři desetinná místa, reálná cena se při vstupu do systému vynásobí konstantou 10 000. Cena 10,50 $ je tak v paměti bezpečně uložena a zpracovávána jako celé číslo 105 000. Tímto přístupem získává systém absolutní matematickou garanci, že při párování nedojde k nechtěnému zaokrouhlení a ztrátě přesnosti [17].

Kromě zajištění absolutní přesnosti přináší celočíselný přístup další kritický benefit, a to determinismus a spolehlivost výkonu. Ačkoliv se historicky uvádělo, že operace s pohyblivou řádovou čárkou jsou pomalejší, na moderních procesorech je jejich instrukční propustnost často srovnatelná s celočíselnou aritmetikou [19]. Zásadní problém však spočívá v hardwarové nepředvídatelnosti jednotek pro pohyblivou řádovou čárku (FPU). Pokud systém při výpočtech narazí na velmi malé hodnoty blížící se nule (subnormální či denormalizovaná čísla), hardware k jejich zpracování často vyžaduje asistenci mikrokódu procesoru, což může výrazně zpomalit provedení dané instrukce [20]. Takový propad výkonu je pro HFT systémy se submikrosekundovou latencí naprosto nepřijatelný. Použití celočíselné aritmetiky toto riziko eliminuje a garantuje stabilní latenci bez ohledu na stav trhu.

## 3.2 Paměťová hierarchie a mezipaměť (*cache*)

Vývoj výpočetní techniky v posledních desetiletích narazil na technologické omezení, které se v odborné literatuře označuje jako *memory wall*. Zatímco

moderní CPU dosahují vysokého výpočetního výkonu a díky superskalární architektuře dokážou vykonávat více instrukcí během jediného taktu, propustnost operační paměti (RAM) rostla nepoměrně pomaleji [21]. Pokud procesor požaduje data z hlavní paměti, musí na ně čekat řádově stovky procesorových cyklů. V praxi to znamená, že výpočetní jednotka není plně vytížena a celková propustnost systému klesá [21, 22]. Z tohoto důvodu je mezi procesorem a operační pamětí implementována víceúrovňová vyrovnávací paměť (L1, L2 a L3 *cache*). Tyto paměti jsou fyzicky integrovány blíže k výpočetnímu jádru, mají menší kapacitu, ale poskytují přístup s minimální latencí v jednotkách cyklů [23].

### 3.2.1 Prostorová lokalita a přenos dat

Klíčovým aspektem pro optimalizaci výkonu je mechanismus přenosu dat. Přístup do paměti neprobíhá po jednotlivých bajtech či proměnných. Veškerá data se mezi RAM a mezipamětí přenášejí v blocích fixní délky označovaných jako mezipaměťové linky (*cache lines*), které mají na moderních procesorových architekturách typicky velikost 64 B [21, 24]. Při požadavku na načtení konkrétní adresy procesor do mezipaměti automaticky načte i veškerá data, která s ní v paměti fyzicky sousedí. Současné procesory navíc využívají hardwarový *prefetcher*, který analyzuje vzorce přístupu do paměti. Pokud algoritmus zpracovává data sekvenčně, *prefetcher* tuto šablonu rozpozná a začne do mezipaměti načítat další bloky ještě před jejich explicitním vyžádáním programem [21, 23, 25].

Z těchto hardwarových vlastností vyplývá stěžejní požadavek pro návrh nízkolatenčních systémů v jazyce C++, a to zajištění prostorové lokality (*spatial locality*). Tento koncept vyžaduje, aby data, která jsou zpracovávána společně, byla v operační paměti uložena fyzicky bezprostředně vedle sebe [20]. Pouze za těchto podmínek procesor efektivně využije každou načtenou *cache line* a hardwarový prediktor dokáže spolehlivě odhadnout budoucí paměťové přístupy, čímž je zajištěn nepřetržitý přísun dat pro výpočetní jednotky [21, 23].

### 3.2.2 Asymptotická složitost vs. reálný výkon

V kontextu moderního hardwaru se teoretická asymptotická složitost algoritmů nemusí plně shodovat s reálnou výkonností softwaru. V teoretické informatice se typicky uvádí, že vkládání či odstraňování prvků uprostřed spojového seznamu (`std::list`) má časovou složitost $\mathcal{O}(1)$, zatímco u kontinuálního pole (`std::vector`) odpovídá složitosti $\mathcal{O}(n)$. V praktických aplikacích jsou však výsledky odlišné. Spojový seznam nebo standardní asociativní pole (`std::map`) alokují každý svůj uzel nesouvisle v dynamické paměti (*heap*). Při iteraci přes takovou strukturu procesor přistupuje k paměťovým adresám náhodně (*pointer chasing*), nedokáže využít mechanismus predikce a dochází k častým výpad-

kům mezipaměti (*cache misses*) [22, 23]. Naopak kontinuální datové struktury, jako jsou `std::vector` nebo `std::array`, výkonnostně překonávají spojové seznamy i při operacích s velkým počtem prvků, jelikož optimálně využívají paměťovou hierarchii a minimalizují celkovou paměťovou latenci [21, 22].

### 3.2.3 Datově orientovaný návrh

Jako reakce na tato hardwarová specifika se ve vývoji vysoce výkonných systémů, včetně burzovních párovacích jader, osvědčuje nasazení DOD [23, 26]. Na rozdíl od tradičního objektově orientovaného programování (OOP), které modeluje struktury podle logických vazeb entit reálného světa, se DOD primárně zaměřuje na to, jakým způsobem data procházejí procesorem a jeho mezipaměťmi [23, 26]. V praxi tento přístup například znamená přechod od klasických polí struktur (AoS) k dekompozici dat do plochých nezávislých struktur polí (SoA). Tím se minimalizuje využití nepřímých ukazatelů na kritické cestě a předchází se neefektivní fragmentaci při přístupu k paměti [21, 23].

## 3.3 Dynamická alokace a správa paměti

Při vývoji burzovního jádra v jazyce C++ představuje správa paměti kritickou oblast, kde se běžné programovací postupy dostávají do přímého rozporu s požadavky na deterministickou latenci. Ve standardním softwarovém inženýrství je dynamická alokace paměti pomocí operátoru `new` rutinní operací. Na úrovni operačního systému se však jedná o komplexní proces. Program musí požádat OS o přidělení bloku na haldě (*heap*), což vyžaduje synchronizaci pomocí zámků (*mutex*), aby se zabránilo souběžnému přístupu ke sdílenému alokátoru [20]. Důsledkem je vysoce nedeterministická latence. Zatímco jedna alokace může trvat desítky nanosekund, jiná může vlivem čekání na zámek trvat i mikrosekundy (*latency spikes*). Časté alokování navíc způsobuje fragmentaci paměti, což vede k postupné ztrátě prostorové lokality, častějším výpadkům mezipaměti a degradaci celkového výkonu systému [20].

Pro nízkolatenční systémy je nezbytně nutné vyhnout se dynamické alokaci na kritické cestě (*hot path*). Problematický není pouze operátor `new`, ale i dealokace pomocí `delete`. Ta v systémové knihovně `glibc` spouští stovky řádků kódu pro správu volné paměti, čímž s vysokou pravděpodobností vyřadí z instrukční mezipaměti užitečný kód a zásadně naruší plynulost výpočtu [27].

### 3.3.1 Prealokace a paměťové fondy

Řešením tohoto problému je prealokace, kdy se veškerá paměť vyžádá od operačního systému jednorázově při startu aplikace v rámci zahřívací fáze (*warm-up*). Během obchodování již systém na kritické cestě neinteraguje s alokátorem OS [8, 20]. K vytváření objektů za běhu se využívá návrhový vzor *object pool*

nebo technika *arena allocator* [20]. Párovací jádro si předem vytvoří pole instancí (např. volných objednávek), které následně pouze recykluje přepisem atributů, což zaručuje konstantní časovou složitost získání objektu $\mathcal{O}(1)$ [8].

Z tohoto důvodu se v kritické cestě nesmí používat ani uzlové kontejnery (*node containers*), jako jsou `std::map` či `std::list`, které alokují každý prvek samostatně na haldě. Tyto struktury jsou nahrazovány souvislými kontejnery typu `std::vector`, doplněnými o vlastní implementaci vyhledávání, čímž se efektivně obchází standardní alokátor a udržuje prostorová lokalita [28].

### 3.3.2 Minimalizace režie C++ abstrakcí

Tento striktní přístup je v kontrastu s doporučeními pro moderní C++, která preferují používání chytrých ukazatelů pro zajištění paměťové bezpečnosti. Využití `std::shared_ptr` je však na kritické cestě nevhodné, jelikož správa sdíleného vlastnictví vyžaduje dodatečnou alokaci řídicího bloku a provádění atomických operací nad čítačem referencí, což prokazatelně snižuje výpočetní výkon [29]. Architektura nízkolatenčních systémů se proto primárně spoléhá na prealokované datové struktury a holé ukazatele (*raw pointers*), které zajišťují přístup k datům bez jakékoliv skryté režie. V případech, kdy je nutné exaktně spravovat vlastnictví objektu, se využívá `std::unique_ptr` v kombinaci s vlastními lokálními alokátory, jež neinteragují s operačním systémem [29, 30].

## 3.4 Architektura procesoru a predikce skoků

Moderní procesory nečekají na dokončení jedné instrukce, než začnou zpracovávat další. Místo toho využívají mechanismu zvaného zřetězené zpracování (*instruction pipelining*). Díky němu je paralelně zpracováváno více instrukcí v různých fázích instrukčního cyklu. Zásadní problém pro tuto architekturu tvoří podmíněné skoky, typicky příkazy `if`. Jakmile procesor narazí na podmíněné větvení, nedokáže deterministicky určit, kterými instrukcemi by měl instrukční linku (*pipeline*) dále naplnit. V tento moment se aktivuje hardwarový prediktor skoků (*branch predictor*), který se na základě předchozí historie pokusí odhadnout další tok instrukcí, což procesoru umožní začít zpracovávat příslušné instrukce spekulativně [20, 31]. Pokud se však prediktor zmýlí a zvolí větev, která se ve skutečnosti neuplatní (*branch misprediction*), procesor musí veškerou spekulativní práci zahodit, vyprázdnit *pipeline* a začít načítat instrukce ze správné větve. Tato penalizace stojí systém desítky procesorových cyklů, což pro nízkolatenční systémy znamená kritické zdržení [20, 31].

### 3.4.1 Bezskokové programování a volba algoritmů

Výše zmíněné chování procesoru v praxi ukazuje, že teoretická asymptotická složitost algoritmu nemusí odpovídat reálné výpočetní propustnosti. Při zpra-

cování seřazených dat se standardně využívají algoritmy založené na binárním vyhledávání (např. `std::lower_bound`) s časovou složitostí $\mathcal{O}(\log n)$. Analýzy burzovních limitních knih ovšem prokazují, že půlení intervalu generuje podmíněné skoky s pravděpodobností splnění blízkou 50 %. V takovém případě hardwarový prediktor selhává a procesor vykazuje vysokou míru chybných predikcí větvení [28]. Na kratších paměťových úsecích proto lineární vyhledávání dosahuje vyššího výkonu. Ačkoliv vyžaduje více operací, umožňuje implementaci bez podmíněných skoků (*branchless programming*). Logické podmínky jsou v kódu nahrazeny aritmetickými operacemi či bitovými maskami. Tento přístup záměrně aplikuje redundantní výpočty, čímž eliminuje riziko vyprázdnění *pipeline* [31]. Tato snaha o maximální soulad softwaru s fyzickou architekturou hardwaru je v oboru známá jako *mechanical sympathy*.

### 3.4.2 Instrukční mezipaměť a rozložení kódu

Kromě datové mezipaměti (D-cache) ovlivňuje latenci systému také instrukční mezipaměť (I-cache), která slouží k ukládání strojového kódu. Za účelem efektivního využití omezené kapacity I-cache na kritické cestě se využívají nápovědy pro kompilátor, jako je makro `__builtin_expect` či atributy `[[likely]]` a `[[unlikely]]` zavedené ve standardu C++20 [20, 31]. Je nezbytné zdůraznit, že tyto značky primárně neovlivňují hardwarový prediktor skoků [27]. Jejich hlavní funkcí je instruovat překladač k optimalizaci rozložení kódu (*code layout*) tak, aby byly chybové nebo málo pravděpodobné větve fyzicky umístěny mimo hlavní sekvenci instrukcí [20]. Kritická cesta si díky tomu zachovává paměťovou kontinuitu, což minimalizuje výpadky v I-cache a zamezuje jejímu zahlcení instrukcemi pro zřídka prováděné operace, jako je logování či ošetření výjimek [27].

### 3.4.3 Zahřívání mezipaměti (*cache warming*)

Hardwarový prediktor skoků funguje výhradně na základě nedávné historie [31]. Kritická cesta párovacího jádra se však v reálném provozu může plně aktivovat jen velmi zřídka, například pouze při náhlé tržní události. Během období nečinnosti prediktor svou historii ztratí, I-cache se přepíše jiným kódem a při skutečném tržním příkazu pak s vysokou pravděpodobností dojde k chybné predikci a skokovému nárůstu latence. Standardním řešením tohoto problému v HFT systémech je umělé zahřívání mezipaměti (*cache warming*) [20, 27]. Do párovacího jádra jsou nepřetržitě posílány fiktivní objednávky (*dummy orders*), které projdou celou kritickou cestou algoritmů totožně jako reálná data. Aby nedošlo k ovlivnění stavu trhu, jsou veškeré výstupy těchto operací zahozeny na výstupní bráně dříve, než se rozešlou účastníkům trhu. Hardwarový prediktor je tímto způsobem udržován v natrénovaném stavu a instrukční i datová mezipaměť zůstávají neustále připravené na okamžitou reakci [27].

## 3.5 Interakce s OS Linux

Operační systém Linux je z podstaty svého návrhu koncipován jako víceúlohové prostředí (*multitasking*), jehož hlavním cílem je spravedlivé a bezpečné rozdělování hardwarových prostředků mezi všechny běžící procesy [32]. V kontextu HFT, kde je striktně vyžadována deterministická submikrosekundová latence, se však tento standardní přístup stává kritickým omezujícím faktorem. Párovací jádro proto musí interakci s operačním systémem na své kritické cestě minimalizovat, nebo ji dokonce zcela eliminovat [33].

### 3.5.1 Minimalizace systémových volání

Jakékoliv přerušení výpočtu a odevzdání řízení jádru operačního systému (systémové volání neboli *syscall*) je z hlediska výkonu značně nákladná operace. Systémová volání mají na kritické cestě nízkolatenčních aplikací zásadní vliv na výkon [27]. Běžné I/O operace, jako je zápis na disk, volání síťových funkcí nebo dotazování stavu soketů, nutí procesor přepnout kontext, což vede ke ztrátě dat v mezipaměti a k narušení běhu instrukční *pipeline* [27]. Z tohoto důvodu musí být kritická cesta (*hot path*) párovacího jádra navržena tak, aby v optimálním případě neobsahovala žádná systémová volání [27].

### 3.5.2 Afinita a izolace vláken

Operační systém standardně využívá plánovač, který vlákna přesouvá mezi různými jádry procesoru za účelem vyvážení zátěže [32]. Každé takové přepnutí kontextu trvá mikrosekundy a způsobuje rozsáhlé výpadky v L1 a L2 mezipaměti [5, 34]. Řešením je afinita vláken (*thread affinity* či *CPU pinning*). Pomocí nástrojů operačního systému (např. `taskset`) lze explicitně dedikovat klíčová vlákna párovacího jádra konkrétním fyzickým jádrům procesoru [5, 20]. Tím se operačnímu systému zakáže na těchto jádrech spouštět jakékoliv jiné procesy a eliminuje se riziko vynuceného přepnutí kontextu. Vlákno párovacího jádra tak běží ve vlastním nekonečném cyklu zcela nerušeně a bez kontextových přepnutí [5]. Vliv plánovače a izolace jader je dále rozebrán v podkapitole 4.4.4.

### 3.5.3 Správa virtuální paměti

Procesory využívají pro rychlý překlad virtuálních adres paměti na fyzické speciální vyrovnávací paměť překladu adres (TLB). Pokud proces přistupuje k velkým objemům paměti rozděleným do standardních 4KB stránek (*pages*), kapacita TLB se rychle vyčerpá. Každý výpadek (*TLB miss*) nutí procesor procházet stránkovací tabulky OS, což dočasně zastaví výpočet [20]. Pro minimalizaci tohoto jevu využívají HFT systémy obrovské stránky (*huge pages*), které mají v Linuxu velikost nejčastěji 2 MB, případně až 1 GB [20]. Například 200 MB paměti tak namísto 51 200 standardních stránek obsadí pouhých 100

obrovských stránek, což výrazně snižuje zátěž na TLB a zrychluje adresaci. V nízkolatenčních aplikacích je navíc kritické využívat pouze explicitně alokované obrovské stránky (EHP), nikoliv transparentní obrovské stránky (THP). Explicitní stránky jsou totiž rezervovány předem a nepodléhají odkládání na disk ani defragmentaci, čímž se zcela eliminuje nedeterministická doba adresace [20]. Další techniky pro ochranu paměti před zásahy OS, včetně uzamykání fyzické paměti pomocí volání `mlock`, jsou detailně popsány v podkapitole 4.4.1.

Kapitola 4

# Metodika měření a testování párovacího jádra

*Tato kapitola popisuje metodiku a prostředí pro vývoj, profilování a zátěžové testování párovacího jádra. Pro dosažení deterministické submikrosekundové latence je klíčová exaktní specifikace použitého hardwaru, softwarových nástrojů a testovacích dat. Zvláštní pozornost je věnována postupům pro eliminaci výkonnostního zkreslení, které zajišťují maximální přesnost a opakovatelnost naměřených výsledků.*

## 4.1 Testovací prostředí

Kritickým předpokladem pro validní vyhodnocení výkonnostních metrik párovacího jádra je definice stabilního a deterministického testovacího prostředí. Vzhledem k tomu, že se práce zaměřuje na optimalizace na hranici hardwarových limitů, nelze interpretovat naměřené metriky bez znalosti specifikace použité výpočetní platformy. Následující podkapitoly proto vymezují hardwarovou i softwarovou infrastrukturu, která tvoří základ pro veškerá měření a budoucí reprodukovatelnost výsledků.

### 4.1.1 Hardwarové prostředí

Veškerá výkonnostní měření, zátěžové testy a profilování zdrojového kódu probíhají na dedikované pracovní stanici s architekturou x86_64. Pro zajištění reprodukovatelnosti naměřených výsledků je specifikace použitého hardwaru detailně shrnuta v tabulce 4.1. Znalost těchto parametrů je nezbytná, neboť kapacita procesorových mezipamětí a propustnost operační paměti určují absolutní fyzikální limity pro dosažitelnou latenci párovacího jádra.

**Tabulka 4.1** Hardwarová specifikace testovacího prostředí

| Komponenta / Parametr | Specifikace |
|---|---|
| Procesor (CPU) | AMD Ryzen 5 5625U |
| Fyzická / logická jádra | 6 / 12 |
| Základní frekvence | 2,27 GHz |
| Maximální frekvence | 4,30 GHz |
| L1 datová mezipaměť (L1d) | 32 KiB na jádro |
| L1 instrukční mezipaměť (L1i) | 32 KiB na jádro |
| L2 mezipaměť | 512 KiB na jádro |
| L3 mezipaměť | 16 MiB (sdílená) |
| Operační paměť (RAM) | 16 GB LPDDR4, 4266 MT/s |

Z uvedené hardwarové specifikace vyplývá, že nejrychlejší instrukční paměť (L1i) disponuje kapacitou pouze 32 KiB na jedno výpočetní jádro. Tento striktní limit hraje stěžejní roli při analýze vlivu optimalizace v době linkování (LTO), neboť agresivní vkládání funkcí (*inlining*) překladačem může vést k jejímu zahlcení a následné degradaci výkonu vlivem instrukčních výpadků mezipaměti. Sdílená L3 mezipaměť o kapacitě 16 MiB naopak představuje limitující faktor pro maximální efektivní velikost prealokovaných paměťových fondů. Udržení kritických datových struktur v tomto prostoru je klíčové pro eliminaci přístupů s vysokou latencí do hlavní operační paměti.

### 4.1.2 Softwarové prostředí

Párovací jádro je vyvíjeno a testováno v prostředí operačního systému Linux. Kompletní přehled použitých softwarových nástrojů, překladačů a testovacích frameworků je shrnut v tabulce 4.2.

**Tabulka 4.2** Softwarová specifikace a testovací nástroje

| Software / Nástroj | Verze / Specifikace |
|---|---|
| Distribuce OS | Linux Mint 22.3 |
| Verze jádra | 6.17.0-19-generic |
| Standard jazyka | C++23 |
| Překladač GCC | GCC 15.2.0 |
| Překladač Clang | Clang 18.1.3 |
| Sestavovací systém | CMake 3.28.3 |
| Testovací framework | Google Test 1.17.0 |
| Benchmarkový framework | Google Benchmark 1.9.5 |
| Profilovací nástroj | Linux perf 6.17.13 |

Volba linuxového prostředí je pro výkonné párovací jádro nezbytná, neboť poskytuje přístup k nízkoúrovňovým systémovým voláním a umožňuje

absolutní kontrolu nad chováním hardwaru. Pro dosažení submikrosekundové latence je kritická především možnost exaktní správy fyzické paměti a striktní manipulace s procesorovým plánovačem. Konkrétní techniky, které tyto mechanismy využívají k eliminaci výkonnostního zkreslení a dosažení determinismu, jsou detailně rozebrány v navazující sekci 4.4.

K verifikaci správnosti párovacích algoritmů a exaktnímu měření nanosekundových metrik slouží knihovny GoogleTest a Google Benchmark. Ty vhodně doplňuje systémový profiler `perf`, který umožňuje hlubší analýzu běhu programu na úrovni jednotlivých instrukcí. Metodika testování rovněž zahrnuje výkonnostní srovnání generovaného strojového kódu mezi překladači GCC a Clang.

Projekt je sestavován v souladu se standardem jazyka C++23. K dosažení maximálního výkonu a zároveň k zachování plné kompatibility s profilerem `perf` je produkční sestavení (*release*) konfigurováno pomocí následujících kompilačních příznaků:

`-O3`
: Aktivuje nejvyšší úroveň standardních optimalizací překladače zaměřených na rychlost běhu programu.

`-march=native`
: Instruuje překladač, aby generoval strojový kód optimalizovaný přímo pro architekturu přítomného procesoru. Umožňuje využití pokročilých vektorových instrukcí (SIMD), jako je například instrukční sada AVX2. Zpracování více datových položek v jediném cyklu je kritické pro rychlé prohledávání a kopírování paměťových bloků.

`-flto`
: Zapíná optimalizaci LTO. Umožňuje analyzovat celý program globálně a provádět vkládání funkcí napříč zdrojovými soubory, čímž eliminuje režii na jejich volání.

`-g`
: Zajišťuje generování ladicích symbolů, které profilovači umožňují přesně mapovat zachycené strojové instrukce na konkrétní řádky C++ zdrojového kódu.

`-fno-omit-frame-pointer`
: Zabraňuje odstranění ukazatelů na zásobníkový rámec. Bez těchto ukazatelů by nástroj `perf` nedokázal zrekonstruovat korektní hierarchii volání (*call graph*).

## 4.2 Charakteristika testovacích dat

Pro exaktní měření softwarového výkonu hrají klíčovou roli vstupní data. Je nezbytné, aby testovací datové sady co nejvěrněji odrážely charakter zpráv, se

kterými systém pracuje v produkčním prostředí. V kontextu vysokofrekvenčního obchodování je naivní generování náhodných dat nevhodné. Náhodná distribuce nerespektuje reálnou mikrostrukturu trhu a uměle zkresluje efektivitu, s jakou jádro data zpracovává, a to v kladném i záporném smyslu. Z tohoto důvodu jsou testovací data pro potřeby měření rozdělena do dvou nezávislých kategorií.

### 4.2.1 Reálná tržní data

Jako referenční bod pro stanovení výchozí výkonnostní metriky jádra a k verifikaci deterministického chování párovacích algoritmů práce využívá datasety z akademického portálu LOBSTER (*Limit Order Book System – The Efficient Reconstructor*). Tento portál poskytuje vysoce kvalitní rekonstrukci tržních událostí a vývoje limitní knihy na americké burze NASDAQ. Zdrojová data vycházejí z proprietárního datového toku NASDAQ Historical TotalView-ITCH, který zachycuje veškerou aktivitu s nanosekundovou přesností a bez jakékoliv agregace.

Pro potřeby testování jsou využity vzorkové datové sady z obchodního dne 21. 6. 2012. Měření probíhají na vysoce likvidních finančních instrumentech, jako jsou Apple (AAPL), Google (GOOG) či Microsoft (MSFT).

Portál nabízí tyto sady v několika úrovních hloubky trhu, nejčastěji 1, 5 a 10. Zvolená úroveň přímo určuje dimenzionalitu obou souborů. V souboru `orderbook` definuje šířku matice (čtyři sloupce pro každou úroveň), zatímco u souboru `message` funguje jako filtr propouštějící události, které mění stav knihy v požadované hloubce. Tím je garantováno, že oba soubory obsahují za všech okolností identický počet řádků, což je klíčové pro verifikaci správnosti párovacích algoritmů.

Oba datové formáty jsou podrobněji specifikovány níže.

#### Soubor se zprávami

Soubor zpráv tvoří chronologický záznam tržních událostí, které vedly ke změně stavu limitní knihy. Každý řádek souboru odpovídá jedné zprávě z burzy a obsahuje následující atributy:

**Čas (*time*)**
: Časová značka události vyjádřená v sekundách od půlnoci s přesností na nanosekundy.

**Typ události (*event type*)**
: Číselný indikátor tržní akce. Formát LOBSTER definuje následující typy událostí:

- `1`: Vložení nové limitní objednávky (*submission*).
- `2`: Částečné zrušení objednávky (*cancellation*).

- `3`: Úplné odstranění objednávky (*deletion*).
- `4`: Exekuce viditelné objednávky (*execution*).
- `5`: Exekuce skryté objednávky (*hidden execution*).
- `6`: Indikace křížového (aukčního) obchodu (*cross trade*).
- `7`: Zpráva o pozastavení nebo obnovení obchodování (*trading halt*).

**ID objednávky (*order id*)**
Unikátní referenční číslo přidělené burzou pro jednoznačnou identifikaci v limitní knize objednávek.

**Velikost (*size*)**
Počet obchodovaných akcií.

**Cena (*price*)**
Cena za akcii vyjádřená v dolarech a vynásobená 10 000. Tato konverze je pro implementaci jádra stěžejní, neboť umožňuje využití rychlé celočíselné aritmetiky namísto operací s pohyblivou řádovou čárkou.

**Směr (*direction*)**
Indikátor strany trhu (`-1` pro prodejní stranu *ask*, `1` pro nákupní stranu *bid*).

**Tabulka 4.3** Struktura souboru zpráv ve formátu LOBSTER (`message`). Upraveno podle LOBSTER [35].

| Time (sec) | Event Type | Order ID | Size | Price | Direction |
|---|---|---|---|---|---|
| ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ |
| 34713.685155243 | 1 | 206833312 | 100 | 118600 | -1 |
| 34714.133632201 | 3 | 206833312 | 100 | 118600 | -1 |
| 34715.521344122 | 4 | 183749211 | 50 | 118500 | 1 |
| 34719.193632201 | 7 | 0 | 0 | 0 | -1 |
| ⋮ | ⋮ | ⋮ | ⋮ | ⋮ | ⋮ |

## Soubor stavů limitní knihy

Soubor stavů limitní knihy tvoří maticovou reprezentaci dostupné likvidity (nabídky a poptávky) v čase, reflektující chronologický příchod zpráv o tržních událostech. Pro účely vývoje párovacího jádra slouží primárně jako reference pro otestování funkční správnosti párovacích algoritmů.

Každá sledovaná cenová úroveň (hloubka trhu) je v souboru reprezentována čtveřicí hodnot pro úroveň $n$, kde $n \in \{1, 2, \ldots, L\}$ a $L$ udává maximální sledovanou hloubku limitní knihy.

**Ask Price *n***
Cena v $n$-té cenové hladině na prodejní straně.

**Ask Size *n***
Dostupné množství v $n$-té cenové hladině na prodejní straně.

**Bid Price *n***
Cena v $n$-té cenové hladině na nákupní straně.

**Bid Size *n***
Dostupné množství v $n$-té cenové hladině na nákupní straně.

**Tabulka 4.4** Struktura souboru stavů limitní knihy ve formátu LOBSTER (`orderbook`). Upraveno podle LOBSTER [35].

| Ask Price 1 | Ask Size 1 | Bid Price 1 | Bid Size 1 | ... |
|---|---|---|---|---|
| ⋮ | ⋮ | ⋮ | ⋮ | ⋮ |
| 1186600 | 9484 | 1186500 | 8800 | ... |
| 1186600 | 9384 | 1186500 | 8800 | ... |
| ⋮ | ⋮ | ⋮ | ⋮ | ⋱ |

Datová sada o hloubce 10 úrovní tedy tvoří matici o 40 sloupcích (seřazených od nejlepších cen po nejhorší). Pro korektní verifikaci jádra je nutné zohlednit dvě stěžejní vlastnosti tohoto formátu:

**Řádková synchronizace**
Oba soubory na sebe přesně navazují řádek po řádku.

**Výplňové hodnoty**
Neobsazené cenové hladiny vyplňují umělé hodnoty (`9999999999` pro *ask*, `-9999999999` pro *bid*), které musí framework při porovnávání ignorovat.

### 4.2.2 Syntetická data

Ačkoliv reálná data z portálu LOBSTER poskytují nezbytný referenční bod pro verifikaci funkční správnosti i pro základní výkonnostní srovnání, jejich nevýhodou je pro účely extrémního zátěžového testování omezená velikost. Poskytnuté denní vzorky obsahují řádově statisíce tržních událostí. Vzhledem k vysoké propustnosti vyvíjeného párovacího jádra je takový objem dat zpracován ve zlomcích sekundy. Tato velmi krátká doba běhu znesnadňuje s dostatečnou statistickou spolehlivostí měřit výkon a objektivně vyhodnocovat dopad aplikovaných nízkoúrovňových optimalizací. Výsledky takto krátkých měření na malém vzorku navíc mohou být zkresleny šumem operačního systému a fázemi zahřívání procesorové mezipaměti (*cache warm-up*).

Z tohoto důvodu je pro účely výkonnostního profilování nezbytné implementovat vlastní generátor syntetických dat. Jeho primárním cílem je schopnost vyprodukovat volitelné množství (až desítky milionů) tržních událostí, které udrží jádro v plném provozu po dobu několika sekund. Aby však takové měření mělo relevantní vypovídající hodnotu a netestovalo jádro na nesmyslných vstupech, musí generátor co nejvěrněji simulovat chování reálného finančního trhu. To klade na návrh generátoru následující požadavky:

**Cenová distribuce (*price distribution*)**
Rozdělení generovaných cen limitních objednávek nesmí být napříč cenovým spektrem rovnoměrné. Podle vzoru reálných trhů se musí koncentrovat převážně v těsné blízkosti středu mezi aktuální nejlepší nabídkou a poptávkou (*mid-price*).

**Životnost objednávek (*order lifespan*)**
Algoritmus musí simulovat aktivní rušení objednávek v čase. Bez tohoto mechanismu by docházelo k nereálnému a teoreticky nekonečnému hromadění zpráv v limitní knize, což by degradovalo výkon paměťových a vyhledávacích struktur.

**Exekuční poměr (*fill rate*)**
Generátor musí simulovat realistický procentuální poměr mezi pasivním vkládáním objednávek (poskytováním likvidity) a agresivními tržními příkazy, jež vedou k reálné exekuci a likviditu z knihy odčerpávají.

## 4.3 Nástroje pro testování a měření výkonu

Pro zajištění maximální spolehlivosti a vysokého výkonu párovacího jádra je nutné zvolit odpovídající sadu vývojářských nástrojů. Zatímco funkční správnost je kritická pro přesné provádění obchodů, výkonnostní měření a profilování jsou nezbytná pro detekci úzkých hrdel (*bottlenecks*) a minimalizaci latence. K těmto účelům se využívají následující standardní nástroje z ekosystému jazyka C++ a operačního systému Linux.

### 4.3.1 GoogleTest

Ověření funkční správnosti a striktního determinismu jádra zajišťuje open-source framework GoogleTest [36]. Sady jednotkových a integračních testů slouží k ověření, že párovací logika a prioritizace objednávek fungují bezchybně i v okrajových případech. Framework se v rámci této práce intenzivně využívá právě ve spojení s reálnými daty z portálu LOBSTER. Automatizované integrační testy krok za krokem aplikují příchozí tržní zprávy a porovnávají výsledný vnitřní stav limitní knihy s referenčním `orderbook` souborem. Jakoukoliv případnou odchylku v ceně či množství framework okamžitě zachytí a zaznamená.

Jelikož jakékoliv zásahy do kódu za účelem optimalizace nesou riziko zanesení nových chyb, uplatňuje se při vývoji striktně regresní testování. Celá sada automatizovaných testů se proto spouští opakovaně po každé aplikované úpravě jádra.

### 4.3.2 Google Benchmark

Pro detailní měření výkonu na úrovni jednotlivých funkcí (*microbenchmarking*) se využívá knihovna Google Benchmark [37]. V kontextu vysoce výkonného párovacího jádra je tento nástroj klíčový pro izolované testování rychlosti kritických operací, jako je vložení nové objednávky, zrušení objednávky nebo vyhledání odpovídající protistrany. Knihovna automaticky řídí počet iterací nutných pro dosažení statisticky významných výsledků a prostřednictvím svých vestavěných funkcí (např. `benchmark::DoNotOptimize`) poskytuje nezbytnou ochranu proti agresivním optimalizacím překladače. Zabraňuje tak mimo jiné odstranění mrtvého kódu (DCE), které by jinak mohlo výsledky měření zcela znehodnotit.

Podobně jako u regresních testů funkčnosti se tyto benchmarky spouštějí po každé úpravě jádra. Díky tomu lze systematicky sledovat vliv implementovaných změn na celkový výkon a proaktivně předcházet jeho nechtěnému propadu (*performance regression*). Naměřené výsledky navíc slouží jako objektivní metrika při hledání implementačních kompromisů, případně jako podklad pro zavrhnutí takových úprav, jejichž výkonnostní přínos je zanedbatelný.

### 4.3.3 Linux `perf`

Zatímco Google Benchmark slouží k izolovaným měřením částí kódu, pro komplexní analýzu celého programu se využívá nástroj `perf` [38], integrovaný přímo v linuxovém jádře. Tento výkonný profiler umožňuje hloubkovou inspekci běhu aplikace na úrovni hardwarových instrukcí procesoru s naprosto minimální výpočetní režií (*overhead*).

Nástroj dokáže přistupovat k interním hardwarovým čítačům procesoru a poskytovat detailní statistiky o průběhu zpracování. Zaměřuje se zejména na metriky, jako je počet výpadků mezipaměti (*cache misses*), chybných předvídání skoků (*branch mispredictions*) nebo výpadků stránek paměti na úrovni operačního systému (*page faults*). V rámci této práce se `perf` využívá při testování pod plnou zátěží k identifikaci nejvíce zatížených úseků kódu (*hotspots*). Včasná detekce těchto hardwarových a softwarových penalizací je pro pochopení chování aplikace a dosažení nízkých latencí u vysokofrekvenčních systémů naprosto stěžejní. Pro intuitivní vizualizaci naměřených dat a rychlou orientaci v hierarchii volání funkcí (*call stack*) se vygenerované výstupy profilování následně převádějí do podoby plamenných grafů (*flame graphs*).

## 4.4 Zajištění determinismu měření

Měření latencí v řádu desítek nanosekund tvoří v prostředí moderních operačních systémů s preemptivním multitaskingem značný problém. Operační systém a hardwarové optimalizace procesoru vnášejí do běhu programu nedeterminismus a výkonnostní šum. Aby měly výsledky z nástrojů jako Google Benchmark a `perf` objektivní vypovídající hodnotu, je nutné před každým měřením zajistit stabilní a deterministické testovací prostředí. Toho lze docílit pomocí následujících technik.

### 4.4.1 Uzamykání fyzické paměti

Standardní správa paměti v operačním systému Linux využívá techniku stránkování. Pokud proces přistoupí k paměti, která není aktuálně v RAM, dojde k výpadku stránky (*page fault*) a jádro OS musí data načíst z úložiště, což v kontextu vysokofrekvenčního obchodování způsobuje kritické zpoždění v řádu mikrosekund.

K zamezení tohoto nežádoucího jevu během výkonnostního měření se využívá systémové volání `mlock` [39], které operačnímu systému striktně zakazuje přesunout alokovanou paměť měřicího procesu do odkládacího prostoru na disku (*swap*).

### 4.4.2 Deaktivace randomizace paměti

Operační systém Linux standardně využívá bezpečnostní techniku randomizace rozvržení adresního prostoru (ASLR), která při každém spuštění programu náhodně mění počáteční adresy datových struktur v paměti. V kontextu benchmarkingu v C++ tato randomizace narušuje konzistentní zarovnání dat do bloků vyrovnávací paměti (*cache lines*) a vnáší do výsledků šum [40].

Pro dosažení striktní reprodukovatelnosti výsledků napříč vícero spuštěními zátěžových testů je nezbytné tuto funkci dočasně deaktivovat. Tohoto stavu lze docílit buď modifikací systémového parametru `randomize_va_space`, případně přímým využitím funkce v rámci knihovny Google Benchmark – `benchmark::MaybeReenterWithoutASLR`.

### 4.4.3 Stabilizace frekvence a konfigurace CPU

Moderní procesory za účelem co nejefektivnějšího řízení spotřeby energie a tepelného vyzařování využívají techniky dynamického škálování frekvence a při nečinnosti přecházejí do hlubokých úsporných stavů (*C-states*) [41]. Výkyvy frekvencí a případné probouzení jádra z těchto stavů však mají kritický dopad na výsledky mikrosekundových měření, do kterých vnášejí náhlé latence a vysoký nedeterminismus. U mobilních procesorů, jako je referenční testovací model AMD Ryzen 5 5625U, je toto řízení výkonu obzvláště agresivní

z důvodu striktnějších termálních a energetických limitů. Pro účely exaktního měření je proto nezbytné dynamické hardwarové optimalizace (zde reprezentované technologií Precision Boost 2 [42]) deaktivovat, zamezit přechodům do úsporných stavů a nastavit politiku operačního systému (*CPU governor*) tak, aby procesor poskytoval maximální a stabilní frekvenci [43].

Dalším faktorem, který může mít vliv na stabilitu výsledků měření, je souběžné zpracování vláken (SMT), jehož konfigurace se provádí v rozhraní firmwaru (UEFI). Referenční procesor AMD Ryzen 5 5625U standardně rozděluje svých 6 fyzických jader na 12 logických vláken. Podle technické dokumentace technologie SMT pro architekturu Zen [44] sdílejí dvě logická vlákna v rámci jednoho fyzického jádra kritické prostředky, zejména vysoce propustnou L1 a L2 mezipaměť. Teoreticky může asynchronní běh jiných systémových procesů na sousedním logickém vlákně vést k nežádoucímu vytlačování dat z mezipaměti (*cache eviction*), což by negativně ovlivnilo latenci párovacího jádra. Jelikož však reálný dopad SMT na výkon nízkolatenčních aplikací závisí na konkrétní implementaci i charakteru zátěže a není dopředu znám, bude jeho vliv v rámci této práce experimentálně ověřen. Deaktivace SMT v UEFI tak slouží jako jeden z konfiguračních scénářů pro dosažení maximální hardwarové izolace a determinismu.

### 4.4.4 Izolace vlákna na úrovni OS

Standardní plánovač úloh operačního systému tvoří i přes své pokročilé algoritmy významný zdroj výkonnostního šumu. Historicky byl v linuxových jádrech využíván zcela spravedlivý plánovač (CFS) [32], který byl však v moderních verzích jádra (od verze 6.6, včetně zde referenční 6.17) nahrazen novým algoritmem s nejbližším možným virtuálním termínem (EEVDF) [45]. Ačkoliv je primárním úkolem těchto plánovačů spravedlivě rozdělovat procesorový čas mezi všechny běžící procesy [32], pro nízkolatenční aplikace je tento sdílený přístup kritický. Nevyhnutelně totiž vede k pravidelnému přerušování běhu aplikací a k přepínání kontextu (*context switch*). Softwarová režie přepnutí kontextu se pohybuje v jednotkách mikrosekund [34] (což bylo na testovacím stroji empiricky ověřeno nástrojem `perf` s průměrným výsledkem 7 $\mu$s). Kritickým důsledkem pro párovací systém je však absolutní ztráta lokality dat. Když plánovač dočasně přidělí procesorové jádro jinému procesu, dojde k okamžitému vytlačení pečlivě načtených datových struktur limitní knihy z L1 a L2 mezipaměti zpět do pomalé hlavní paměti RAM [20].

Pro dosažení deterministické latence je proto nezbytné párovacímu jádru vyhradit exkluzivní výpočetní prostředky a zcela jej chránit před zásahy plánovače. Toho lze v moderních linuxových distribucích docílit parametrem jádra `isolcpus`. Tento parametr při startu operačního systému odebírá specifikovaná jádra z fondu dostupných prostředků, čímž plánovači striktně zakazuje na ně umisťovat jakékoliv běžné procesy z uživatelského prostoru. Následným krokem je explicitní svázání běhu testovacího vlákna s tímto izolovaným jádrem,

nastavení procesorové afinity (*CPU pinning*). Tohoto stavu se dosahuje buď na úrovni příkazové řádky nástrojem `taskset`, nebo programově přímo uvnitř zdrojového kódu pomocí systémového volání `pthread_setaffinity_np` [46].

### 4.4.5 Exaktní hardwarové měření času

V prostředí nízkolatenčních systémů nedostačují k přesnému profilování kódu standardní softwarová rozhraní. Běžně využívané funkce operačního systému, jako je POSIXové volání `clock_gettime()` nebo jeho abstrakce `std::chrono::high_resolution_clock` ve standardní knihovně jazyka C++, dosahují i při optimalizovaném průchodu přes virtuální dynamický sdílený objekt (vDSO) režie v řádu desítek nanosekund [47]. Při měření operací v limitní knize, jejichž trvání se pohybuje ve stejném řádu, by takový nástroj zanesl do výsledků nezanedbatelnou chybu měření.

Pro účely exaktního měření se proto využívá přímé čtení 64bitového registru čítače časových značek (TSC). Běžnou instrukci `rdtsc` by mohl procesor kvůli exekuci mimo pořadí (*out-of-order execution*) libovolně přesunout a měření zcela znehodnotit. K zajištění determinismu se proto používá instrukce `rdtscp` (v C++ `__rdtscp`). Ta je hardwarově serializační – před přečtením vyčká na dokončení všech předchozích instrukcí v programovém toku [48].

Díky podpoře *invariant TSC* u procesorů Zen navíc registr tiká konstantní frekvencí nezávisle na aktuálním napájecím stavu procesoru (*P-states*, *C-states*). Jelikož je měřicí vlákno pomocí afinity izolováno na jediném jádře, je plně eliminován problém desynchronizace časovačů napříč procesorem (*TSC skew*). Získaný rozdíl cyklů lze následně snadno převést na nanosekundy pomocí fixní základní frekvence procesoru [49].

## 4.5 Metodika srovnání s referenčními řešeními

Implementované párovací jádro je s referenčními řešeními srovnáváno na základě výsledků běhů nad historickými tržními daty z platformy LOBSTER.

Srovnání probíhá podle výkonnostních metrik definovaných v podkapitole 5.3.2. Všechny porovnávané systémy podporují shodné párovací algoritmy a pravidla priority, čímž je zaručena korektnost a spravedlnost testování. Získané výkonnostní rozdíly tak plynou výhradně z odlišných implementačních a architektonických rozhodnutí.

Jelikož referenční řešení vykazují určité vstupně-výstupní a architektonické odlišnosti, je pro objektivní srovnání nezbytné provést v jejich zdrojových kódech dílčí úpravy, aby je bylo možné spouštět nad identickými datasety a za stejných podmínek. Konkrétní úpravy zahrnují především umožnění čtení LOBSTER dat a sjednocení formátu výpisu měřených metrik. V případě řešení od Piyushe Kumara je navíc mírně upraven tok řízení tak, aby systém předával objednávky k párování přímo a sekvenčně. Tím dochází k izolaci párovacího

algoritmu od jeho původní síťové infrastruktury, což zajišťuje maximálně korektní a přímé srovnání čistého výkonu jader.

Dále je dodrženo, že běh všech referenčních implementací probíhá ve zcela identickém hardwarovém a softwarovém prostředí, jaké je definováno pro předkládané párovací jádro této práce.

# Kapitola 5
# Návrh řešení

*Tato kapitola se věnuje návrhu softwarové architektury párovacího jádra. První část kapitoly vymezuje rozsah implementace a jasně stanovuje funkční i nefunkční požadavky na systém. Funkční požadavky definují formát vstupních a výstupních zpráv, pravidla pro správu limitní knihy a logiku párování objednávek. Nefunkční požadavky určují výkonnostní limity a meze asymptotické časové složitosti. Následně je představen výchozí návrh řešení. Ačkoliv tento návrh splňuje zadání z hlediska funkční správnosti, z výkonnostního hlediska se záměrně jedná o neefektivní řešení, které bude v následující kapitole iterativně optimalizováno. Závěrečná část specifikuje testovací scénáře a metriky, které jsou nezbytné pro objektivní vyhodnocení výkonnostního posunu.*

## 5.1 Vymezení rozsahu a požadavky na systém

Před návrhem architektury je potřeba definovat funkční a nefunkční požadavky na párovací jádro. Tyto požadavky stanovují jak burzovní logiku, tak výkonnostní limity, kterých musí finální optimalizovaný systém dosahovat.

### 5.1.1 Funkční požadavky

Funkční požadavky definují burzovní logiku a operace, které jádro podporuje. Pro účely této práce je rozsah záměrně omezen na základní mechanismy. Cílem tohoto omezení je zabránit zkreslení výkonnostních měření dodatečnou obchodní logikou a současně zajistit, aby funkčnost jádra odpovídala referenčním řešením, se kterými je porovnáváno. Navržená architektura musí být kompatibilní s datovým formátem platformy LOBSTER.

**Zpracování tržních událostí**
Systém musí umět přijímat a zpracovávat základní typy zpráv, konkrétně vložení (1 – *submission*), částečné zrušení (2 – *cancellation*) a úplné odstranění (3 – *deletion*). Zprávy o exekuci (4 a 5 – *execution*) jádro na vstupu nepřijímá. Pro účely ověření správnosti párování jsou tyto události během testování překládány na příchozí agresivní objednávky, které jádro následně spáruje, a exekuce tak samo vygeneruje. Speciální události typu aukčních obchodů (6 – *cross trade*) a pozastavení obchodování (7 – *trading halt*) jsou z rozsahu práce vyřazeny.

**Podporované typy objednávek**
Jádro zpracovává výhradně limitní objednávky (*limit order*). Tržní objednávky (*market order*) jako samostatný typ nejsou podporovány, neboť jednak nejsou obsaženy v testovacích datech, a jednak jejich chování lze plně nasimulovat odesláním agresivní limitní objednávky s odpovídající exekuční cenou. Složitější typy, jako jsou stop objednávky (*stop-loss*), jsou z rozsahu práce vyřazeny.

**Logika párování**
Systém musí implementovat cenovou a časovou prioritu (*price-time*). Přednost má vždy objednávka s lepší cenou. V případě shody cenových hladin je upřednostněna dříve přijatá objednávka (princip FIFO). Alternativní párovací algoritmy, jako je například proporcionální párování (*pro-rata*), jsou z rozsahu vyřazeny. Nejsou součástí referenčních řešení a pro měření vlivu optimalizací nemají další přínos.

**Stav limitní knihy**
Jádro musí udržovat oddělenou nabídkovou (*ask/sell*) a poptávkovou (*bid/buy*) stranu, plynule aktualizovat objemy a odstraňovat spárované objednávky. Vnitřní stav musí po zpracování každé události exaktně odpovídat záznamům LOBSTER dat a umožňovat export pro účely verifikace.

### 5.1.2 Nefunkční požadavky

Nefunkční požadavky definují cílové technologické a výkonnostní standardy, které jsou od moderních burzovních systémů striktně vyžadovány. V kontextu této práce slouží jako hlavní metrika úspěšnosti. Zatímco výchozí implementace tyto požadavky záměrně nesplňuje, aby demonstrovala výkonnostní limity standardních struktur (např. `std::map` a dynamické alokace paměti), finální optimalizované jádro na nich musí bezpodmínečně stavět.

**Oddělení exekučního modelu**
Měření výkonu izoluje operace uvnitř limitní knihy objednávek. Systém neřeší režii spojenou s příjmem dat po síti ani synchronizaci vláken. Vstupní

data jsou předem načtena do souvislého bloku paměti (`std::vector`) a odtud sekvenčně zpracovávána. Výstupní události se pro eliminaci zpoždění ukládají do prealokovaného pole.

### Časová složitost operací

Kritické operace (vložení, vyhledání a zrušení objednávky) vyžadují minimální asymptotickou složitost. Cílem je dosáhnout konstantního času $\mathcal{O}(1)$, maximálně logaritmické složitosti $\mathcal{O}(\log n)$. Jakákoliv operace s lineární složitostí $\mathcal{O}(n)$ je na kritické cestě nepřípustná.

### Propustnost a latence

Optimalizované jádro musí zvládnout zpracování desítek milionů událostí za sekundu (*throughput*). Doba zpracování jednotlivých objednávek se musí pohybovat v řádu nanosekund. Zásadní je také minimalizace rozptylu latence (*jitter*) a potlačení extrémních zpoždění na 99. a 99,9. percentilu (*tail latency*).

### Celočíselná reprezentace dat

Pro deterministickou latenci a absolutní přesnost nesmí jádro využívat datové typy s pohyblivou řádovou čárkou (`float`, `double`). Ceny, objemy i vstupní časové značky (*timestamps*) musí být před vstupem do párovací logiky převedeny a zpracovávány výhradně jako celočíselné hodnoty (*fixed-point*).

### Eliminace dynamické alokace

Na rozdíl od výchozí verze má cílová optimalizovaná architektura na své kritické cestě (*hot path*) striktně zakázáno dynamicky alokovat paměť na haldě. Veškerá paměť pro uchování stavu limitní knihy musí být prealokována při startu.

### Lokalita dat

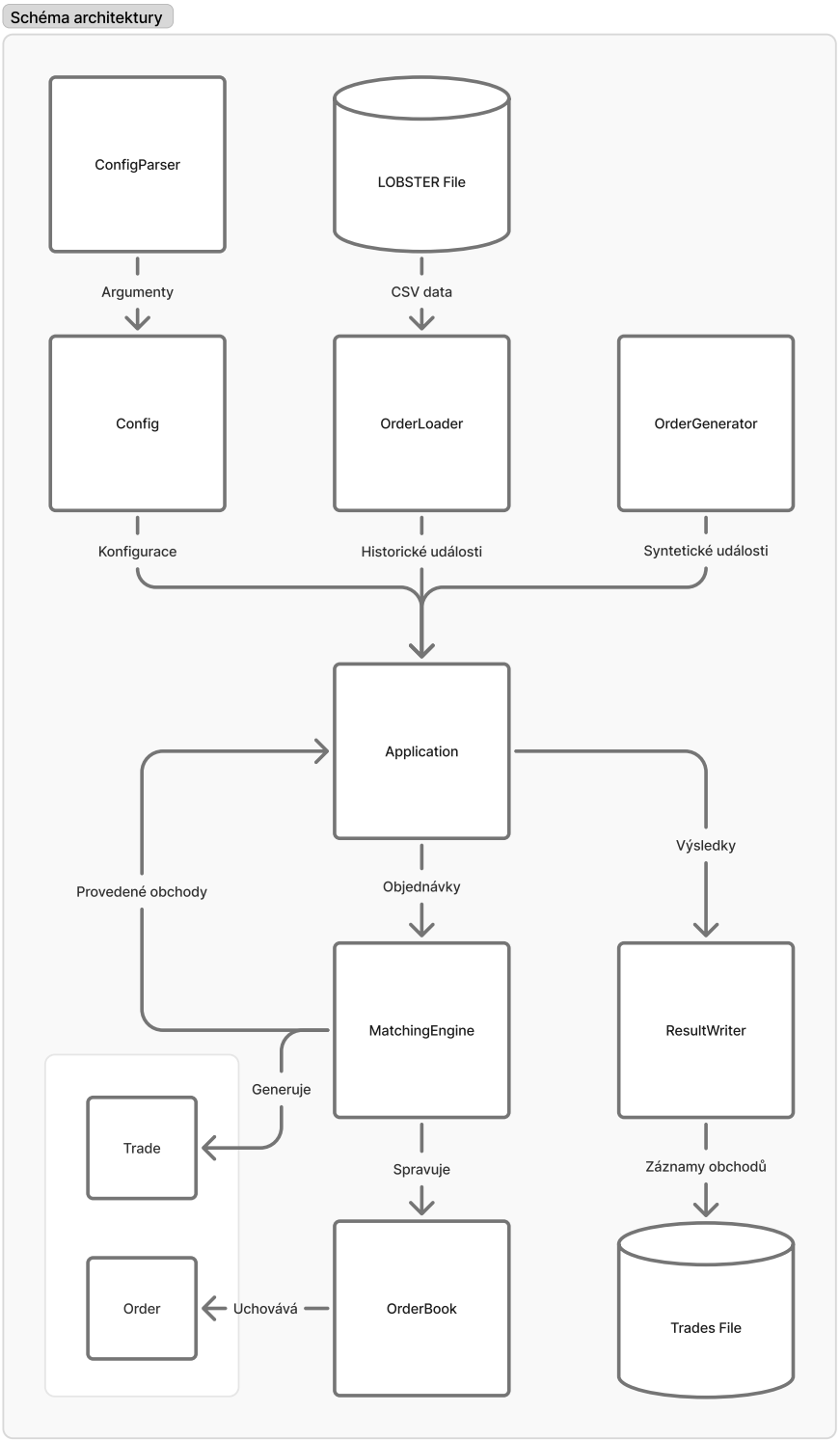
Architektura musí plně respektovat paměťovou hierarchii procesoru (*mechanical sympathy*). Datové struktury musí eliminovat *pointer chasing* typický pro uzlové struktury, maximalizovat prostorovou lokalitu a minimalizovat výpadky mezipaměti (*cache misses*).

### Vláknový model a afinita

Přepínání kontextu OS trvá mikrosekundy a způsobuje nežádoucí přepisování dat v L1 a L2 mezipamětích (*cache pollution*). Pro eliminaci tohoto nepřípustného zpoždění musí párovací jádro běžet výhradně v jednom vlákně uzamčeném na dedikované fyzické jádro procesoru (*core pinning*).

## 5.2 Základní architektura párovacího jádra

Tato kapitola uvádí návrh výchozí verze párovacího jádra a nezbytných komponent pro jeho chod. V návrhu je kladen důraz na modularitu systému a princip jedné odpovědnosti (SRP). Architektura je zatím postavena na klasickém objektovém paradigmatu (OOP), jehož primárním cílem v této fázi není maximalizace výkonu systému. Návrhy optimalizací jsou iterativně zaváděny v kapitole 6 zaměřené na vývoj jádra. Tato výchozí architektura je navržena tak, aby byla logika párování objednávek striktně oddělena od obslužných vstupně-výstupních operací a konfigurace. Z hlediska softwarového návrhu se systém skládá z následujících logických bloků a tříd (viz obrázek 5.1).




**Obrázek 5.1** Schéma architektury výchozího řešení

### 5.2.1 Konfigurace a řízení běhu

`Application`
: Hlavní řídicí třída. Zodpovídá za inicializaci a konfiguraci systému, propojení datových toků, prealokaci paměti pro tržní data a spuštění párovací smyčky.

`ConfigParser`
: Zpracovává vstupní parametry od uživatele (argumenty příkazové řádky) a validuje cesty k testovacím souborům či přepínače režimu běhu.

`Config`
: Datová struktura, do které `ConfigParser` ukládá načtenou konfiguraci. Tato třída je následně předávána ostatním komponentám pro zajištění jednotného přístupu k nastavení systému.

### 5.2.2 Příprava, generování a export dat

`OrderLoader`
: Třída zodpovědná za načítání a parsování historických zpráv z portálu LOBSTER (soubor `message`). Její klíčovou rolí je převod textových záznamů ve formátu hodnot oddělených čárkou (CSV) do interní celočíselné reprezentace.

`OrderGenerator`
: Generátor syntetických tržních dat. Slouží k přípravě rozsáhlých datových sad obsahujících desítky milionů zpráv, které jsou nezbytné pro zátěžové testování a měření propustnosti.

`ResultWriter`
: Zajišťuje formátovaný výpis výsledků měření a export chronologického seznamu všech vygenerovaných obchodů z paměti na disk.

### 5.2.3 Burzovní doména a párovací jádro

`Order`
: Základní datová entita reprezentující jednu tržní objednávku. Uchovává klíčové informace, jako je unikátní identifikátor, typ příkazu, cenová hladina, zbývající objem, indikátor strany trhu ($bid/ask$) a časová značka přijetí systémem.

`Trade`
: Datová struktura reprezentující provedený obchod. Vzniká ve chvíli, kdy jádro nalezne shodu mezi nabídkou a poptávkou. Obsahuje vlastní identifikátor, identifikátory obou spárovaných objednávek, časovou značku, realizační cenu a zobchodovaný objem.

`MatchingEngine`
: Zastřešující řídicí prvek burzovní logiky. Přijímá validované tržní události, aplikuje pravidla cenově-časové priority (*price-time*) a interaguje s příslušnou limitní knihou. Zodpovídá za párování objednávek a generování výsledných obchodů (`Trade`).

`OrderBook`
: Datová struktura reprezentující stav limitní knihy. V této základní implementaci spoléhá na standardní uzlové kontejnery jazyka C++. Objednávky jsou podle strany rozděleny do dvou asociativních kontejnerů, kde klíčem je cenová hladina a hodnotou je chronologicky seřazený seznam čekajících objednávek.

## 5.3 Scénáře testování a měření výkonu

Zatímco předchozí kapitola 4 definovala testovací prostředí a obecnou metodiku, tato sekce specifikuje konkrétní testovací scénáře navržené pro ověření korektnosti a změření výkonu párovacího jádra. Scénáře jsou navrženy tak, aby garantovaly, že aplikované optimalizace nenaruší logickou správnost jádra a současně umožní objektivní srovnání výkonnostního posunu vždy vůči předchozí, dosud nejrychlejší vývojové iteraci.

### 5.3.1 Scénáře testování funkčnosti

Burzovní systémy tvoří kritickou doménu, která podléhá přísnému dohledu finančních regulátorů a pracuje se značnými objemy klientských finančních prostředků. Z tohoto důvodu je vyžadována absolutní korektnost systému a jakékoliv logické chyby při zpracování a párování objednávek jsou zcela nepřípustné. Testování funkční správnosti proto tvoří klíčovou a neoddělitelnou součást celého vývoje. V rámci této práce je metodika testování rozdělena do dvou částí. Jednotkové testy izolovaně ověřují správnost doménových i podpůrných komponent, zatímco integrační testy validují chování celého systému prostřednictvím komplexních scénářů nad reálnými historickými daty LOBSTER.

#### Jednotkové testy

Zatímco princip regresního testování a volba frameworku GoogleTest jsou popsány v kapitole 4.3.1, v této fázi návrhu je klíčové definovat architekturu testovacích sad. Ty jsou navrženy tak, aby systematicky pokrývaly výhradně veřejné rozhraní doménových i podpůrných tříd.

V souladu s osvědčenými postupy softwarového inženýrství nejsou privátní metody testovány přímo. Vzhledem k tomu, že vnitřní implementace tříd podléhá během iterativních optimalizací významným změnám, testy vázané na

tyto detaily by opakovaně selhávaly a vyžadovaly by neustálou údržbu. Zaměření na relativně stabilní veřejné rozhraní tak minimalizuje nutnost častých úprav testů během vývoje. Mezi navržené testovací scénáře patří:

**Cenová a časová priorita**
Ověření, že limitní objednávky na stejné cenové hladině jsou vnitřními strukturami řazeny a následně párovány striktně v pořadí (FIFO), v jakém do systému dorazily.

**Částečné a vícenásobné plnění**
Validace stavu, kdy agresivní objednávka není zobchodována celá a její zbytek musí být zapsán do limitní knihy, nebo naopak svým objemem převyšuje protistranu na nejlepší ceně a musí být postupně spárována na několika cenových hladinách za sebou.

**Modifikace a rušení**
Zajištění, že zrušení existující objednávky korektně upraví agregovaný objem na dané cenové hladině, bezpečně odstraní záznam z paměti a nenaruší integritu ani návaznost okolních objednávek.

**Okrajové případy**
Ověření chování systému při krajních situacích, jako je pokus o párování v kompletně prázdné knize, vložení objednávky s nulovým objemem nebo situace, kdy exekuce odstraní poslední existující cenovou hladinu na jedné ze stran knihy.

## Integrační testy

Zatímco metodika testování nad daty LOBSTER je popsána v kapitole 4.3.1, tato sekce definuje návrh integračních testů. Scénáře testů jsou koncipovány jako kontinuální datový tok (*pipeline*). Nejprve jsou historické tržní zprávy načteny a pak sekvenčně předávány párovacímu jádru ke zpracování. Výstupní verifikace spočívá v průběžné kontrole vnitřního stavu limitní knihy. Po aplikaci tržní události (ze souboru `message`) je aktuální stav limitní knihy jádra porovnán s odpovídajícím řádkem v referenčním souboru `orderbook`.

Testovací scénáře jsou navrženy tak, aby simulovaly reálnou zátěž nad daty s různou úrovní hloubky trhu. Neomezují se pouze na nejlepší nabídku a poptávku (*top of book*), ale zahrnují i hlubší úrovně limitní knihy (*market depth*). Cílem těchto testů je ověřit schopnost systému korektně zpracovávat vkládání, modifikace a rušení objednávek i na vzdálených cenových hladinách, čímž se prokazuje stabilita vnitřních datových struktur. Podmínkou úspěšného průchodu je striktní shoda sledovaných stavů limitní knihy s referenčními historickými daty. Tento přístup zaručuje, že aplikované optimalizace nenaruší základní korektnost párovací logiky.

### 5.3.2 Benchmarky a měření výkonu

V návaznosti na zajištění funkční správnosti systému představuje měření výkonu druhou klíčovou fázi testování. Jelikož hlavním cílem této práce je optimalizace párovacího jádra pro dosažení co nejnižší latence a nejvyšší propustnosti, je nezbytné navrhnout měřicí infrastrukturu, která poskytne přesné metriky. Tato infrastruktura musí být schopna spolehlivě profilovat izolované operace na úrovni nanosekund, stejně jako komplexní chování paměťových struktur pod vysokou zátěží burzovních událostí. Následující sekce definují zvolené měřicí scénáře i metriky nezbytné pro objektivní vyhodnocení aplikovaných optimalizací.

#### Scénáře Google Benchmark

Pro exaktní měření výkonu jednotlivých izolovaných operací nad datovými strukturami je využit nástroj Google Benchmark (popsaný v kapitole 4.3.2). Navržená sada mikrobenchmarků systematicky pokrývá všechny kritické cesty párovacího jádra. Výběr scénářů reflektuje specifika limitní knihy a testuje jak operace v konstantním čase, tak výpočetně náročnější zásahy do datových struktur. Sada obsahuje následující benchmarky:

`InsertSamePrice a InsertNewPrice`
: Dvojice benchmarků testující vkládání pasivních objednávek. První scénář měří přidání objednávky na existující cenovou hladinu, což představuje rychlé připojení na konec seznamu. Druhý scénář naopak testuje paměťově náročnější založení zcela nové cenové hladiny, což vynucuje alokaci nového uzlu ve vnitřní vyhledávací struktuře.

`CancelPartial a CancelFull`
: Rozdělení operace zrušení na dva hardwarově odlišné scénáře. Částečné zrušení testuje primárně rychlou úpravu alokovaného objemu. Úplné zrušení je naopak paměťově nejnáročnější případ, kdy po indexaci prvku následuje jeho vyřazení z datových struktur, úprava ukazatelů a případná dealokace paměti.

`MatchSingle a MatchSweep`
: Tyto benchmarky izolují proces realizace obchodu. První scénář ověřuje nejrychlejší možnou cestu, tedy spárování agresivní objednávky s aktuálním vrcholem knihy bez nutnosti procházet další úrovně. Druhý scénář naopak simuluje průchod trhem (*sweep*), kdy velká objednávka svým objemem vyčerpá několik cenových hladin za sebou. Tento test prověřuje efektivitu sekvenčního čtení z paměti a hardwarovou režii při odstraňování prázdných cenových uzlů.

`FullWorkload`
: Simuluje chování reálného trhu. Náhodně střídá operace vložení, zrušení a exekuce, čímž záměrně zneplatňuje instrukční i datovou mezipaměť procesoru (*cache thrashing*) a brání efektivnímu využití hardwarového prediktoru skoků (*branch predictor*).

## Sledované výkonnostní metriky

Vzhledem k požadavkům na nízkou latenci nelze výkonnost jádra hodnotit pouze na základě průměrných hodnot. Měřicí infrastruktura je proto navržena tak, aby sledovala komplexní distribuci latencí a celkovou propustnost.

Před započetím samotného měření je vždy spuštěn jeden úvodní zahřívací běh, během kterého systém zpracuje celý dataset naprázdno. Tento krok je nezbytný pro naplnění mezipaměti procesoru a stabilizaci hardwarové predikce skoků. Následně je spuštěno deset ostrých iterací, přičemž finální výkonnostní metriky jsou agregovány výhradně z těchto měřených běhů. Tím je zaručeno, že do statistik nebudou započítány zkreslené hodnoty způsobené studeným startem (*cold start*).

Za běhu systému nad reálnými i syntetickými daty jsou následně zaznamenávány a vyhodnocovány tyto dvě klíčové metriky:

### Propustnost zpráv

Udává průměrný počet zpracovaných tržních zpráv za jednu sekundu v průběhu celého měření. Vzhledem k vysoké efektivitě implementace v C++ je tato hodnota měřena v milionech objednávek za sekundu. Společně s distribucí latencí tvoří stěžejní ukazatel výkonu – zatímco latence sleduje odezvu zpracování jedné objednávky, propustnost reflektuje celkovou kapacitu systému pod maximální zátěží.

### Distribuce latence

Zpracování jednotlivých zpráv je měřeno s přesností na nanosekundy. Ve vypsaných statistikách jsou záměrně separovány hodnoty mediánu (p50) a vysokých percentilů (p90, p99 a p99,9). V doméně HFT je právě hodnota p99,9 (*tail latency*) nejdůležitějším ukazatelem stability systému, neboť odhaluje ojedinělé výkonnostní propady způsobené špatným paměťovým zarovnáním, výpadky mezipaměti nebo vlivem operačního systému (*OS jitter*).

Kapitola 6

# Iterativní vývoj párovacího jádra

*Tato kapitola detailně popisuje implementaci a postupnou optimalizaci burzovního párovacího jádra v jazyce C++. Vývoj probíhá po iteracích. Výchozím bodem je plně funkční, avšak z hlediska výkonu naivní řešení, které využívá standardní objektové principy a uzlové datové struktury. Následující iterace krok za krokem představují aplikaci pokročilých optimalizací, jejichž cílem je co nejefektivnější využití hardwaru. Patří mezi ně eliminace dynamické alokace paměti za běhu, transformace datových struktur pro maximalizaci paměťové lokality a minimalizace nepředvídatelného větvení kódu. Výkonnostní dopad každé provedené změny na celkovou propustnost a latenci systému je exaktně vyhodnocován vůči předchozí iteraci. Z důvodu zachování plynulosti a čitelnosti textu jsou v jednotlivých podkapitolách prezentovány pouze stěžejní metriky, přičemž kompletní výpisy všech naměřených benchmarků jsou k dispozici v příloze této práce.*

## 6.1 Výchozí implementace

První vývojová iterace se zaměřuje na implementaci výchozího řešení. Tohoto cíle je dosaženo využitím standardního objektového přístupu a vestavěných datových kontejnerů ze standardní knihovny jazyka C++. Primárním účelem této iterace je vytvořit funkčně správné párovací jádro, které striktně dodržuje cenově-časovou prioritu.

Z hlediska výkonu se však jedná o nevhodnou architekturu, neboť zatím efektivně nevyužívá vlastnosti hardwaru a hierarchii paměti. Jak ukazuje výpis kódu 6.1, třída `OrderBook` spravuje nabídkovou a poptávkovou stranu trhu pomocí asociativního kontejneru `std::map`. Klíčem je zde cenová hladina a hodnotou chronologicky seřazený obousměrně zřetězený seznam (`std::list`) ob-

sahující jednotlivé objednávky. Tato třída poskytuje rozhraní formou veřejných metod, které využívá třída `MatchingEngine` ke zpracování objednávek.

**Výpis kódu 6.1** Definice třídy `OrderBook` pro výchozí implementaci

```
class OrderBook {
public:
    void insert(const Order& order);
    void cancel(const Order& order);

    void removeBestBid();
    void removeBestAsk();
    Order* bestBid();
    Order* bestAsk();

    std::vector<OrderBookLevel>
    bookState(std::size_t depth) const;

private:
    std::map<Price, std::list<Order>, std::greater<>> bids;
    std::map<Price, std::list<Order>> asks;
};
```

Právě tato kombinace struktur tvoří kritické úzké hrdlo celého systému. Každé vložení objednávky vyžaduje dynamickou alokaci na haldě (uzel seznamu), v případě vložení na novou cenovou hladinu i uzel mapy. Data jsou kvůli tomu fyzicky roztroušena po celé operační paměti. Procesor tak při snaze o průchod knihou, zejména při lineárním vyhledávání objednávek k odstranění, naráží na neustálé výpadky mezipaměti (*cache misses*). Stovky procesorových cyklů jsou tak ztraceny pasivním čekáním na dodání dat z hlavní paměti (*pointer chasing*).

### 6.1.1 Úprava integračních testů

Původním záměrem pro integrační testování bylo využití historických tržních dat z platformy LOBSTER. Během implementace se však tento přístup ukázal jako nevhodný. Jelikož soubory zaznamenávající tržní události a stavy limitní knihy zachycují situaci až v průběhu obchodního dne, nejsou výchozí stavy limitní knihy před začátkem měření známy.

Kvůli tomu dochází k problému při zpracování dat. Jakmile se v párovacím jádře vyčerpají všechny známé objednávky, limitní kniha se vyprázdní, ale v souboru následují další záznamy vycházející ze stavu, který jádro nezná. Tuto situaci by bylo možné ošetřit zvláštní podmínkou. Problém ale přetrvává u požadavků na zrušení objednávky s neznámým ID. V takovém případě udá-

lost v jádře nezanechá žádné změny, což vede k desynchronizaci s testovacími daty.

Zároveň je nežádoucí zanášet do kritické cesty jádra dodatečnou logiku ošetřující specifické testovací scénáře, neboť takové větvení by systém v produkčním nasazení neúměrně zpomalovalo. Udržování více verzí kódu by pak bylo náchylné na vznik chyb v produkčním prostředí, i když by se testovací verze jevila jako funkční.

Z těchto důvodů byly pro účely validace a měření jádra implementovány dva typy alternativních testů:

**Testy cenově-časové priority**
: Menší, deterministicky definované scénáře. Slouží k ověření logické správnosti párovacího algoritmu a striktnímu dodržování pravidla priority podle ceny a času.

**Syntetické zátěžové testy**
: Testy využívající komponentu `OrderGenerator`. Tento generátor umožňuje deterministicky vytvářet velký objem logicky konzistentních událostí s platnými identifikátory, což poskytuje stabilní prostředí pro zátěžové testování a měření výkonnosti systému bez závislosti na neúplných historických datech.

### 6.1.2 Měření výkonu pomocí TSC

Pro exaktní vyhodnocení výkonnostních rozdílů mezi jednotlivými iteracemi jádra je využito přímé čtení TSC. Jelikož při měření operací na úrovni nanosekund hraje roli každá instrukce, je nutné zamezit procesoru ve vykonávání instrukcí mimo pořadí (*out-of-order execution*). Toho je docíleno využitím funkce `_mm_lfence`, která funguje jako paměťová bariéra a garantuje dokončení všech předchozích instrukcí před čtením času. V kombinaci s funkcemi `__rdtsc` a `__rdtscp` lze exaktně měřit počet cyklů procesoru, které uplynuly během zpracování každé jednotlivé objednávky.

Pro převod naměřených cyklů na absolutní čas si aplikace při startu dynamicky kalibruje převodní poměr porovnáním TSC s časem ze standardní knihovny (`std::chrono`). Aby tato hodnota zůstala validní, je nutné eliminovat veškeré dynamické změny frekvence procesoru. Toho je v měřicím prostředí dosaženo pomocí systémových příkazů níže. Vzhledem ke specifikům linuxového ovladače `cpufreq`, který přijímá pouze definované frekvenční kroky, je pro dosažení fyzické hardwarové frekvence 2,27 GHz nutné nástroji předat zaokrouhlenou hodnotu 2,3 GHz (viz výpis 6.2).

**Výpis kódu 6.2** Zamknutí frekvence a vypnutí úsporných stavů CPU

```
# Vypnutí škálování frekvence CPU
echo 0 | sudo tee /sys/devices/system/cpu/cpufreq/boost
# Zamknutí CPU frekvence na 2.27 GHz
sudo cpupower frequency-set -g performance --min 2.3GHz --max 2.3GHz
# Zakázání přechodů do úsporných stavů (C-states)
sudo cpupower idle-set -D 0
# Sledování nastavené konfigurace
watch -n 1 sudo cpupower monitor
```

Po dokončení benchmarků je možné systém uvést do původního stavu pomocí těchto příkazů:

**Výpis kódu 6.3** Obnovení výchozího nastavení procesoru

```
# Zapnutí škálování frekvence CPU
echo 1 | sudo tee /sys/devices/system/cpu/cpufreq/boost
# Přechod do úsporného režimu a odemknutí frekvence
sudo cpupower frequency-set -g powersave --min 800MHz --max 3.1GHz
# Povolení úsporných stavů
sudo cpupower idle-set -E
```

Spouštění benchmarků probíhá z důvodu izolace pouze ze systémového terminálu po čistém startu stroje. Je tak zamezeno běhu paměťově náročných aplikací, jako jsou vývojová prostředí nebo webové prohlížeče. Tyto aplikace by na pozadí způsobovaly hardwarová přerušení a využívaly sdílenou L3 mezipaměť. Její přepisování by do naměřených výsledků vneslo značný rozptyl a znemožnilo by spolehlivé zhodnocení dílčích optimalizací.

### 6.1.3 Výsledky měření

Výchozí implementace párovacího jádra byla podrobena stanoveným mikrobenchmarkům pomocí frameworku Google Benchmark. Výsledky shrnuté v tabulce 6.1 ukazují značné výkonnostní rozdíly mezi jednotlivými operacemi. Pro každý test je uveden jeho název (**Benchmark**), počet objednávek v limitní knize (**Rozsah**), průměrná doba trvání jedné operace (**Latence**) a naměřená asymptotická časová složitost (**Složitost**). Proměnná $n$ v zápisu složitosti udává celkový počet aktivních objednávek aktuálně uložených v limitní knize.

Nejlepších výsledků na maximálně zaplněné limitní knize (1 000 000 uložených objednávek) dosahují operace s konstantní časovou složitostí $\mathcal{O}(1)$. Mezi ně patří vložení objednávky na existující cenovou hladinu (`InsertSamePrice`) v čase 36,6 ns či okamžité spárování (`MatchSingle`) v čase 41,3 ns. Založení nové cenové hladiny (`InsertNewPrice`) naproti tomu trvá 354,0 ns, jelikož vy-

žaduje průchod stromovou strukturou `std::map` se složitostí $\mathcal{O}(\log n)$. I přes relativně nízké absolutní hodnoty je rychlost těchto operací zbytečně brzděna režií spojenou s dynamickou alokací každého nového uzlu na haldě.

Kritické omezení výchozí implementace se plně projevuje při rušení objednávek a agresivním párování (`MatchSweep`). Jelikož jádro nedisponuje přímým přístupem k prvkům, vyhledání a následné částečné zrušení objednávky v hluboké knize (`CancelPartial`) trvá až 4,7 ms při milionu prvků. V případě úplného zrušení (`CancelFull`) doba roste na 5,7 ms. V obou případech se potvrzují teoretická očekávání o degradaci do lineární časové složitosti $\mathcal{O}(n)$ z důvodu iterativního průchodu strukturou `std::list`. V případě scénáře `MatchSweep`, který simuluje agresivní tržní objednávku likvidující desítky cenových hladin, byl rozsah z důvodu výpočetní náročnosti a pro zachování realističnosti omezen na 50 000 prvků. Zpracování tohoto objemu agresivní objednávky trvá 4,6 ms, čímž se potvrzuje očekávaná lineární asymptotická složitost $\mathcal{O}(n)$. V kontextu HFT by toto zpoždění, způsobené lineárním průchodem přes `std::list` a nedeterministickými skoky za ukazateli po paměti, vedlo k okamžitému zahlcení systému.

**Tabulka 6.1** Výsledky měření operací výchozí implementace

| Benchmark | Rozsah | Latence | Složitost |
|---|---|---|---|
| `InsertSamePrice` | 1 000 000 | 36,6 ns | $\mathcal{O}(1)$ |
| `InsertNewPrice` | 1 000 000 | 354,0 ns | $\mathcal{O}(\log n)$ |
| `CancelPartial` | 1 000 000 | 4,7 ms | $\mathcal{O}(n)$ |
| `CancelFull` | 1 000 000 | 5,7 ms | $\mathcal{O}(n)$ |
| `MatchSingle` | 1 000 000 | 41,3 ns | $\mathcal{O}(1)$ |
| `MatchSweep` | 50 000 | 4,6 ms | $\mathcal{O}(n)$ |
| `FullWorkload` | 1 000 000 | 65,9 ns | $\mathcal{O}(1)$ |

Pro zhodnocení chování v komplexních scénářích byly provedeny makrobenchmarky nad celými datovými sadami. Při těchto testech se však významně projevuje efekt pozorovatele (*observer effect*). Samotné čtení TSC a aplikace paměťových bariér pro každou zprávu snižuje celkovou propustnost systému, proto tabulky oddělují čistou propustnost od běhu s aktivním měřením latence (parametr `-l`).

Zpracování historických dat z portálu LOBSTER (tabulka 6.2) slouží k demonstraci výkonu při reálné tržní distribuci vkládání, rušení a párování. Spouštění běhu s čistou propustností a varianty s měřením latence lze provést pomocí příkazů uvedených ve výpisu 6.4.

**Výpis kódu 6.4** Spuštění párovacího jádra s LOBSTER daty

```
# Měření čisté propustnosti (ukázka pro ticker AAPL)
./matching_engine -i \
    ../data/LOBSTER/AAPL/AAPL_2012-06-21_message_lvl10.csv
# Propustnost s měřením latence
./matching_engine -l -i \
    ../data/LOBSTER/AAPL/AAPL_2012-06-21_message_lvl10.csv
```

Na těchto reálných datech se propustnost bez měření pohybuje okolo 2,5 milionu zpráv za sekundu. Výrazný rozptyl mezi mediánem (50. percentil) a krajními percentily (99. a vyšší percentil) je důsledkem operací rušení ve fragmentované paměti.

**Tabulka 6.2** Propustnost a latence výchozí implementace (LOBSTER data)

| **Metrika** | **AAPL (lvl 10)** | **AMZN (lvl 10)** | **MSFT (lvl 10)** |
|---|---|---|---|
| Počet zpráv | 365 401 | 258 329 | 635 351 |
| Míra spárování (%) | 10,20 | 7,61 | 5,26 |
| Propustnost – bez měření (mil./s) | 2,20 | 2,41 | 2,51 |
| Propustnost – s měřením (mil./s) | 1,88 | 2,08 | 2,02 |
| Průměrná latence (ns) | 497 | 437 | 469 |
| Minimální latence (ns) | 50 | 50 | 50 |
| Percentil 50 (ns) | 170 | 160 | 130 |
| Percentil 90 (ns) | 490 | 550 | 750 |
| Percentil 99 (ns) | 10 362 | 8 200 | 3 995 |
| Percentil 99,9 (ns) | 15 819 | 12 105 | 36 948 |
| Maximální latence (ns) | 29 346 | 40 591 | 67 978 |

Syntetické sady (tabulka 6.3) fungují jako zátěžové testy hraničních scénářů. Parametry zabudovaného generátoru umožňují přesně zacílit na specifickou neefektivní část systému (viz výpis 6.5).

**Výpis kódu 6.5** Spuštění párovacího jádra s generováním dat

```
# Pouze vkládání (100 % vkládání, 0 % rušení, žádný cenový překryv)
./matching_engine -l --generate 100000 --cancel 0 \
    --price 100000 --offset 5000 --deviation 0
# Časté rušení (50% pravděpodobnost rušení existujících objednávek)
./matching_engine -l --generate 100000 --cancel 50 \
    --price 100000 --offset 100 --deviation 0
# Časté párování (agresivní cenový překryv generující 69 % spárování)
./matching_engine -l --generate 100000 --cancel 10 \
    --price 100000 --offset 0 --deviation 50
```

Výsledky ukazují zajímavý dopad efektu pozorovatele. Zatímco u rychlého čistého vkládání se použití TSC a paměťových bariér projevuje propadem propustnosti z 15,12 na 7,18 mil./s, u scénářů s častým rušením a párováním je propustnost shodná bez ohledu na měření (0,02 a 0,16 mil./s). Logika lineárního vyhledávání a mazání uzlů je natolik pomalá, že zcela maskuje paměťovou režii bariér a čtecích instrukcí.

**Tabulka 6.3** Propustnost a latence výchozí implementace (syntetická data)

| **Metrika** | **100 % vkládání** | **50 % rušení** | **69 % párování** |
|---|---|---|---|
| Počet zpráv | 100 000 | 100 000 | 100 000 |
| Míra spárování (%) | 0,00 | 0,00 | 69,49 |
| Propustnost – bez měření (mil./s) | 15,12 | 0,02 | 0,16 |
| Propustnost – s měřením (mil./s) | 7,18 | 0,02 | 0,16 |
| Průměrná latence (ns) | 106 | 57 301 | 6 248 |
| Minimální latence (ns) | 60 | 50 | 50 |
| Percentil 50 (ns) | 80 | 790 | 190 |
| Percentil 90 (ns) | 260 | 200 605 | 520 |
| Percentil 99 (ns) | 520 | 248 110 | 150 548 |
| Percentil 99,9 (ns) | 961 | 271 728 | 181 967 |
| Maximální latence (ns) | 13 467 | 287 306 | 195 604 |

Naměřené hodnoty potvrzují, že standardní uzlové struktury jsou pro HFT nevhodné kvůli nepredikovatelné latenci při vyhledávání a silné fragmentaci paměti. Tyto výsledky tak slouží jako referenční hodnoty pro následující optimalizace. Vzhledem k výrazným prodlevám při rušení objednávek je prvním krokem nadcházející kapitoly nahrazení seznamu sekvenčním kontejnerem a zavedení přímého mapování identifikátorů na paměťové pozice, což odstraní lineární hledání.

## 6.2 Optimalizace datových struktur a algoritmů

Předchozí analýza výchozí implementace jasně identifikovala hlavní architektonické slabiny systému: vysokou míru fragmentace paměti, nepredikovatelné skoky za ukazateli (*pointer chasing*) a neefektivní lineární vyhledávání objednávek. Tyto nedostatky jsou spjaty s použitím standardních uzlových kontejnerů (`std::map`, `std::list`).

Tato kapitola proto uvádí postupný přechod od objektového k datově orientovanému přístupu. Hlavním cílem tohoto přístupu je transformace datových struktur tak, aby odpovídaly způsobu, jakým moderní procesory načítají data do mezipaměti (*cache*). Optimalizace probíhá v několika navazujících krocích. Prvním z nich je algoritmické odstranění lineární časové složitosti při rušení objednávek pomocí přímého indexování. Následně je řešena eliminace dynamických alokací paměti přes vlastní paměťové fondy (*memory pools*), a nakonec dochází k úplné linearizaci stromových struktur, doplněné o efektivní vyhledávání cenových hladin pomocí bitových operací.

### 6.2.1 Přímé indexování objednávek

Nejzávažnější zjištěnou limitací výchozí implementace byla nutnost lineárního vyhledávání při rušení existujících objednávek. Nalezení konkrétní objednávky ve spojovém seznamu `std::list` degradovalo složitost operace na $\mathcal{O}(n)$, což při milionu prvků způsobovalo latenci v řádech milisekund.

Prvním krokem k řešení tohoto problému je zavedení efektivního vyhledávacího mechanismu a nahrazení spojového seznamu sekvenčním paměťovým blokem. Jak ilustruje výpis kódu 6.6, limitní kniha nově využívá kontejner `std::vector` pro ukládání objednávek na cenové hladině. Rychlý přístup k objednávkám zajišťuje nově přidaná hashovací tabulka `std::unordered_map`, která mapuje unikátní identifikátor objednávky (`OrderId`) přímo na její fyzické umístění v paměti (`OrderLocation`). Tato struktura v sobě nese informaci o straně obchodu, cenové hladině a přesném indexu uvnitř daného pole.

**Výpis kódu 6.6** Definice třídy `OrderBook` s využitím přímého indexování

```
class OrderBook {
public:
    void insert(const Order& order);
    void cancel(const Order& order);

    void eraseLocation(OrderId orderId);
    Order* bestBid();
    Order* bestAsk();

    std::vector<OrderBookLevel>
    bookState(std::size_t depth) const;

private:
    std::unordered_map<OrderId, OrderLocation> locations;

    std::map<Price, std::vector<Order>, std::greater<>> bids;
    std::map<Price, std::vector<Order>> asks;
};
```

Tato změna přístupu k existujícím objednávkám přináší očekávaný výsledek. Z naměřených dat vyplývá, že průměrná doba potřebná ke zrušení objednávky v plně zatížené knize (`CancelPartial`) skokově klesla z původních 4,7 ms na 12,0 ns. Asymptotická časová složitost byla redukována z $\mathcal{O}(n)$ na čisté $\mathcal{O}(1)$. Rychlost této operace je tak nyní zcela nezávislá na aktuálním objemu dat v limitní knize.

Ačkoliv byl primární problém s rušením objednávek vyřešen, benchmark odhalil skrytou cenu za použití standardní hashovací tabulky a dynamického vektoru. Zavedení vrstvy `std::unordered_map` a správa souvislého bloku paměti v `std::vector` přidaly do systému značnou konstantní režii.

Kritická operace okamžitého spárování z čela fronty (`MatchSingle`) se z původních 41 ns více než stonásobně prodloužila na 4,2 µs. I když si operace zachovala asymptotickou složitost $\mathcal{O}(1)$, její absolutní latence je pro HFT nepřijatelná. Ztráta výkonu se projevila i na smíšené zátěži (`FullWorkload`), jejíž průměrná latence vzrostla ze 66 ns na 346 ns.

Tato iterace demonstruje, že samotné dosažení optimální asymptotické složitosti $\mathcal{O}(1)$ nedostačuje, pokud je vykoupeno těžkopádnou dynamickou správou paměti uvnitř standardních kontejnerů jazyka C++. Pro další posun je proto nezbytné převzít plnou kontrolu nad alokací paměti, čemuž se věnuje následující podkapitola.

### 6.2.2 Eliminace dynamických alokací

Jak ukázala předchozí iterace, použití standardních sekvenčních a asociativních kontejnerů sice řeší algoritmickou složitost, ale vnáší do systému značnou výkonnostní režii. Hlavním zdrojem této režie je alokátor `std::allocator`, který si při každém vložení do `std::map` nebo `std::unordered_map` vyžádá paměť od OS (zpravidla prostřednictvím funkce `malloc`). Tyto zásahy operačního systému narušují běh programu, způsobují fragmentaci paměti a vyvolávají výpadky stránek (*page faults*).

Standard C++17 nabízí řešení v podobě polymorfních alokátorů paměti (`std::pmr`), které umožňují definovat vlastní paměťové fondy bez nutnosti modifikovat definice kontejnerů. Byly proto otestovány dvě varianty těchto alokátorů:

1. **`std::pmr::unsynchronized_pool_resource`**
   Tento alokátor udržuje oddělené fondy pro různé velikosti paměťových bloků, což minimalizuje fragmentaci. Přestože optimalizuje využití paměti a na rozdíl od své synchronizované varianty zcela eliminuje režii zamykání vláken, vyhledávání vhodného bloku a správa vnitřních fondů nevedly k požadovanému zvýšení propustnosti.

2. **`std::pmr::monotonic_buffer_resource`**
   Alokátor navržený pro vysokou rychlost. Paměť pouze lineárně ukrajuje ze souvislého bloku a nikdy ji neuvolňuje (až do zničení samotného alokátoru). Jak ukazuje výpis 6.7, pro tento účel byl předem alokován rozsáhlý paměťový blok (`preallocBuffer`), nad kterým kontejner operuje.

**Výpis kódu 6.7** Využití `monotonic_buffer_resource` pro eliminaci volání OS

```
class OrderBook {
// ...
private:
    std::unique_ptr<std::byte[]> preallocBuffer;
    std::pmr::monotonic_buffer_resource memoryPool;

    using PmrOrderVector = std::pmr::vector<Order>;
    using PmrAsksMap = std::pmr::map<Price, PmrOrderVector>;
    // ...
};
```

Přechod na `monotonic_buffer_resource` však nepřinesl očekávané plošné zrychlení, naopak odhalil skryté limity uzlových kontejnerů. Přestože u vybraných operací (například `MatchSweep`) klesla latence přibližně o 10 %, jiné

scénáře naopak zaznamenaly mírný propad výkonnosti. Kritické okamžité párování (`MatchSingle`) zůstalo téměř beze změny na výrazně vysoké hodnotě okolo 4,2 µs. Smíšená zátěž (`FullWorkload`) se po nasazení tohoto alokátoru dokonce zpomalila o přibližně 10 % (z původních 346 ns na 382 ns).

Hlavním problémem zde zůstává fakt, že ačkoliv `std::pmr` zrychluje získání paměti z prealokovaného bloku, nezabrání její fragmentaci. Při použití těchto kontejnerů procesor nadále vynakládá značné množství času na iterování přes uzly, přičemž režie spojená s virtuálním voláním polymorfního alokátoru (*virtual dispatch*) u malých objektů zcela překonala přínosy absence volání `malloc`.

K dosažení absolutní kontroly nad správou paměti bylo přistoupeno k úplnému odstranění standardní dynamické alokace a nahrazení ukazatelů za celočíselné indexy do prealokovaných datových polí. Strukturu limitní knihy s vlastním paměťovým fondem ilustruje výpis kódu 6.8:

**Výpis kódu 6.8** Struktura `OrderBook` s využitím vlastního paměťového fondu

```
class OrderBook {
public:
    explicit OrderBook(std::size_t maxOrders);
    // ...
private:
    std::vector<Order> ordersPool;
    std::vector<OrderIndex> freeIndices;

    std::map<Price, std::vector<OrderIndex>, std::greater<>> bids;
    std::map<Price, std::vector<OrderIndex>> asks;
    std::unordered_map<OrderId, OrderIndex> locations;

    OrderIndex allocateOrder(const Order& order);
    void freeOrder(OrderIndex index);
};
```

Při startu párovacího jádra je alokováno pole `ordersPool`, dimenzované na maximální očekávaný počet objednávek. Cenové hladiny (`bids`, `asks`) pak neuchovávají kopie objektů `Order`, ale pouze celočíselné indexy odkazující do tohoto fondu. Uvolněné indexy po zrušených objednávkách jsou okamžitě recyklovány prostřednictvím pomocného pole `freeIndices`.

Tato datově orientovaná architektura je zásadním průlomem. Oddělení správy paměti od logiky stromových struktur vedlo k zásadnímu snížení latence ve smíšené zátěži (`FullWorkload`) na 76,3 ns, což znamená téměř pětinásobné zrychlení oproti polymorfním alokátorům. Úplné zrušení objednávky (`CancelFull`) proběhne za průměrných 30,6 ns. Ačkoliv je tato hodnota nepatrně vyšší než u `monotonic_buffer_resource` (21,5 ns) z důvodu nutnosti

vracet uvolněné indexy zpět do zásobníku, stále se drží o několik řádů hluboko pod výsledky původní výchozí implementace z první iterace.

Naměřená data ovšem poukazují ještě na jeden nedostatek této verze jádra. Jelikož jsou indexy na cenových hladinách nyní uchovávány ve vektorech (`std::vector<OrderIndex>`), smazání spárované objednávky z čela fronty vyžaduje plošný posun všech následujících položek v paměti. V důsledku toho, ačkoliv je vkládání a rušení výrazně rychlé, operace (`MatchSingle`) u milionu prvků degradovala na 116 µs, čímž se propadla do neefektivní lineární časové složitosti $\mathcal{O}(n)$.

Při hledání řešení tohoto problému by se jako první nabízelo prosté převrácení logiky vektoru, tedy vkládání nových objednávek na jeho začátek a odebírání těch nejstarších z konce pomocí operace `pop_back()`, která probíhá v čase $\mathcal{O}(1)$. Tento postup by sice okamžité párování skutečně zrychlil, ale danou výpočetní zátěž by pouze přenesl na operaci vkládání (`InsertSamePrice`). Jelikož je z důvodu zachování cenově-časové priority (FIFO) nutné udržovat chronologické řazení, vložení každé nové objednávky na nultý index by opět vyžadovalo posun celého obsahu vektoru, čímž by systém opět degradoval do $\mathcal{O}(n)$. V kontextu HFT je však naprosto kritické udržet obě tyto stěžejní operace v konstantním čase nezávisle na sobě.

### 6.2.3 Linearizace stromových struktur

Pro vyřešení problému s přesouváním paměti při spárování objednávek a za účelem dosažení absolutní hardwarové efektivity bylo nutné přehodnotit samotné základy ukládání tržních dat. Dosavadní implementace stále spoléhala na standardní uzlové struktury pro reprezentaci cenových hladin (`std::map`) a vyhledávání objednávek (`std::unordered_map`). Tyto struktury byly v poslední vývojové iteraci kompletně odstraněny a nahrazeny přímým mapováním do prealokovaných jednorozměrných polí.

Kritický problém lineární časové složitosti u operace okamžitého párování (`MatchSingle`) byl vyřešen odstraněním `std::vector` na úrovni cenové hladiny. Jak ukazuje výpis struktury `PriceLevel`, cenová hladina nově definuje frontu objednávek pouze pomocí dvou ukazatelů (indexů) na začátek a konec. Propojení objednávek na dané ceně je realizováno formou intruzivního spojového seznamu (*intrusive linked list*). Každý objekt `Order` v paměťovém fondu si v sobě udržuje index na následující prvek. Díky tomuto přístupu se spárované objednávky z paměti fyzicky nemažou ani neposouvají, ale systém v čase $\mathcal{O}(1)$ pouze posune ukazatel `headIdx` cenové hladiny na následující prvek v řadě.

**Výpis kódu 6.9** Architektura ploché limitní knihy s indexováním hladin

```
struct PriceLevel {
    OrderIndex headIdx = INVALID_INDEX;
    OrderIndex tailIdx = INVALID_INDEX;
};

class OrderBook {
// ...
private:
    std::vector<Order> ordersPool;
    std::vector<OrderIndex> freeIndices;
    std::vector<OrderIndex> locations;

    std::vector<PriceLevel> bids;
    std::vector<PriceLevel> asks;

    PriceBitset bidBits;
    PriceBitset askBits;
    // ...
};
```

Další zásadní optimalizací byla eliminace hashovací tabulky pro mapování identifikátorů. Místo ní se pro vyhledávání lokace objednávek využívá přímé indexování do pole `locations`, kde roli indexu hraje ID objednávky. Vzhledem k tomu, že externí burzovní identifikátory mohou být rozsáhlá 64bitová čísla nebo dokonce textové řetězce, není možné alokovat souvislé pole indexované přímo těmito hodnotami (z důvodu nerealistických paměťových nároků). Z toho důvodu byla zavedena pomocná komponenta `OrderIdMapper`. Ta na vstupním rozhraní systému překládá externí identifikátory na kompaktní sekvenci interních indexů číslovaných od nuly a umožňuje i zpětné mapování při ukládání realizovaných obchodů. Tím je garantováno, že pole `locations` zabírá v operační paměti pouze nezbytný prostor.

### 6.2.4 Bitové masky pro hledání cenových hladin

Odstraněním stromové struktury `std::map` a jejím nahrazením za souvislé pole hladin (kde indexem je přímo burzovní cena) sice systém získal přístup ke konkrétní cenové hladině v čase $\mathcal{O}(1)$, ztratil však možnost iterace pouze po neprázdných hladinách přes seřazené klíče. Pokud dojde k vyčerpání aktuálně nejlepší nabídky či poptávky, systém musí okamžitě lokalizovat nejbližší obsazenou cenovou hladinu. Lineární iterace přes tisíce prázdných prvků vektoru by v takovém případě znamenala nepřijatelné výkonnostní zpoždění.

Pro zachování konstantní složitosti tohoto vyhledávání využívá optimalizované jádro hierarchické bitové masky (třída `PriceBitset`). Každá cenová hladina v systému je reprezentována jediným bitem. Při vložení objednávky na novou hladinu je příslušný bit nastaven na hodnotu 1, při jejím vyprázdnění na 0.

Struktura je rozdělena do čtyř úrovní (`l0` až `l3`), přičemž každý bit ve vyšší úrovni agreguje stav 64 bitů z úrovně nižší. Díky tomuto uspořádání a využití specifických instrukcí procesoru pro efektivní hledání prvního nastaveného bitu (např. funkcí `__builtin_ctzll`) dokáže systém přeskočit obrovské bloky prázdných cenových hladin pomocí jediné operace. Vzhledem k pevně dané a konstantní hloubce této struktury je nejbližší aktivní hladina nalezena maximálně ve čtyřech krocích. Tento přístup tak zaručuje konstantní časovou složitost $\mathcal{O}(1)$.

### 6.2.5 Výsledky měření

Nasazení architektury ploché limitní knihy s hierarchickými bitovými maskami představuje dosavadní vrchol všech optimalizací. Naměřené hodnoty v tabulce 6.4 potvrzují dosažení konstantních složitostí u všech operací, kromě `MatchSweep`, kde je lineární složitost dána její podstatou. Operace `CancelFull` spadla z původních 5,7 ms na 6,0 ns. Okamžité párování (`MatchSingle`) se stabilizovalo na 26,9 ns, čímž systém získal zpět deterministické chování nezávislé na objemu dat.

**Tabulka 6.4** Výsledky měření operací architektury ploché limitní knihy

| Benchmark | Rozsah | Latence | Složitost |
|---|---|---|---|
| `InsertSamePrice` | 1 000 000 | 60,9 ns | $\mathcal{O}(1)$ |
| `InsertNewPrice` | 1 000 000 | 32,9 ns | $\mathcal{O}(1)$ |
| `CancelPartial` | 50 000 | 6,0 ns | $\mathcal{O}(1)$ |
| `CancelFull` | 50 000 | 6,0 ns | $\mathcal{O}(1)$ |
| `MatchSingle` | 1 000 000 | 26,9 ns | $\mathcal{O}(1)$ |
| `MatchSweep` | 50 000 | 1,9 ms | $\mathcal{O}(n)$ |
| `FullWorkload` | 1 000 000 | 29,6 ns | $\mathcal{O}(1)$ |

Výsledky mikrobenchmarku pro vkládání objednávek do stejné cenové hladiny (`InsertSamePrice`) primárně potvrzují, že navržená datová struktura dosahuje požadované asymptotické časové složitosti $\mathcal{O}(1)$. Jak je patrné z výstupu, velikost limitní knihy nemá vliv na logickou složitost algoritmu.

Detailní pohled na naměřené absolutní časy však odhaluje vliv paměťové hierarchie procesoru (*cache hierarchy*). Křivka latence není dokonale plochá, ale tvoří tři zřetelné výkonnostní stupně, které korespondují s postupným vyčerpáváním kapacity jednotlivých úrovní mezipaměti.

**L1/L2 mezipaměť (do 65 tisíc objednávek)**
Párovací jádro s prealokovanými vnitřními strukturami se vejde do nejrychlejších mezipamětí procesoru. Operace vkládání vykazuje latenci okolo 32 ns. Díky sekvenčnímu uložení dat v paměti dokáže hardwarový *prefetcher* efektivně maskovat latenci L2 mezipaměti, čímž eliminuje výkonnostní skok při přechodu z L1 cache.

**L3 mezipaměť (od 100 tisíc do cca 500 tisíc objednávek)**
Při tomto zaplnění již celková velikost vnitřních struktur jádra přesahuje kapacitu L2 mezipaměti. Procesor musí data adresovat v pomalejší sdílené L3 mezipaměti, což zvyšuje latenci na přibližně 46 ns.

**Hlavní paměť RAM (nad 1 milion objednávek)**
V tomto množství přesahuje paměťová náročnost jádra a jeho struktur běžnou kapacitu L3 mezipaměti. Přístup k datům je limitován rychlostí hlavní paměti RAM, což stabilizuje latenci na hodnotě 61 ns.

Makrobenchmarky nad historickými daty z portálu LOBSTER (tabulka 6.5) demonstrují schopnost jádra zpracovávat reálný tržní provoz s propustností přesahující 30 milionů zpráv za sekundu. Ve srovnání s výchozí implementací (2,5 mil./s) jde o dvanáctinásobné zvýšení výkonu. Průměrná latence se snížila z 470 ns na 76 ns. Zásadní je zejména eliminace nejvyšší latence (*tail latency*) – percentil 99,9 klesl z desítek mikrosekund na hodnoty okolo 1,6 až 3,1 µs.

**Tabulka 6.5** Propustnost a latence ploché limitní knihy (LOBSTER data)

| Metrika | AAPL (lvl 10) | AMZN (lvl 10) | MSFT (lvl 10) |
|---|---|---|---|
| Počet zpráv | 365 401 | 258 329 | 635 351 |
| Míra spárování (%) | 10,20 | 7,61 | 5,26 |
| Propustnost – bez měření (mil./s) | 28,52 | 27,91 | 30,55 |
| Propustnost – s měřením (mil./s) | 8,53 | 8,57 | 8,74 |
| Průměrná latence (ns) | 78 | 77 | 76 |
| Minimální latence (ns) | 40 | 40 | 40 |
| Percentil 50 (ns) | 60 | 60 | 70 |
| Percentil 90 (ns) | 100 | 90 | 90 |
| Percentil 99 (ns) | 350 | 310 | 200 |
| Percentil 99,9 (ns) | 1 652 | 2 663 | 3 104 |
| Maximální latence (ns) | 32 807 | 10 664 | 143 465 |

Nejsilnější důkaz o efektivitě datově orientovaného návrhu přinášejí hraniční syntetické testy (tabulka 6.6). Scénáře, které v první výchozí iteraci systém prakticky paralyzovaly kvůli iterativním průchodům, nyní bez problému

běží s propustností v řádech desítek milionů zpráv za sekundu. Scénář simulující vysokou zátěž s 50% pravděpodobností rušení vykazuje čistou propustnost 24,88 mil./s, což v porovnání s původními 0,02 mil./s znamená více než tisícinásobné zlepšení. Podobně i zátěžový test s agresivním 69% podílem párování dosahuje vysoce stabilní propustnosti 16,97 mil./s.

**Tabulka 6.6** Propustnost a latence ploché limitní knihy (syntetická data)

| **Metrika** | **100 % vkládání** | **50 % rušení** | **69 % párování** |
|---|---|---|---|
| Počet zpráv | 100 000 | 100 000 | 100 000 |
| Míra spárování (%) | 0,00 | 0,00 | 69,49 |
| Propustnost – bez měření (mil./s) | 16,37 | 24,88 | 16,97 |
| Propustnost – s měřením (mil./s) | 7,07 | 8,30 | 7,24 |
| Průměrná latence (ns) | 102 | 82 | 97 |
| Minimální latence (ns) | 60 | 40 | 40 |
| Percentil 50 (ns) | 70 | 60 | 90 |
| Percentil 90 (ns) | 80 | 80 | 130 |
| Percentil 99 (ns) | 180 | 100 | 200 |
| Percentil 99,9 (ns) | 3 314 | 3 234 | 3 144 |
| Maximální latence (ns) | 13 828 | 190 530 | 9 283 |

Základní architektonické cíle stanovené v úvodu této kapitoly se tak podařilo beze zbytku naplnit. Převedením abstrakcí do jazyka hardwaru vzniklo vysoce propustné a deterministické párovací jádro s časovou složitostí $\mathcal{O}(1)$ pro všechny klíčové operace.

Ačkoliv průměrná a mediánová latence dosáhla hodnot hluboko pod hranicí jedné mikrosekundy, data z makrobenchmarků ještě odhalují prostor pro zlepšení. Hodnoty na nejvyšších percentilech (*tail latency*) a maximální zaznamenané latence (dosahující až 190 µs u syntetických dat a 143 µs na reálných datech) poukazují na to, že systém je stále náchylný k vnějším vlivům operačního systému a hardwarovým narušením. Následující kapitoly se proto přesunou od algoritmické složitosti k izolaci běhového prostředí a pokročilým nízkoúrovňovým optimalizacím. Jejich hlavním cílem již nebude primárně další zvyšování absolutní propustnosti, ale izolace běžícího vlákna a stabilizace těchto výkyvů latence.

## 6.3 Izolace běhového prostředí

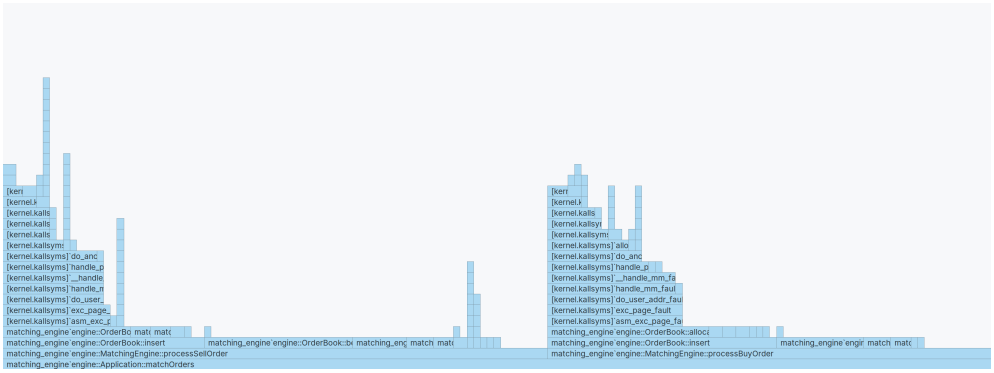
Ačkoliv přechod na ploché datové struktury v limitní knize objednávek vedl k výraznému zvýšení propustnosti a snížení průměrné latence, samotný kód

aplikace neběží v plně izolovaném prostředí. Moderní operační systémy využívají techniky jako preemptivní multitasking, virtuální paměť a komplexní správu napájení, což vnáší do běhu deterministických systémů značný rozptyl (*jitter*). Pro aplikace v oblasti HFT, kde je kritická právě stabilita odezvy v nejvyšších percentilech (*tail latency*), je nezbytné omezit tyto vlivy operačního systému na minimum.

### 6.3.1 Uzamčení fyzické paměti

Jedním z hlavních zdrojů nepředvídatelných prodlev je mechanismus virtuální paměti a stránkování. I když je v systému dostatek volné operační paměti, může OS rozhodnout o přesunu nepoužívaných stránek paměti do odkládacího prostoru na disku (*swap*). Opětovný přístup k těmto datům pak vyvolá výpadek stránky (*page fault*), který vyžaduje synchronní načtení dat z disku, což je z pohledu HFT nepřípustné.

Pro potvrzení tohoto jevu byla provedena analýza poslední implementace pomocí nástroje `perf`. Jak ukazuje plamenný graf na obrázku 6.1, i během kritické cesty zpracování objednávek dochází k častým výpadkům stránek a jejich obsluze v jádře OS (symboly `page_fault`), což vnáší do měření mikrosekundové prodlevy. Tyto výpadky jsou způsobeny zejména „líným“ mapováním paměti (*demand paging*), kdy OS přiděluje fyzické stránky až v okamžiku prvního zápisu.



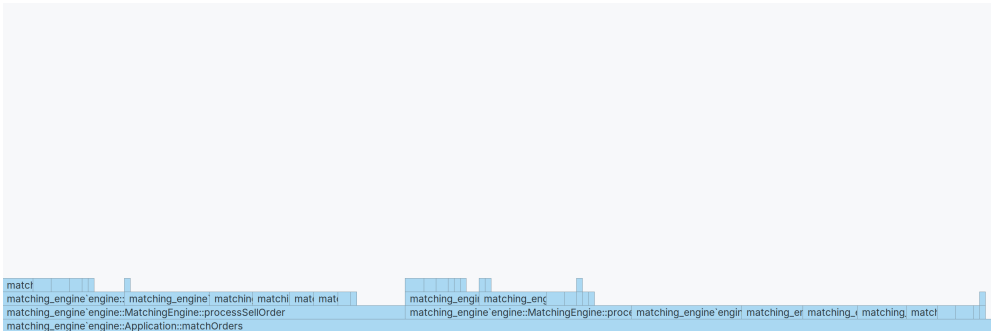

**Obrázek 6.1** Plamenný graf zachycující výpadky stránek na kritické cestě

K eliminaci tohoto rizika využívá párovací jádro funkci `mlockall` s příznaky `MCL_CURRENT | MCL_FUTURE`. Tato operace instruuje operační systém, aby veškerou aktuálně alokovanou paměť procesu i veškerou paměť, která bude alokována v budoucnu, uzamkl ve fyzické RAM a zabránil jejímu odsunutí na disk.

Je však nutné podotknout, že operační systém Linux z bezpečnostních důvodů standardně omezuje množství paměti, kterou může neprivilegovaný uživatel takto uzamknout (prostřednictvím limitu `memlock`). Pokud se aplikace

pokusí tento limit překročit, operační systém alokaci zamítne, což typicky vede k pádu programu s chybou `std::bad_alloc`. Pro úspěšné nasazení je proto nezbytné tento limit před spuštěním aplikace adekvátně navýšit, či zcela zrušit, například pomocí nástroje `prlimit` (vyvoláním příkazu `sudo prlimit --memlock=unlimited --pid $$`).

Efekt tohoto opatření je patrný na obrázku 6.2, kde po aktivaci uzamčení paměti symboly pro obsluhu výpadků stránek zcela zmizely.




**Obrázek 6.2** Plamenný graf po aplikaci uzamčení paměti pomocí `mlockall`

Kromě odstranění symbolů obsluhy výpadků stránek z profilování se efekt uzamčení paměti projevuje zejména na stabilitě měření a maximální latenci. V konfiguraci bez uzamčení paměti vykazuje první běh benchmarku (*warmup*) typicky o 10 % vyšší latenci, neboť během něj dochází k prvotnímu fyzickému mapování stránek. Po aplikaci `mlockall` se doba trvání úvodního běhu nejen sjednotila s následujícími iteracemi, ale došlo také k celkovému snížení průměrné latence i hodnot na nejvyšších percentilech (*tail latency*). To jednoznačně potvrzuje, že veškerá potřebná paměť byla do RAM uzamčena a zpřístupněna již při inicializaci aplikace. Tento propad maximálních latencí (z 95 µs na 18 µs) ilustrují výpisy z běhů 6.10 a 6.11.

**Výpis kódu 6.10** Výsledky běhu před aplikací `mlockall`

```
Loading orders from file:
../data/LOBSTER/MSFT/MSFT_2012-06-21_message_lvl10.csv
Loaded 635351 relevant order events.
Running 11 iterations...

Warmup:        80.91 ms
[ 1/10] Bench: 74.43 ms
[ 2/10] Bench: 74.35 ms
[ 3/10] Bench: 74.12 ms
[ 4/10] Bench: 74.40 ms
[ 5/10] Bench: 74.30 ms
[ 6/10] Bench: 74.33 ms
[ 7/10] Bench: 73.93 ms
[ 8/10] Bench: 74.27 ms
[ 9/10] Bench: 73.88 ms
[10/10] Bench: 74.47 ms

============= Benchmark results =============
--- General ---
Processed orders:    635351
Created trades:      33398
Match rate:          5.26 %
Avg duration:        74.25 ms
Avg throughput:      8.56 M orders/s

--- Latency ---
Min:                 40 ns
Max:                 95422 ns
Average:             77 ns
Median (p50):        70 ns
90. perc (p90):      90 ns
99. perc (p99):      170 ns
99.9. perc (p99.9):  3454 ns
=============================================
```

**Výpis kódu 6.11** Výsledky běhu po aplikaci `mlockall`

```
Memory locked successfully
Loading orders from file:
../data/LOBSTER/MSFT/MSFT_2012-06-21_message_lvl10.csv
Loaded 635351 relevant order events.
Running 11 iterations...

Warmup:        71.73 ms
[ 1/10] Bench: 71.75 ms
[ 2/10] Bench: 71.88 ms
[ 3/10] Bench: 71.75 ms
[ 4/10] Bench: 71.60 ms
[ 5/10] Bench: 71.59 ms
[ 6/10] Bench: 71.67 ms
[ 7/10] Bench: 71.51 ms
[ 8/10] Bench: 71.60 ms
[ 9/10] Bench: 72.96 ms
[10/10] Bench: 71.73 ms

============= Benchmark results =============
--- General ---
Processed orders:    635351
Created trades:      33398
Match rate:          5.26 %
Avg duration:        71.80 ms
Avg throughput:      8.85 M orders/s

--- Latency ---
Min:                 40 ns
Max:                 18075 ns
Average:             73 ns
Median (p50):        70 ns
90. perc (p90):      90 ns
99. perc (p99):      150 ns
99.9. perc (p99.9):  1462 ns
=============================================
```

### 6.3.2 Afinita vláken a izolace jader

V moderních operačních systémech dochází k neustálému přepínání kontextu mezi běžícími procesy a k jejich dynamické migraci napříč jádry procesoru. To vede k výrazné degradaci výkonu vlivem ztráty lokality dat v mezipamětích (L1/L2 *cache*) a k nutnosti opakovaně načítat instrukce a tržní data z mnohem pomalejší hlavní paměti RAM. Pro dosažení striktně deterministického chování byla do párovacího jádra implementována podpora pro připnutí vlákna ke konkrétnímu procesorovému jádru (*CPU pinning*) pomocí funkce `pthread_setaffinity_np`. Tím je zajištěno, že kritické exekuční vlákno (které v rámci navržené architektury operuje výhradně jednovláknově) nemigruje napříč více jádry procesoru a může plně těžit z exkluzivního využití lokální mezipaměti.

Pro dosažení absolutní izolace byla navíc využita rekonfigurace jádra operačního systému na úrovni zavaděče GRUB. Pomocí parametru `isolcpus` jsou vybraná jádra zcela vyjmuta z kompetence standardního plánovače úloh. Tím se minimalizuje vliv asynchronních přerušení od ostatních uživatelských procesů či vláken OS a výrazně se snižuje počet vynucených přepnutí kontextu (*context switches*). Kromě toho byl aplikován parametr `nohz_full`, který na izolovaných jádrech deaktivuje pravidelná přerušení časovače (*tickless kernel*), čímž se eliminuje další významný zdroj mikrosekundového zpoždění (*jitter*). Konkrétní nastavení parametrů v konfiguraci zavaděče GRUB ilustruje výpis 6.12:

**Výpis kódu 6.12** Konfigurace zavaděče GRUB pro izolaci procesorových jader

```
GRUB_CMDLINE_LINUX_DEFAULT="quiet splash isolcpus=10,11 \
nohz_full=10,11"
```

### 6.3.3 Výsledky měření

Srovnání dosud nejvýkonnější architektury ploché limitní knihy s verzí využívající izolaci prostředí potvrdilo, že omezení vlivu operačního systému je pro determinismus aplikace zcela klíčové. Zatímco absolutní průměrná latence se u většiny operací změnila jen nepatrně, došlo k významné stabilizaci celého systému a k výraznému poklesu latence na nejvyšších percentilech (*tail latency*).

Tabulka 6.7 zachycuje výsledky mikrobenchmarků běžících v izolovaném prostředí. Oproti předchozí verzi vedla striktní izolace k mírným odchylkám absolutní latence v rozsahu od zlomků po jednotky nanosekund (např. nárůst u `MatchSingle` z 26,9 ns na 29,3 ns). Jednoznačná příčina tohoto jevu není známa. Vzhledem k tomu, že se tyto rozdíly pohybují na hranici přesnosti měření, lze tento rozptyl přisoudit přirozenému šumu a zásahu do interní správy testovací knihovny, jejíž výchozí plánování prostředků mohlo být pro

tyto specifické a izolované zátěže efektivnější. Jak ovšem dokládají následující měření, toto marginální kolísání je plně kompenzováno dosažením celkové stability a deterministického chování systému pod reálnou zátěží.

**Tabulka 6.7** Výsledky mikrobenchmarků s aktivní izolací prostředí

| Benchmark | Rozsah | Latence | Složitost |
|---|---|---|---|
| `InsertSamePrice` | 1 000 000 | 34,6 ns | $\mathcal{O}(1)$ |
| `InsertNewPrice` | 1 000 000 | 35,4 ns | $\mathcal{O}(1)$ |
| `CancelPartial` | 50 000 | 6,1 ns | $\mathcal{O}(1)$ |
| `CancelFull` | 50 000 | 6,7 ns | $\mathcal{O}(1)$ |
| `MatchSingle` | 1 000 000 | 29,3 ns | $\mathcal{O}(1)$ |
| `MatchSweep` | 50 000 | 1,2 ms | $\mathcal{O}(n)$ |
| `FullWorkload` | 1 000 000 | 32,5 ns | $\mathcal{O}(1)$ |

Skutečný dopad optimalizací na úrovni OS se naplno projevil v makrobenchmarcích. Jak dokládá tabulka 6.8 reprezentující zátěž na reálných datech z portálu LOBSTER, systém dokázal zpracovávat přes 33 milionů zpráv za sekundu (nárůst o cca 9 % oproti neizolované verzi). Nejvýznamnějším přínosem této iterace je ovšem eliminace výkyvů latence (*jitter*). V případě datasetu MSFT klesla maximální zaznamenaná latence více než osminásobně, a to ze 143 µs na 17,5 µs. Na hladině percentilu 99,9 se latence systému u většiny testovaných instrumentů dostala poprvé pod hranici 1 µs (a u nejrozsáhlejší sady MSFT se k ní těsně přiblížila).

**Tabulka 6.8** Propustnost a latence po izolaci prostředí (LOBSTER data)

| Metrika | AAPL (lvl 10) | AMZN (lvl 10) | MSFT (lvl 10) |
|---|---|---|---|
| Počet zpráv | 365 401 | 258 329 | 635 351 |
| Míra spárování (%) | 10,20 | 7,61 | 5,26 |
| Propustnost – bez měření (mil./s) | 28,97 | 30,13 | 33,32 |
| Propustnost – s měřením (mil./s) | 8,70 | 8,80 | 8,94 |
| Průměrná latence (ns) | 75 | 74 | 72 |
| Minimální latence (ns) | 40 | 40 | 40 |
| Percentil 50 (ns) | 70 | 70 | 70 |
| Percentil 90 (ns) | 100 | 90 | 90 |
| Percentil 99 (ns) | 300 | 270 | 150 |
| Percentil 99,9 (ns) | 811 | 941 | 1 281 |
| Maximální latence (ns) | 10 373 | 9 693 | 17 525 |

Úspěšnost izolace a afinity potvrzují i testy se syntetickou zátěží (tabulka 6.9). U testu s 50% mírou rušení objednávek, jenž v předchozích iteracích představoval jeden z nejnáročnějších scénářů, vzrostla čistá propustnost oproti minulé verzi z 24,88 na 36,46 mil./s. Hodnota percentilu 99,9 se napříč syntetickými scénáři stabilizovala v rozmezí 120 až 270 ns a maximální zaznamenaná odchylka napříč statisíci operacemi nepřesáhla 6,6 µs (oproti 190 µs v předchozí verzi).

**Tabulka 6.9** Propustnost a latence po izolaci prostředí (syntetická data)

| Metrika | 100 % vkládání | 50 % rušení | 69 % párování |
|---|---|---|---|
| Počet zpráv | 100 000 | 100 000 | 100 000 |
| Míra spárování (%) | 0,00 | 0,00 | 69,49 |
| Propustnost – bez měření (mil./s) | 28,88 | 36,46 | 18,43 |
| Propustnost – s měřením (mil./s) | 8,83 | 9,40 | 7,53 |
| Průměrná latence (ns) | 73 | 67 | 93 |
| Minimální latence (ns) | 60 | 40 | 40 |
| Percentil 50 (ns) | 70 | 70 | 90 |
| Percentil 90 (ns) | 80 | 80 | 130 |
| Percentil 99 (ns) | 90 | 90 | 200 |
| Percentil 99,9 (ns) | 130 | 120 | 270 |
| Maximální latence (ns) | 6 479 | 6 579 | 6 338 |

Výsledky přesvědčivě prokazují, že nasazení plochých datových struktur v součinnosti s `mlockall` a afinitou k hardwarově izolovanému procesorovému jádru tvoří vysoce synergický celek. Aplikace tak neztrácí procesorový čas obsluhou výpadků stránek, nečeká na přesuny dat v paměti a operační systém nevyřazuje její exekuční vlákno z plánovače úloh. Výsledkem je párovací jádro, které dosahuje vysoce deterministické latence i v náročných zátěžových podmínkách.

## 6.4 Nízkoúrovňové optimalizace

Ačkoliv vhodně zvolená architektura a izolace operačního systému tvoří nezbytný základ pro dosažení deterministicky nízké latence a maximální propustnosti, k dosažení absolutní efektivity je nutné aplikovat nízkoúrovňové techniky zohledňující specifika hardwaru. Tato podkapitola se zaměřuje na mikrooptimalizace kódu, které minimalizují režii přímo na úrovni procesoru.

### 6.4.1 Zarovnání pro CPU mezipaměť

Jak je popsáno v sekci 3.2.1, moderní procesory nenačítají data z RAM po jednotlivých bajtech, ale v ucelených blocích nazývaných mezipaměťové linky (*cache lines*). Na použitém testovacím stroji bylo pomocí příkazu `getconf LEVEL1_DCACHE_LINESIZE` ověřeno, že velikost jedné linky je 64 B. Pokud datová struktura překračuje hranici této linky (*cache line straddling*), je procesor nucen provést dvě paměťová čtení namísto jednoho, což do kritické cesty vnáší nezanedbatelné zpoždění.

Základní datovou entitou celého párovacího jádra je struktura `Order`. Aby byla zajištěna její optimální lokalita a zamezilo se nezarovnanému přístupu k paměti, byl původní datový model rozdělen na dvě části. Vstupní zprávy z vnějšího světa nově zapouzdřuje struktura `OrderMessage`. Naproti tomu interní struktura `Order` byla paměťově redukována pouze na atributy nezbytné pro párování, které byly doplněny o metadata pro vnitřní správu limitní knihy (`OrderBook`). Tato interní reprezentace objednávky je pak pomocí klíčového slova `alignas(32)` explicitně zarovnána a fixována přesně na velikost 32 B.

**Výpis kódu 6.13** Explicitní zarovnání struktury `Order` na hranici 32 B

```
struct OrderMessage {
    Timestamp timestamp;
    OrderId orderId;
    Price price;
    Quantity quantity;
    EventType eventType;
    Side side;
};

struct alignas(32) Order {
    Timestamp timestamp;
    OrderId orderId;
    Price price;
    Quantity quantity;
    OrderIndex nextOrderIdx;
};

static_assert(sizeof(Order) == 32,
    "Order struct not aligned with cache line!");
```

Tato mikrooptimalizace garantuje, že se do jedné 64 B mezipaměťové linky vejdou přesně dvě objednávky bez jakéhokoliv paměťového přesahu. Kontejner reprezentující paměťový fond (`std::vector`) navíc díky tomuto pravidlu zarovná celý paměťový blok tak, že počáteční objednávka začíná přesně na

začátku *cache line*. Klíčové slovo `static_assert` zde slouží jako ochranný mechanismus, který zastaví kompilaci v případě, že by budoucí úprava struktury toto zarovnání porušila.

Měření vlivu této úpravy ukázalo, jak kritické je pro HFT systémy zarovnání dat. Tabulka 6.10 prezentuje průběžné výsledky mikrobenchmarků. Oproti nezarovnané verzi došlo k významnému poklesu absolutních časů u většiny operací. Čas operace `MatchSingle` klesl z 29,3 ns na 24,1 ns. Stejný trend lze pozorovat i u celkového syntetického zatížení (`FullWorkload`), které zrychlilo z 32,5 ns na 26,7 ns. Obě operace rušení objednávek (`CancelPartial` i `CancelFull`) navíc díky těmto úpravám stabilně pokořily hraniční mez 6 ns.

**Tabulka 6.10** Výsledky mikrobenchmarků po paměťovém zarovnání

| Benchmark | Rozsah | Latence | Složitost |
|---|---|---|---|
| `InsertSamePrice` | 1 000 000 | 26,7 ns | $\mathcal{O}(1)$ |
| `InsertNewPrice` | 1 000 000 | 29,7 ns | $\mathcal{O}(1)$ |
| `CancelPartial` | 50 000 | 5,9 ns | $\mathcal{O}(1)$ |
| `CancelFull` | 50 000 | 5,9 ns | $\mathcal{O}(1)$ |
| `MatchSingle` | 1 000 000 | 24,1 ns | $\mathcal{O}(1)$ |
| `MatchSweep` | 50 000 | 1,18 ms | $\mathcal{O}(n)$ |
| `FullWorkload` | 1 000 000 | 26,7 ns | $\mathcal{O}(1)$ |

Díky faktu, že procesor již nemusí zbytečně čekat na dodatečné načítání dat z nižších úrovní paměti, se toto plošné zrychlení operací pozitivně propsalo i do propustnosti a latencí nad reálnými LOBSTER daty. Kompletní vyhodnocení makrobenchmarků, které shrnuje vliv všech popsaných nízkoúrovňových optimalizací, je detailně rozebráno v závěrečné sekci této kapitoly.

### 6.4.2 Eliminace instrukčních skoků

Dalším logickým krokem k maximalizaci propustnosti a dosažení deterministicky nízké latence byla snaha o eliminaci instrukčních skoků a větvení na kritické cestě (*branchless programming*). K dosažení tohoto cíle bylo implementováno a testováno několik nezávislých technik.

Prvním pokusem bylo odstranění rozhodovacího bloku `switch` v metodě `processOrder`, který na základě typu události (`EventType`) volá příslušné metody limitní knihy. Toto větvení bylo nahrazeno tabulkou ukazatelů na funkce (*dispatch table*). Dále došlo ke sjednocení metod `processBuyOrder` a `processSellOrder` do jediné šablonované funkce, což mělo snížit redundanci kódu a zefektivnit překlad. Jak ovšem ukázaly mikro a makro benchmarky, žádná z těchto úprav nevedla ke zvýšení výkonu celého párovacího jádra. Moderní procesory disponují vysoce pokročilými prediktory skoků (*branch predictors*), které klasický `switch` predikují s téměř stoprocentní úspěšností, zatímco

nepřímá volání přes ukazatele na funkce (*indirect branches*) mohou úspěšnou predikci naopak ztížit.

Následně byl testován explicitní softwarový *prefetching* pomocí intrinsické funkce `_mm_prefetch`, který měl za úkol přednačítat data z paměti do L1/L2 mezipaměti ještě před jejich zpracováním. Ačkoliv tato technika v izolovaných mikrobenchmarcích přinesla zrychlení v řádu zlomků nanosekund, v reálném zátěžovém testu nad LOBSTER daty paradoxně způsobila pokles celkové propustnosti přibližně o 5 %. Tento jev je typickým důsledkem zahlcení paměťové sběrnice příliš agresivním přednačítáním.

Posledním experimentem v této oblasti bylo nasazení kompilátorových atributů `[[likely]]` a `[[unlikely]]` pro explicitní nápovědu větvení. I v tomto případě zůstaly výsledky syntetických testů beze změny a u reálných dat došlo k nepatrnému propadu propustnosti. Veškeré tyto techniky tak neprokázaly na cílovém hardwaru požadovaný efekt, byly zavrženy a jako referenční bod byla ponechána předchozí iterace s explicitním zarovnáním.

### 6.4.3 Optimalizace správy paměťového fondu

Jelikož standardní kontejner `std::vector` přináší nezanedbatelnou míru abstrakce a s ní spojenou režii při interní správě, byl proveden zásah přímo do jádra alokačního mechanismu. Původní řešení spoléhalo na separátní vektor volných indexů (`freeIndices`). Tento dodatečný kontejner byl zcela odstraněn a nahrazen návrhovým vzorem *free list*.

Správa uvolněných paměťových bloků je nyní realizována pouze pomocí jediné stavové proměnné `firstFreeIdx`. Pokud je objednávka z knihy odstraněna, její původní datová struktura `Order` v poli `ordersPool` je využita k uložení ukazatele na další volný blok v řadě (výpis 6.14). Tím je docíleno toho, že systém nevyužívá žádnou dodatečnou paměť pro evidenci volného místa.

**Výpis kódu 6.14** Správa volných bloků pro objednávky pomocí vzoru *free list*

```
OrderIndex OrderBook::allocateOrder(const Order& order) {
    if (firstFreeIdx != INVALID_INDEX) {
        OrderIndex idx = firstFreeIdx;
        firstFreeIdx = ordersPool[idx].nextOrderIdx;

        ordersPool[idx] = order;
        ordersPool[idx].nextOrderIdx = INVALID_INDEX;

        return idx;
    }

    if (ordersPool.size() >= ordersPool.capacity())
        throw std::runtime_error(
            "OrderBook memory pool exhausted!");

    ordersPool.push_back(order);
    ordersPool.back().nextOrderIdx = INVALID_INDEX;
    return static_cast<OrderIndex>(ordersPool.size() - 1);
}

void OrderBook::freeOrder(OrderIndex index) {
    ordersPool[index].nextOrderIdx = firstFreeIdx;
    firstFreeIdx = index;
}
```

Odstranění dynamické správy dodatečného vektoru a snížení místa zabíraného v mezipaměti (*cache footprint*) přineslo výrazný skok ve výkonnosti. Jak ukazuje tabulka 6.11, kromě operace částečného rušení objednávky (kde se čas vlivem změny struktury paměti posunul z 5,9 ns na stále velmi nízkých 6,8 ns), došlo k plošnému zrychlení napříč všemi operacemi.

Vkládání nových objednávek zrychlilo přibližně o 2 ns. Výrazného zlepšení dosáhla kritická operace párování `MatchSingle`, která se dostala na 22,2 ns. Metrika komplexního zatížení (`FullWorkload`) navíc klesla na finálních 24,4 ns. Tato optimalizace jednoznačně potvrzuje, že pro HFT systémy je striktní redukce paměťových manipulací a snižování počtu pomocných a stavových proměnných klíčem k absolutnímu výkonu.

**Tabulka 6.11** Výsledky mikrobenchmarků po nasazení vzoru *free list*

| Benchmark | Rozsah | Latence | Složitost |
|---|---|---|---|
| `InsertSamePrice` | 1 000 000 | 24,8 ns | $\mathcal{O}(1)$ |
| `InsertNewPrice` | 1 000 000 | 28,4 ns | $\mathcal{O}(1)$ |
| `CancelPartial` | 50 000 | 6,8 ns | $\mathcal{O}(1)$ |
| `CancelFull` | 50 000 | 5,9 ns | $\mathcal{O}(1)$ |
| `MatchSingle` | 1 000 000 | 22,2 ns | $\mathcal{O}(1)$ |
| `MatchSweep` | 50 000 | 1,0 ms | $\mathcal{O}(n)$ |
| `FullWorkload` | 1 000 000 | 24,4 ns | $\mathcal{O}(1)$ |

### 6.4.4 Optimalizační příznaky kompilace

Závěrečnou fází nízkoúrovňových optimalizací bylo nasazení pokročilých kompilátorových technik využívajících hlubokou analýzu kódu a specifika cílové architektury. Prvním krokem byla aplikace příznaku `-march=native`. Ten instruuje překladač, aby vygenerovaný strojový kód striktně optimalizoval pro fyzický procesor, na kterém probíhá překlad. Kompilátor díky tomu nemusí zachovávat zpětnou kompatibilitu se staršími x86 architekturami a může naplno využít moderní vektorové instrukce a hardwarová specifika testovacího stroje. Jak ukázala měření, pouhé zapnutí tohoto příznaku snížilo čas u kombinované zátěže (`FullWorkload`) na 22,4 ns a urychlilo párování (`MatchSingle`) na 20,6 ns.

Dalším krokem byla aktivace LTO. Při standardním překladu kompilátor optimalizuje každý zdrojový `.cpp` soubor izolovaně. LTO tento proces odkládá až do fáze linkování, kdy má kompilátor k dispozici kompletní pohled na celou aplikaci. To umožňuje agresivní vkládání těl funkcí (*inlining*) napříč různými třídami a soubory, čímž se zcela eliminuje režie na volání funkcí. Nasazení LTO vedlo k výraznému poklesu latence zejména u operací rušení objednávek – čas operace `CancelPartial` klesl ze 6,8 ns na 1,7 ns, což znamená výrazné zrychlení o téměř 75 %.

K dosažení absolutního technologického maxima byla k LTO navíc přidána profilem řízená optimalizace (PGO). Jak bylo zjištěno v předchozích experimentech, manuální pokusy o nápovědu prediktoru skoků pomocí atributů `[[likely]]` nevedly k úspěchu. PGO tento problém řeší automatizovaně ve třech krocích, které byly pro testovací běh s překladačem GCC 15 provedeny podle následujícího postupu (výpis 6.15).

**Výpis kódu 6.15** Proces generování, tréninku a využití PGO profilu

```
# 1. Kompilace párovacího jádra s instrumentačními senzory
CXX=g++-15 CC=gcc-15 cmake -B cmake-build-pgo \
    -DCMAKE_BUILD_TYPE=Release -DENABLE_PGO_GENERATE=ON
cmake --build cmake-build-pgo --target matching_engine -j

# 2. Běh nad nejrozsáhlejším LOBSTER datasetem pro zápis profilu
cd cmake-build-pgo
./matching_engine -i \
    ../data/LOBSTER/MSFT/MSFT_2012-06-21_message_lvl10.csv \
    -m -c 11
cd ..

# 3. Finální rekompilace s využitím profilu
cmake --build cmake-build-pgo --target clean
CXX=g++-15 CC=gcc-15 cmake cmake-build-pgo \
    -DENABLE_PGO_GENERATE=OFF -DENABLE_PGO_USE=ON
cmake --build cmake-build-pgo --target \
    matching_engine engine_benchmarks -j
```

Jak dokládají výsledky mikrobenchmarků (tabulka 6.12), kombinace těchto tří technik přinesla rekordní výsledky. Tabulka zaznamenává ustálenou časovou konstantu pro $\mathcal{O}(1)$ operace, aby hodnoty nebyly zkresleny fyzickým zahlcením nižších vrstev mezipaměti při větším počtu prvků. Zásadní je propad časů u operací vkládání (`InsertSamePrice`), které po aplikaci PGO klesly o dalších bezmála 30 % (z 25,2 ns na 17,7 ns). K absolutnímu zrychlení došlo u úplného zrušení objednávky (`CancelFull`), které ve finálním sestavení trvá pouhých 1,6 ns.

Za pozornost stojí zdánlivě paradoxní zhoršení latence u operace částečného rušení po aplikaci PGO (z 1,7 ns na 4,0 ns). Tento jev ovšem není chybou, ale přímým důkazem správného fungování profilování. Z analýzy trénovacích dat překladač vyhodnotil, že částečné rušení objednávek se na reálném trhu vyskytuje statisticky méně často než vkládání nových objednávek. Tuto logiku proto přesunul do nekritické (tzv. studené) sekce binárního souboru, čímž uvolnil omezenou kapacitu instrukční mezipaměti (*I-cache*) pro instrukce na kritické cestě. Tento kompromis vedl k žádoucímu zrychlení celkové kombinované zátěže, jejíž latence se zlepšila o dalších 18 % a klesla z 22,0 ns na finálních 18,1 ns.

**Tabulka 6.12** Vliv kompilátorových optimalizací na konstantní operace

| Benchmark | `-march=native` | + LTO | + PGO |
|---|---|---|---|
| `InsertSamePrice` | 23,5 ns | 25,2 ns | 17,7 ns |
| `InsertNewPrice` | 28,5 ns | 29,9 ns | 21,3 ns |
| `CancelPartial` | 6,7 ns | 1,7 ns | 4,0 ns |
| `CancelFull` | 6,8 ns | 2,2 ns | 1,6 ns |
| `MatchSingle` | 20,6 ns | 19,8 ns | 19,8 ns |
| `FullWorkload` | 22,4 ns | 22,0 ns | 18,1 ns |

### 6.4.5 Výsledky měření

Srovnání předchozí iterace využívající izolaci běhového prostředí s touto finální iterací, do které byly implementovány popsané nízkoúrovňové techniky (zarovnání struktur do mezipaměťových linek, využití vzoru *free list* a optimalizační kompilátorové příznaky), jasně demonstruje nezbytnost těchto zásahů pro systémy HFT. Zatímco OS izolace dodala systému determinismus, teprve mikrooptimalizace kódu v součinnosti s překladačem dokázaly odemknout maximální fyzický potenciál procesoru.

Tabulka 6.13 zachycuje výsledky mikrobenchmarků finální verze jádra. Oproti původní izolované verzi (tabulka 6.7 v předchozí kapitole) došlo u naprosté většiny operací k propadu latence o desítky procent. Syntetická komplexní zátěž (`FullWorkload`) zrychlila z 32,5 ns na 18,1 ns. Výrazným případem je operace zrušení celé objednávky (`CancelFull`), která vlivem uvolnění instrukční mezipaměti a agresivního vkládání (*inlining*) klesla ze 6,7 ns na pouhých 1,6 ns.

**Tabulka 6.13** Výsledky měření operací finální implementace

| Benchmark | Rozsah | Latence | Složitost |
|---|---|---|---|
| `InsertSamePrice` | 1 000 000 | 17,7 ns | $\mathcal{O}(1)$ |
| `InsertNewPrice` | 1 000 000 | 21,3 ns | $\mathcal{O}(1)$ |
| `CancelPartial` | 50 000 | 4,0 ns | $\mathcal{O}(1)$ |
| `CancelFull` | 50 000 | 1,6 ns | $\mathcal{O}(1)$ |
| `MatchSingle` | 1 000 000 | 19,8 ns | $\mathcal{O}(1)$ |
| `MatchSweep` | 50 000 | 0,99 ms | $\mathcal{O}(n)$ |
| `FullWorkload` | 1 000 000 | 18,1 ns | $\mathcal{O}(1)$ |

Skutečný přínos nízkoúrovňových optimalizací se naplno projevil v makrobenchmarcích. Jak dokládá tabulka 6.14 reprezentující zátěž nad reálnými daty z portálu LOBSTER, systém překonal dosavadní hranici a dokázal zpracovat 51,06 milionu zpráv za sekundu u datasetu MSFT (nárůst o cca 53 %

oproti předchozí iteraci s 33,32 mil./s). Současně došlo k plošnému snížení latencí. Mediánová latence (50. percentil) klesla na 60 ns. Na hladině percentilu 99,9 se systém u nejvytíženějšího datasetu MSFT bezpečně usadil těsně nad hranicí 1 µs a maximální latence nepřesáhla 16 µs.

**Tabulka 6.14** Propustnost a latence finální implementace (LOBSTER data)

| **Metrika** | **AAPL (lvl 10)** | **AMZN (lvl 10)** | **MSFT (lvl 10)** |
|---|---|---|---|
| Počet zpráv | 365 401 | 258 329 | 635 351 |
| Míra spárování (%) | 10,20 | 7,61 | 5,26 |
| Propustnost – bez měření (mil./s) | 42,36 | 43,99 | 51,06 |
| Propustnost – s měřením (mil./s) | 9,52 | 9,56 | 9,83 |
| Průměrná latence (ns) | 65 | 65 | 62 |
| Minimální latence (ns) | 40 | 40 | 40 |
| Percentil 50 (ns) | 60 | 60 | 60 |
| Percentil 90 (ns) | 80 | 80 | 70 |
| Percentil 99 (ns) | 260 | 240 | 120 |
| Percentil 99,9 (ns) | 711 | 821 | 1 091 |
| Maximální latence (ns) | 10 227 | 8 805 | 15 737 |

Tato vysoká výkonnost je konzistentní i napříč všemi testy se syntetickou zátěží (tabulka 6.15). U scénáře sestávajícího výhradně ze zakládání nových objednávek dosáhlo jádro absolutního maxima propustnosti 55,11 mil./s. U náročného testu s 50% mírou rušení objednávek (který těží z PGO detekce „studených“ větví kódu a správy volných indexů pomocí *free list*) jádro dosáhlo propustnosti 52,52 mil./s, což je strmý nárůst oproti původním 36,46 mil./s. Zásadní je také vliv na stabilitu latence, hodnota percentilu 99,9 u scénáře s 50% rušením klesla na pouhých 90 ns a ani u nejnáročnějšího párovacího scénáře nepřesáhla 230 ns.

**Tabulka 6.15** Propustnost a latence finální implementace (syntetická data)

| Metrika | 100 % vkládání | 50 % rušení | 69 % párování |
|---|---|---|---|
| Počet zpráv | 100 000 | 100 000 | 100 000 |
| Míra spárování (%) | 0,00 | 0,00 | 69,49 |
| Propustnost – bez měření (mil./s) | 55,11 | 52,52 | 24,88 |
| Propustnost – s měřením (mil./s) | 10,35 | 10,14 | 8,29 |
| Průměrná latence (ns) | 57 | 59 | 81 |
| Minimální latence (ns) | 40 | 40 | 40 |
| Percentil 50 (ns) | 60 | 60 | 70 |
| Percentil 90 (ns) | 60 | 70 | 110 |
| Percentil 99 (ns) | 70 | 80 | 170 |
| Percentil 99,9 (ns) | 100 | 90 | 230 |
| Maximální latence (ns) | 7 248 | 6 945 | 10 841 |

Výsledky nepopiratelně prokazují, že datově orientovaný návrh má v jazyce C++ naprosto zásadní vliv na výkonnost výsledné aplikace. Zarovnání datových entit pro 64 B mezipaměťové linky, minimalizace paměťové stopy odstraněním režie pomocných kontejnerů a předání nasbíraných metrik z běhů pokročilému kompilátoru transformovalo navržené párovací jádro do stavu, kdy je jeho výkon limitován již výhradně fyzickou propustností použitého hardwaru.

Kapitola 7

# Srovnání a zhodnocení výsledků měření

*Tato závěrečná kapitola shrnuje a vyhodnocuje vliv jednotlivých vývojových iterací a použitých optimalizačních technik na celkový výkon párovacího jádra, a to z hlediska propustnosti a latence zpracování objednávek. Následně srovnává výkon finální verze jádra s výsledky referenčních implementací. Na závěr navrhuje možnosti dalšího rozvoje implementovaného řešení.*

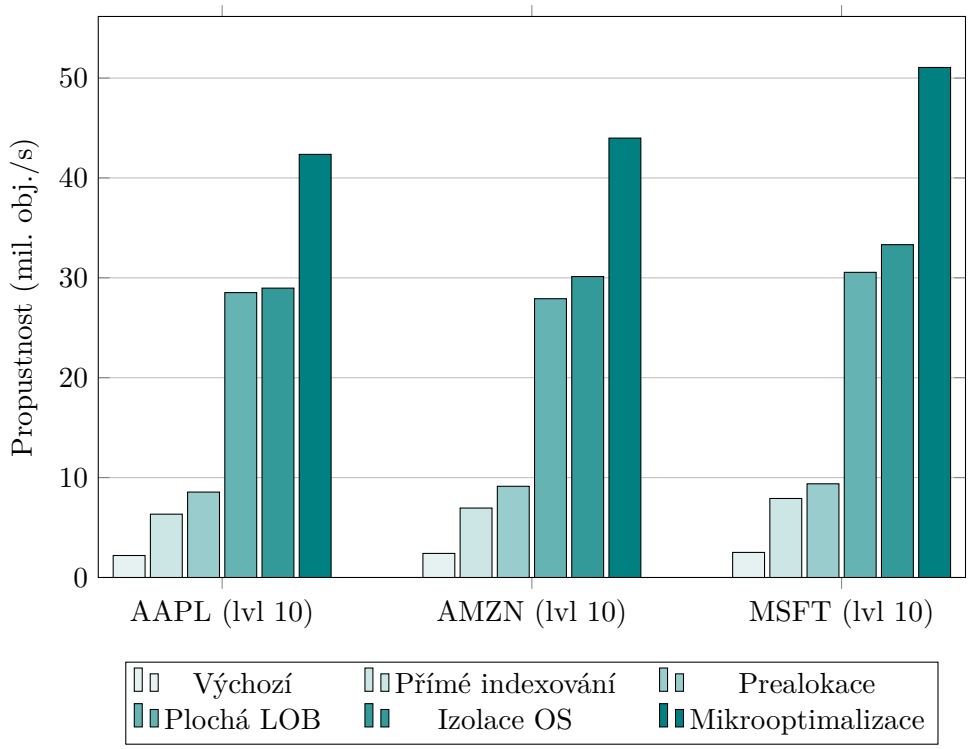
## 7.1 Vyhodnocení propustnosti a latence

V této sekci jsou vyhodnoceny výsledky měření výkonu pro jednotlivé iterace vývoje párovacího jádra. Veškerá měření probíhala v prostředí popsaném v kapitole 4, za využití technik pro zajištění jeho maximálního determinismu, stability a reprodukovatelnosti.

Pro komplexní zhodnocení výkonu byly zvoleny dva primární přístupy: testování na reálných historických datech z platformy LOBSTER (konkrétně pro instrumenty AAPL, AMZN a MSFT o hloubce trhu 10 úrovní) a testování na syntetických deterministicky generovaných datech pokrývajících náročné scénáře.

### 7.1.1 Srovnání propustnosti na LOBSTER datech

Čistá propustnost byla měřena na izolovaných bězích bez aktivního měření latence. Datasety obsahují statisíce událostí s rozložením vložení, rušení a párování objednávek, které přesně odpovídá reálnému chování trhu. Výsledky zachycuje obrázek 7.1.




**Obrázek 7.1** Propustnost verzí jádra na LOBSTER datech

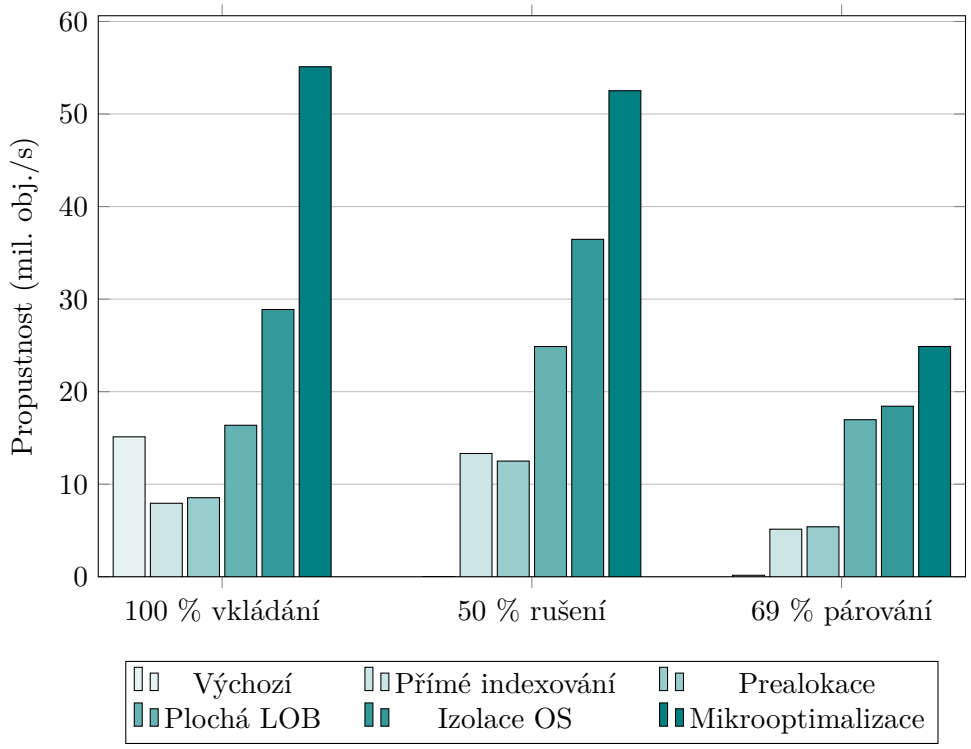
Jak z grafu vyplývá, výchozí implementace na těchto datech dosahovala propustnosti pouze v rozmezí 2,2 až 2,5 milionu operací za sekundu. Zavedení přímého indexování objednávek přineslo první významný nárůst výkonu (téměř trojnásobné zrychlení), který byl následně podpořen odstraněním dynamických alokací na kritické cestě pomocí paměťových fondů, čímž se propustnost přiblížila hranici 9,4 milionu zpráv za sekundu.

Zcela zásadním zlomem je přechod na plochou reprezentaci limitní knihy. Díky eliminaci iterování přes uzly a maximalizaci využití procesorové mezipaměti vzrostl výkon skokově na přibližně 30 milionů zpráv za sekundu. Izolace operačního systému, ačkoliv cílí především na minimalizaci výkyvů latence, přinesla díky eliminaci přepínání kontextů další mírné zlepšení propustnosti.

Poslední fáze vývoje, zahrnující pokročilé nízkoúrovňové optimalizace a nasazení PGO, umožnila efektivně využít plný potenciál hardwaru a maximalizovat celkovou propustnost systému. V případě instrumentu MSFT bylo dosaženo hodnoty přes 51 milionů operací za sekundu, což ve výsledku znamená více než dvacetinásobné zrychlení celého párovacího jádra oproti jeho výchozí verzi.

### 7.1.2 Srovnání propustnosti na syntetických datech

Měření na syntetických datech slouží k otestování chování párovacího jádra v hraničních, uměle vygenerovaných scénářích, které izolují konkrétní typy operací. Byly zvoleny tři specifické scénáře: 100% vkládání nových objednávek, scénář s 50% podílem rušení a vysoce aktivní trh s 69% podílem párování. Výsledky pro tyto scénáře shrnuje obrázek 7.2.




**Obrázek 7.2** Propustnost verzí jádra na syntetických datech

Naměřená data odhalují kritické nedostatky výchozí implementace. Zatímco čisté vkládání objednávek dosahovalo přijatelných 15,1 milionu zpráv za sekundu, při intenzivním rušení objednávek se výkon propadl na naprosté minimum (0,02 milionu). Tento kolaps byl způsoben nutností lineárního vyhledávání v limitní knize.

Zavedení přímého indexování tento defekt zcela eliminovalo. Ačkoliv počáteční režie hashovací tabulky mírně zpomalila vkládání na 7,9 milionu zpráv, propustnost při rušení vzrostla na 13,3 milionu zpráv za sekundu. Od tohoto bodu se výkon jádra ve všech scénářích téměř zcela zvyšoval.

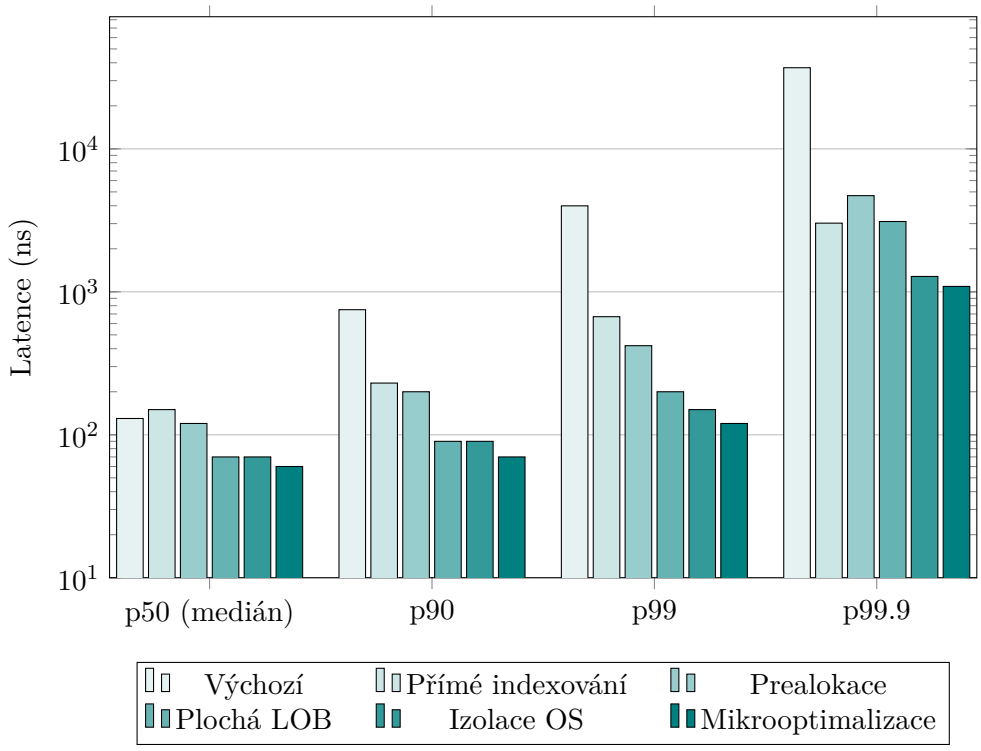
Finální verze jádra po aplikaci datově orientovaného návrhu, izolace prostředí a mikrooptimalizací vykazuje vysoký a stabilní výkon. Propustnost dosáhla u vkládání a rušení až 55,1, respektive 52,5 milionu zpráv za sekundu. Scénář častého párování, který je výpočetně nejnáročnější (vyžaduje modifi-

kaci stavu více objednávek a generování zpráv o obchodech), dosáhl hranice téměř 25 milionů operací za sekundu.

### 7.1.3 Srovnání distribuce latence a její stability

Zatímco propustnost definuje celkovou kapacitu systému, pro HFT je rozhodujícím faktorem především latence zpracování jednotlivých objednávek a minimalizace jejích výkyvů. Výrazné výkyvy v latenci (*jitter*) totiž v reálném nasazení vedou ke ztrátě obchodních příležitostí. Z tohoto důvodu není hlavním předmětem měření průměrná hodnota, jež je náchylná ke zkreslení a může maskovat ojedinělé výkonnostní propady, ale stabilita systému vyjádřená hodnotami na nejvyšších percentilech (*tail latency*).

Pro detailní analýzu chování latence byl z testovací sady záměrně vybrán instrument MSFT. Tento dataset obsahuje více než 635 tisíc událostí, čímž představuje nejvyšší výpočetní zátěž a nejlépe tak simuluje nárazové špičky na trhu. Výsledky měření na 50. (medián), 90., 99. a 99,9. percentilu zachycuje obrázek 7.3. Vzhledem k propastným rozdílům v hodnotách u neoptimalizovaných verzí je svislá osa pro lepší čitelnost vykreslena v logaritmickém měřítku.




**Obrázek 7.3** Distribuce latence zpracování objednávek u jednotlivých verzí jádra

Graf jasně ilustruje, že ačkoliv výchozí verze systému dosahovala relativně přijatelného mediánu 130 ns, její chování v hraničních situacích bylo silně ne-

deterministické. Na 99,9. percentilu se latence vyšplhala až k hranici 37 µs, což bylo způsobeno především výpadky stránek (*page faults*) a nutností dynamické alokace paměti během kritické cesty.

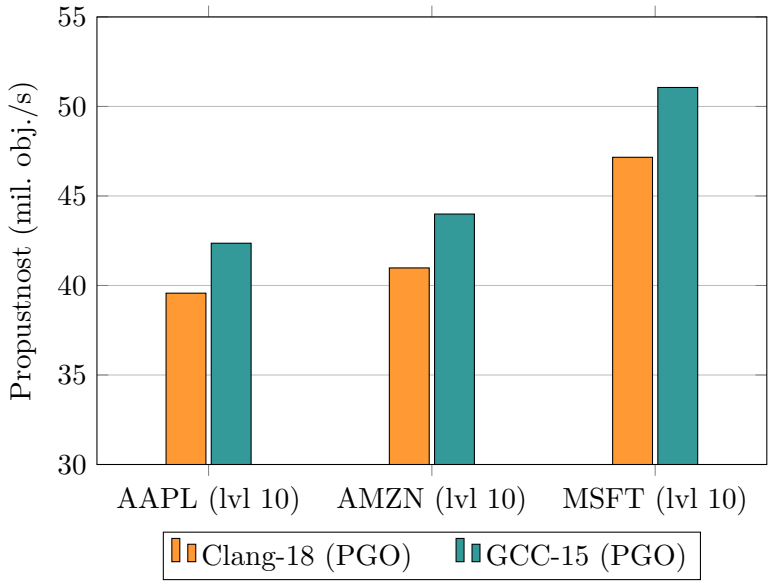
První tři iterace optimalizace (přímé indexování, prealokace a nasazení ploché limitní knihy) měly zásadní vliv primárně na celkovou efektivitu algoritmů a využití procesorové mezipaměti. Díky nim se podařilo snížit medián latence zhruba na polovinu (70 ns) a razantně srazit hodnoty na 90. a 99. percentilu (z 3 995 ns na 200 ns u p99). Na nejvyšším měřeném percentilu (p99.9) však systém stále vykazoval výkyvy překračující 3 µs, které byly způsobeny vnějšími zásahy operačního systému.

Klíčovým krokem pro stabilizaci výrazných výkyvů se proto stala izolace běhového prostředí. Uzamčení paměti a připnutí vlákna na izolované procesorové jádro stlačily hodnotu na 99,9. percentilu z 3 104 ns na 1 281 ns. Finální mikrooptimalizace pak latenci napříč celým spektrem ještě mírně vylepšily. Výsledné jádro tak dosahuje stabilního mediánu 60 ns. Hranici 1 µs jádro mírně překračuje až na 99,9. percentilu (1 091 ns). U běžně zatížených instrumentů, jako jsou AAPL a AMZN, zůstává latence i v těchto hraničních špičkách bezpečně pod 1 µs.

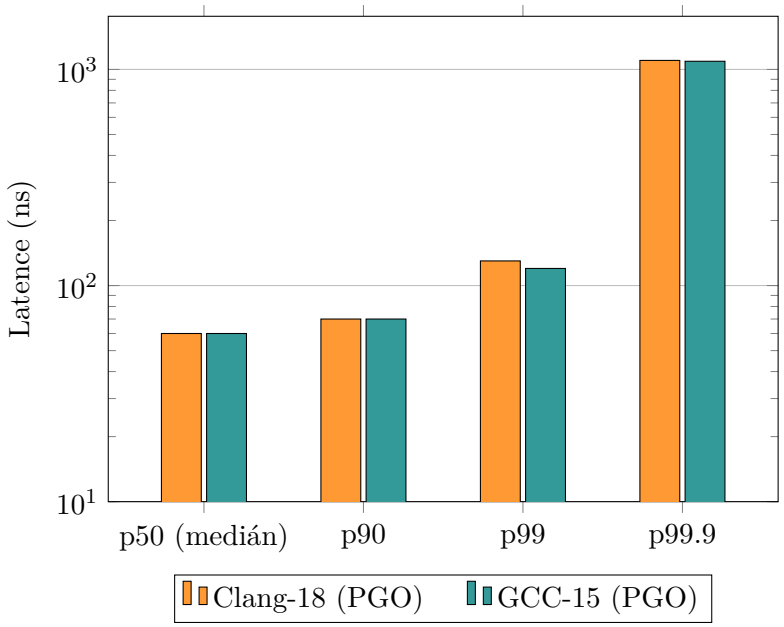
## 7.2 Vliv volby překladače na výsledný výkon

Ačkoliv jazyk C++ poskytuje abstrakce s nulovou režií (*zero-cost abstractions*), finální podoba strojového kódu a jeho interakce s procesorovou mezipamětí silně závisí na konkrétním překladači a jeho optimalizačních heuristikách. Pro ověření maximálního možného výkonu párovacího jádra na cílové architektuře x86_64 byl proveden srovnávací test mezi dvěma nejrozšířenějšími linuxovými kompilátory: GCC (verze 15) a Clang (verze 18). Obě varianty byly kompilovány s nejvyšším stupněm optimalizace a s odpovídajícím profilem PGO.

Jak ilustruje obrázek 7.4, kompilátor GCC dosahuje ve všech testovaných scénářích zhruba o 7 až 8 % vyšší celkové propustnosti než Clang. V případě nejvíce zatíženého instrumentu MSFT zpracovalo jádro zkompilované přes GCC přes 51 milionů objednávek za sekundu, zatímco Clang dosáhl hodnoty 47,16 milionu. Tento rozdíl na architektuře x86_64 bývá historicky připisován pokročilejšímu analyzátoru hodnot (*value profiling*) v rámci PGO a agresivnějším heuristikám pro vkládání funkcí (*function inlining*), díky kterým dokáže GCC snížit režii instrukční mezipaměti.




**Obrázek 7.4** Srovnání propustnosti podle kompilátoru (s PGO)




**Obrázek 7.5** Srovnání latence podle kompilátoru (s PGO)

Srovnání latence na obrázku 7.5 tento závěr doplňuje. Oba překladače zvládly optimalizovat kritické cesty kódu tak efektivně, že v 90 % případů je latence zpracování zcela identická (medián 60 ns, p90 70 ns). Rozdíly se začínají projevovat až v hraničních špičkách (*tail latency*). Zde si GCC udržuje mírný náskok, a to jak na 99. percentilu (120 ns oproti 130 ns), tak na 99,9.

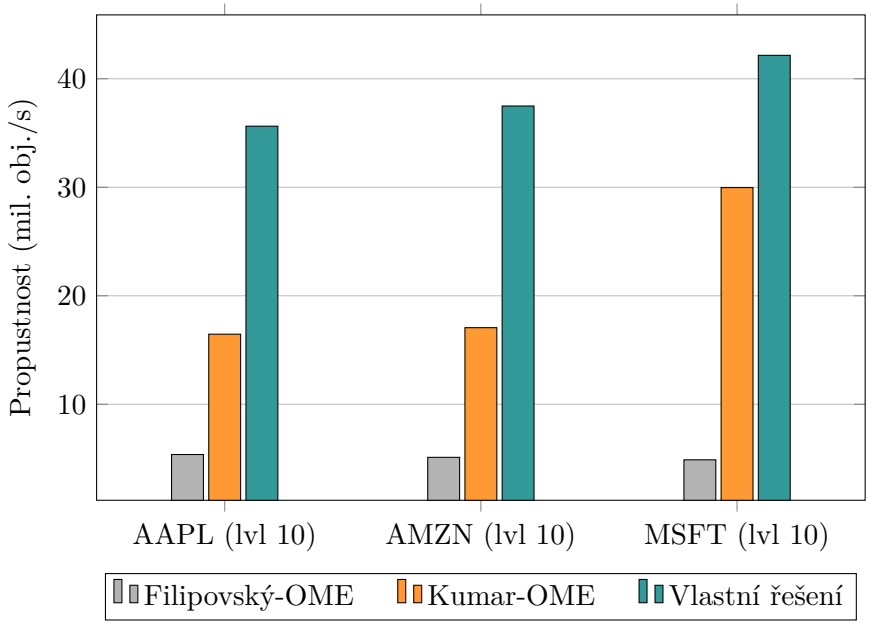
percentilu. Tyto rozdíly jsou však natolik minimální, že mohou být způsobeny i vnějšími faktory, jako je šum operačního systému.

Na základě těchto měření byl pro následující srovnání výkonu s referenčními implementacemi zvolen překladač GCC.

## 7.3 Srovnání s referenčními řešeními

Pro závěrečné srovnání výkonu byla zvolena referenční řešení představená v kapitole 2.6. Pro lepší čitelnost textu budou dále označována zkráceně jako Filipovský-OME a Kumar-OME. Výkonnostní testy probíhaly nad standardními datasety z platformy LOBSTER (úroveň 10).

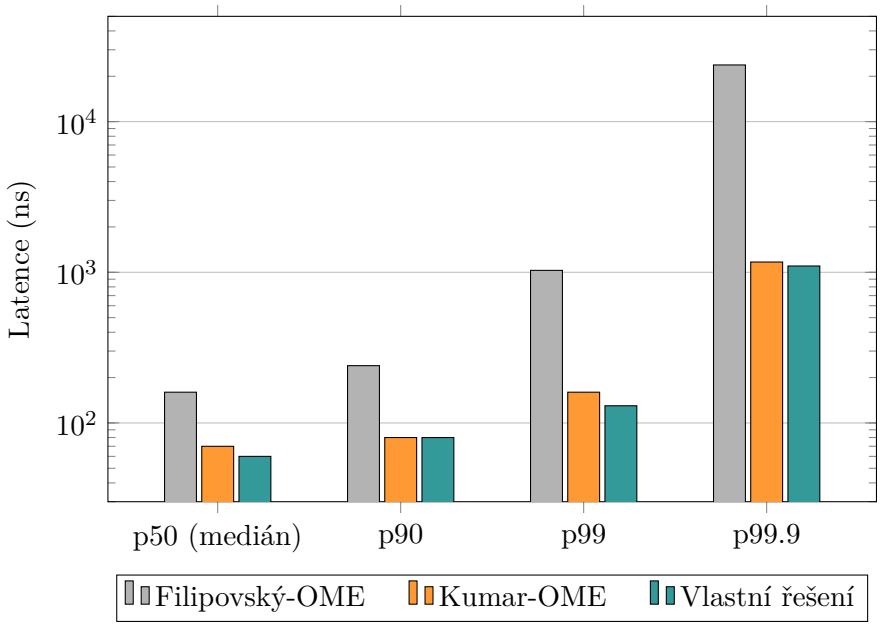
Jelikož referenční implementace nativně nepodporují kompilaci s využitím PGO, byla tato technika pro účely tohoto měření deaktivována i u vlastního řešení. Tím bylo zajištěno maximálně objektivní a férové srovnání vlivu softwarové architektury. Všechna tři párovací jádra byla kompilována překladačem GCC pro produkční nasazení (`CMAKE_BUILD_TYPE=Release`).




**Obrázek 7.6** Srovnání propustnosti vůči referenčním řešením (bez PGO)

Jak ukazuje obrázek 7.6, v oblasti celkové propustnosti vlastní řešení zcela překonává obě referenční implementace napříč všemi testovanými instrumenty. Zatímco řešení Filipovský-OME dosahuje propustnosti zpráv přibližně 5 milionů za sekundu, Kumar-OME vykazuje podstatně pokročilejší architekturu schopnou u nejrozsáhlejšího datasetu MSFT zpracovat téměř 30 milionů zpráv za sekundu. Vlastní řešení však posouvá hranici výkonu až na 42,16 milionu zpráv za sekundu. I bez použití profilové optimalizace je tedy zhruba o 40 %

rychlejší než vysoce optimalizovaný Kumar-OME a více než 8× rychlejší než Filipovský-OME.




**Obrázek 7.7** Srovnání latence vůči referenčním řešením (bez PGO)

Srovnání distribuce latence na nejrozsáhlejším datasetu MSFT nabízí pohled na stabilitu jednotlivých párovacích jader při reálné zátěži (obrázek 7.7). Filipovský-OME vykazuje značnou nestabilitu, přičemž na 99,9. percentilu jeho latence degraduje k hodnotám přesahujícím 23,7 µs.

Kumar-OME dosahuje velmi dobrých výsledků s mediánem 70 ns. Vlastní řešení si však i v tomto ohledu udržuje jasný náskok v rámci celé distribuce. Rozdíl, který začíná u mírně nižšího mediánu (60 ns), se dále prohlubuje v nejvyšších percentilech (*tail latency*). Například na 99. percentilu je vlastní jádro o 30 ns rychlejší. Naměřená data potvrzují, že vlastní řešení překonává referenční systémy jak v celkové propustnosti, tak v dosažení nižší a stabilnější latence.

## 7.4 Návrhy na budoucí rozvoj

Optimalizované a otestované párovací jádro je klíčovou součástí burzovního systému. V aktuálním stavu slouží jako ideální platforma pro výkonnostní testování a experimenty. Pro uplatnění v reálném provozu je však nutné rozšířit jeho funkcionalitu a nahradit některá zjednodušení, která byla záměrně přijata pro účely této práce. Hlavní oblasti, na které by se měl zaměřit budoucí vývoj, shrnuje následující přehled.

### Podpora párování více instrumentů

Aktuální implementace jádra nerozlišuje různé finanční instrumenty. Zpracovává pouze jednotný proud tržních událostí nad jednou limitní knihou. Intuitivním řešením by bylo přidat logiku pro směrování každé příchozí objednávky do příslušné knihy. Tento přístup by však zavedl nežádoucí větvení kódu a zbytečně by zatěžoval jedno procesorové jádro. Z hlediska výkonu je vhodnější architekturou využití techniky *CPU sharding*, kdy je každý instrument (případně skupina instrumentů) spravován nezávisle na vyhrazeném procesorovém jádře.

### Nasazení do síťového provozu

Stávající implementace pro účely maximálního výkonu při měření načítá testovací data přímo do operační paměti a obdobně zachází i s vygenerovanými obchody. Pro reálné nasazení je nezbytné jádro napojit na skutečný datový tok po síti. Interakce s okolím by měla probíhat asynchronně prostřednictvím vstupních a výstupních bezzámkových (*lock-free*) front, do kterých budou ostatní komponenty zapisovat nebo z nich číst. Tento přístup rovněž usnadní perzistentní logování událostí a efektivní distribuci tržních dat veřejnosti.

### Integrace dalších infrastrukturních komponent

Výkonné párovací jádro je nutné začlenit do širšího ekosystému burzovní infrastruktury. To předpokládá implementaci a integraci dalších nezbytných modulů, jako jsou vstupní a výstupní brány (*gateways*), systém pro komplexní správu objednávek (*order management system*) nebo modul pro předobchodní kontrolu rizik (*risk manager*).

### Rozšíření burzovní logiky a typů objednávek

Aktuální implementace se soustředí na maximální výkon při zpracování základních limitních objednávek s využitím prioritního algoritmu *price-time*. Pro plnohodnotné nasazení by bylo žádoucí rozšířit funkcionalitu o pokročilejší typy objednávek (např. *stop-loss*) a alternativní párovací algoritmy (např. *pro-rata*). Integrace této komplexnější obchodní logiky bude nicméně znamenat významnou inženýrskou výzvu z hlediska zachování dosažené deterministicky nízké latence a vysoké propustnosti.

# Kapitola 8
# Závěr

Tato práce si kladla za cíl navrhnout a implementovat vysoce výkonné párovací jádro burzovních objednávek a otestovat limity jeho datových struktur v jazyce C++. V souladu se zadáním byly nejprve zpracovány nezbytné teoretické podklady a stanovena exaktní metodika měření. Následná praktická část úspěšně prokázala, že pro náročné prostředí vysokofrekvenčního obchodování lze navrhnout řešení, které striktně dodržuje burzovní pravidla, aniž by tím utrpěl jeho nekompromisní výkon.

Vývoj jádra probíhal iterativně, jak bylo vytyčeno v cílech práce. Výchozí neoptimalizovaná implementace posloužila jako referenční platforma pro ověření funkční správnosti architektury a k následné identifikaci výkonnostních úzkých hrdel pomocí profilovacích nástrojů. Následná aplikace optimalizací probíhala krok za krokem – od eliminace dynamických alokací paměti, přes návrh plochých datových struktur pro maximální využití procesorové mezipaměti, až po nízkoúrovňové ladění afinity vláken a izolaci jader v operačním systému Linux. U každé vývojové iterace byl detailně zhodnocen a změřen její výkonnostní přínos vůči předchozímu stavu.

Závěrečná analýza naměřených výsledků dokládá, že finální optimalizovaná verze jádra svým výkonem nejen řádově překonala počáteční naivní implementaci, ale dosáhla výrazně lepších výsledků i ve srovnání s dostupnými referenčními řešeními. V exaktním testu bez profilově řízené optimalizace dosáhl navržený systém propustnosti přes 42 milionů zpráv za sekundu, čímž o zhruba 40 % překonal nejpokročilejší cizí referenční jádro. Po aplikaci profilově řízené optimalizace překladače GCC přesáhla celková propustnost systému hranici 51 milionů zpracovaných objednávek za sekundu, a to při zachování stabilního mediánu latence na úrovni 60 ns. Těmito hodnotami systém narazil na fyzikální hardwarové limity procesorové mezipaměti, čímž byl potenciál pro další čistě softwarové zrychlení v jednovláknové architektuře vyčerpán.

Jako logický směr pro další rozvoj projektu se nabízí integrace tohoto samostatného jádra do plnohodnotné softwarové burzy. To by zahrnovalo implementaci nezbytných infrastrukturních komponent, jako jsou vstupní a výstupní

brány tržních dat. Případné nasazení jádra do reálného provozu vyžaduje napojení na živý síťový tok s využitím technologií obcházejících standardní síťový zásobník OS (*kernel bypass*) a zajištění asynchronní komunikace s okolím výhradně prostřednictvím bezzámkových (*lock-free*) front. Z pohledu obchodní funkcionality se následně nabízí rozšíření podporované burzovní logiky o pokročilé typy objednávek a alternativní párovací algoritmy. Technickou výzvou by opět bylo implementovat tato rozšíření burzovní logiky, aniž by to významně degradovalo výkon a deterministicky nízkou latenci. Kapacitu a celkovou propustnost takového systému by pak bylo možné horizontálně škálovat pomocí techniky *CPU sharding* pro jednotlivé obchodované instrumenty.

# Příloha A

# Uživatelská příručka

Tato příloha obsahuje technické požadavky a stručný návod k základnímu sestavení, spuštění a testování implementovaného párovacího jádra.

## A.1 Požadavky na systém

Pro úspěšné sestavení a spuštění projektu je nutné splnit následující požadavky (detailní specifikace testovacího prostředí je uvedena v kapitole 4):

- **Operační systém:** Distribuce na bázi OS Linux.
- **Hardwarová architektura:** Procesor s instrukční sadou x86_64.
- **Překladač:** Kompilátor plně podporující standard C++23 (ověřeno s GCC 15.2.0 a Clang 18.1.3).
- **Sestavovací systém:** Nástroj CMake ve verzi 3.28.3 nebo vyšší.
- **Síťové připojení:** Přístup k internetu pro automatické stažení nezbytných závislostí (knihoven GoogleTest 1.17.0 a Google Benchmark 1.9.5).

## A.2 Sestavení projektu

Sestavovací skript nativně vynucuje standard C++23, kompilaci pro architekturu daného stroje (`-march=native`) a automaticky zapíná optimalizaci LTO/IPO, je-li překladačem podporována.

Pro standardní sestavení s plnými optimalizacemi (*Release*) slouží následující sekvence příkazů spuštěná v kořenovém adresáři projektu:

```
mkdir build && cd build
cmake -DCMAKE_BUILD_TYPE=Release -DCMAKE_CXX_COMPILER=g++-15 ..
cmake --build . -j
```

*Poznámka: Pokročilé možnosti kompilace, jako je dvoufázové sestavení s PGO, jsou detailně popsány přímo v hlavní textové části práce.*

## A.3 Spuštění aplikace

Hlavní spustitelný program nabízí dva primární režimy chodu: načtení tržních událostí ze souboru (formát LOBSTER) nebo zpracování vlastních, náhodně vygenerovaných dat.

### Spuštění se vstupním souborem

Příklad načtení objednávek z CSV souboru, uložení vygenerovaných obchodů do zadaného výstupního souboru a zapnutí měření latence:

```
./matching_engine -i ../data/orders.csv -o trades.csv -l
```

### Spuštění s náhodně generovanými daty

Příklad spuštění s interním generátorem zátěže. Tento příkaz vygeneruje 1 milion objednávek s 10% podílem zrušených objednávek:

```
./matching_engine --generate 1000000 --cancel 10 --seed 42
```

### Nápověda programu

Pro vypsání kompletního seznamu všech dostupných konfiguračních parametrů (včetně ladění afinity vláken, uzamčení paměti či detailního nastavení generátoru) použijte přepínač nápovědy:

```
./matching_engine --help
```

## A.4 Testování a benchmarky

Součástí projektu jsou také automatizované testy a nástroje pro měření výkonu, které se po kompilaci nacházejí ve výstupním `build` adresáři:

- `./unit_tests` – Spustí izolované testy klíčových komponent a logiky párování.
- `./integration_tests` – Ověří celkovou funkčnost jádra (od přijetí zprávy po korektní vygenerování obchodů).
- `./engine_benchmarks` – Spustí sadu výkonnostních mikrobenchmarků postavených nad knihovnou Google Benchmark.

# Bibliografie


1. STAFF OF THE DIVISION OF TRADING AND MARKETS, U.S. SECURITIES AND EXCHANGE COMMISSION (SEC). *Equity Market Structure Literature Review Part II: High Frequency Trading* [online]. 2014-03. Tech. zpr. U.S. Securities a Exchange Commission (SEC). Dostupné z: `https://www.sec.gov/files/marketstructure/research/hft_lit_review_march_2014.pdf`. [Citováno 13. 4. 2026].
2. EUROPEAN SECURITIES AND MARKETS AUTHORITY (ESMA). *MiFID II / MiFIR review report on Algorithmic Trading* [online]. 2021-09. Tech. zpr. ESMA. Dostupné z: `https://www.esma.europa.eu/sites/default/files/library/esma70-156-4572_mifid_ii_final_report_on_algorithmic_trading.pdf`. [Citováno 13. 4. 2026].
3. U.S. SECURITIES AND EXCHANGE COMMISSION (SEC). *Trade To Order Volume Ratios* [online]. 2013-10. Dostupné z: `https://www.sec.gov/data-research/statistics-data-visualizations/trade-order-volume-ratios`. [Citováno 13. 4. 2026].
4. ELECTRONIC TRADING HUB. *Electronic Trading and Order Matching System Basics* [online]. 2024-01. Dostupné z: `https://electronictradinghub.com/electronic-trading-and-order-matching-system-basics/`. [Citováno 14. 4. 2026].
5. SYSTEM DESIGN 101. *Designing a Stock Exchange* [online]. 2026-02. Dostupné z: `https://mintlify.wiki/ByteByteGoHq/system-design-101/case-studies/stock-exchange`. [Citováno 14. 4. 2026].
6. BYTEMONK. *Inside a Real High-Frequency Trading System | HFT Architecture* [online]. 2025-06. Dostupné z: `https://www.youtube.com/watch?v=iwRaNYa8yTw`. [Citováno 14. 4. 2026].
7. DATABENTO. *An introduction to matching engines: A guide by Databento* [online]. 2024-08. Dostupné z: `https://medium.databento.com/an-introduction-to-matching-engines-a-guide-by-databento-d055a125a6f6`. [Citováno 14. 4. 2026].

8. FOWLER, Martin. *The LMAX Architecture* [online]. 2011-07. Dostupné z: https://martinfowler.com/articles/lmax.html. [Citováno 14. 4. 2026].
9. CERTIFIED INVESTMENT RESEARCH ANALYST (CIRA). *Market Orders & Types* [online]. 2022-03. Dostupné z: https://equalifi.org/wp-content/uploads/2022/03/0_20211110121211S3_Market_Orders.pdf. [Citováno 15. 4. 2026].
10. U.S. SECURITIES AND EXCHANGE COMMISSION (SEC). *Investor Bulletin: Understanding Order Types* [online]. 2017-07. Dostupné z: https://www.investor.gov/introduction-investing/general-resources/news-alerts/alerts-bulletins/investor-bulletins-14. [Citováno 15. 4. 2026].
11. FINCHTRADE. *Order Matching Algorithms* [online]. 2026. Dostupné z: https://finchtrade.com/glossary/order-matching-algorithms. [Citováno 15. 4. 2026].
12. CFI TEAM. *Matching Orders* [online]. 2020-07. Dostupné z: https://corporatefinanceinstitute.com/resources/career-map/sell-side/capital-markets/matching-orders/. [Citováno 15. 4. 2026].
13. CME GROUP. *CME matching algorithms explained* [online]. 2025-08. Dostupné z: https://databento.com/blog/cme-matching-algorithms-explained#:~:text=Matching%20algorithms%20determine%20how%20orders%20interact%20with,based%20on%20each%20resting%20order's%20pro%2Drated%20percentage. [Citováno 15. 4. 2026].
14. FILIPOVSKÝ, Jakub. *Performance Driven Stock Exchange Architecture and Implementation*. 2025. Dostupné z: https://dspace.cvut.cz/server/api/core/bitstreams/22533d93-4478-4180-b3d3-6128ce8372ad/content. Bakalářská práce. Fakulta informačních technologií ČVUT v Praze. [Citováno 9. 5. 2026].
15. KUMAR, Piyush. *order-matching-engine* [online]. 2026. Dostupné z: https://github.com/PIYUSH-KUMAR1809/order-matching-engine. [Citováno 9. 5. 2026].
16. FARRIER, John. *Demystifying Floating Point* [online]. 2015. Dostupné z: https://www.youtube.com/watch?v=k12BJGSc2Nc. [Citováno 16. 4. 2026].
17. SUVOROV, Egor. *Using Floating-point in C++: What Works, What Breaks, and Why – Egor Suvorov – CppCon 2025* [online]. 2025. Dostupné z: https://www.youtube.com/watch?v=m83TjrB6wYw. [Citováno 16. 4. 2026].

18. GOLDBERG, David. *What Every Computer Scientist Should Know About Floating Point Arithmetic* [online]. 1991. Dostupné z: https://www.karlin.mff.cuni.cz/~hron/NMMO403/What_every_computer_scientist_should_know_about_floating-point_arithmetic-Goldberg-1991.pdf. [Citováno 16. 4. 2026].

19. TALBOT, Alan. *CppCon 2018: Alan Talbot "Moving Faster: Everyday efficiency in modern C++"* [online]. 2018. Dostupné z: https://www.youtube.com/watch?v=EovBkh9wDnM. [Citováno 17. 4. 2026].

20. BAKHVALOV, Denis. *Performance Analysis and Tuning on Modern CPUs: Learn to write fast software like a pro*. Independently published, 2024. ISBN 9798342846448.

21. MEYERS, Scott. *code::dive conference 2014 – Scott Meyers: Cpu Caches and Why You Care* [online]. 2014. Dostupné z: https://www.youtube.com/watch?v=WDIkqP4JbkE. [Citováno 16. 4. 2026].

22. CARRUTH, Chandler. *CppCon 2014: Chandler Carruth "Efficiency with Algorithms, Performance with Data Structures"* [online]. 2014. Dostupné z: https://www.youtube.com/watch?v=fHNmRkzxHWs. [Citováno 16. 4. 2026].

23. MÜLLER, Jonathan. *Cache-Friendly C++ – Jonathan Müller – CppCon 2025* [online]. 2025. Dostupné z: https://www.youtube.com/watch?v=g_X5g3xw43Q. [Citováno 16. 4. 2026].

24. SIMONDEV. *Memory, Cache Locality, and why Arrays are Fast (Data Structures and Optimization)* [online]. 2021. Dostupné z: https://www.youtube.com/watch?v=247cXLkYt2M. [Citováno 16. 4. 2026].

25. SIMONDEV. *Why Linked Lists vs Arrays isn't a real choice* [online]. 2021. Dostupné z: https://www.youtube.com/watch?v=34ky600VTN0. [Citováno 16. 4. 2026].

26. ACTON, Mike. *CppCon 2014: Mike Acton "Data-Oriented Design and C++"* [online]. 2014. Dostupné z: https://www.youtube.com/watch?v=rX0ItVEVjHc. [Citováno 18. 4. 2026].

27. COOK, Carl. *CppCon 2017: Carl Cook "When a Microsecond Is an Eternity: High Performance Trading Systems in C++"* [online]. 2017. Dostupné z: https://www.youtube.com/watch?v=NH1Tta7purM. [Citováno 18. 4. 2026].

28. GROSS, David. *When Nanoseconds Matter: Ultrafast Trading Systems in C++ – David Gross – CppCon 2024* [online]. 2024. Dostupné z: https://www.youtube.com/watch?v=sX2nF1fW7kI. [Citováno 18. 4. 2026].

29. STROUSTRUP, Bjarne; SUTTER, Herb. *C++ Core Guidelines* [online]. 2025. Dostupné z: https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines. [Citováno 18. 4. 2026].

30. LAKOS, John. *CppCon 2017: John Lakos "Local ('Arena') Memory Allocators (part 1 of 2)"* [online]. 2017. Dostupné z: https://www.youtube.com/watch?v=nZNd5FjSquk. [Citováno 18. 4. 2026].

31. PIKUS, Fedor. *Branchless Programming in C++ – Fedor Pikus – CppCon 2021* [online]. 2021. Dostupné z: https://www.youtube.com/watch?v=gWPhYREFjk. [Citováno 19. 4. 2026].

32. THE LINUX KERNEL ORGANIZATION. *CFS Scheduler* [online]. 2024. Dostupné z: https://docs.kernel.org/scheduler/sched-design-CFS.html. [Citováno 9. 4. 2026].

33. BILOKON, Paul; GUNDUZ, Burak. *C++ design patterns for low-latency applications including high-frequency trading* [online]. 2023. Dostupné z: https://arxiv.org/pdf/2309.04259. [Citováno 19. 4. 2026].

34. GREGG, Brendan. *Linux perf Examples – Scheduler Analysis* [online]. 2020. Dostupné z: https://www.brendangregg.com/perf.html#SchedulerAnalysis. [Citováno 9. 4. 2026].

35. LOBSTER. *LOBSTER – Academic limit order book data* [online]. 2013. Dostupné z: https://data.lobsterdata.com. [Citováno 2. 4. 2026].

36. GOOGLE INC. *GoogleTest – C++ testing and mocking framework* [online]. 2026. Dostupné z: https://github.com/google/googletest. [Citováno 3. 4. 2026].

37. GOOGLE INC. *Google Benchmark – A microbenchmark support library* [online]. 2026. Dostupné z: https://github.com/google/benchmark. [Citováno 3. 4. 2026].

38. LINUX FOUNDATION. *perf(1) – Linux manual page* [online]. 2026. Dostupné z: https://man7.org/linux/man-pages/man1/perf.1.html. [Citováno 3. 4. 2026].

39. LINUX FOUNDATION. *mlock(2) – Linux manual page* [online]. 2026. Dostupné z: https://man7.org/linux/man-pages/man2/mlock.2.html. [Citováno 7. 4. 2026].

40. GOOGLE INC. *Google Benchmark – User's guide* [online]. 2026. Dostupné z: https://google.github.io/benchmark. [Citováno 3. 4. 2026].

41. UEFI FORUM. *Advanced Configuration and Power Interface (ACPI) Specification, Release 6.6* [online]. 2025. Dostupné z: https://uefi.org/specs/ACPI/6.6/08_Processor_Configuration_and_Control.html. [Citováno 8. 4. 2026].

42. ADVANCED MICRO DEVICES, INC. *AMD Ryzen™ Technology: Precision Boost 2 Performance Enhancement* [online]. 2026. Dostupné z: https://www.amd.com/en/resources/support-articles/faqs/CPU-PB2.html. [Citováno 8. 4. 2026].

43. THE LINUX KERNEL ORGANIZATION. *CPU Performance Scaling* [online]. 2026. Dostupné z: https://docs.kernel.org/admin-guide/pm/cpufreq.html#generic-scaling-governors. [Citováno 8. 4. 2026].

44. ADVANCED MICRO DEVICES, INC. *NO COMPROMISE: DRIVING SERVER PERFORMANCE AND EFFICIENCY WITH AMD EPYC™ AND SMT* [online]. 2025. Dostupné z: https://www.amd.com/content/dam/amd/en/documents/epyc-business-docs/white-papers/amd-epyc-smt-technology-brief.pdf. [Citováno 8. 4. 2026].

45. THE LINUX KERNEL ORGANIZATION. *EEVDF Scheduler* [online]. 2024. Dostupné z: https://docs.kernel.org/scheduler/sched-eevdf.html. [Citováno 9. 4. 2026].

46. LINUX FOUNDATION. *pthread_setaffinity_np(3) – Linux manual page* [online]. 2025. Dostupné z: https://man7.org/linux/man-pages/man3/pthread_setaffinity_np.3.html. [Citováno 9. 4. 2026].

47. BAKHVALOV, Denis. *Performance Analysis and Tuning on Modern CPUs* [online]. 2024. Dostupné z: https://github.com/dendibakh/perf-book/releases/tag/2.0_release. Kapitola 2.5: Software and Hardware Timers. [Citováno 9. 4. 2026].

48. ADVANCED MICRO DEVICES, INC. *AMD64 Architecture Programmer's Manual Volume 2: System Programming* [online]. 2026. Dostupné z: https://docs.amd.com/v/u/en-US/24593_3.44_APM_Vol2. Kapitola 13.2.4: Time-Stamp Counter. [Citováno 10. 4. 2026].

49. ADVANCED MICRO DEVICES, INC. *AMD64 Architecture Programmer's Manual Volume 3: General-Purpose and System Instructions* [online]. 2025. Dostupné z: https://docs.amd.com/v/u/en-US/24594_3.37. Kapitola 4: System Instruction Reference, RDTSCP. [Citováno 10. 4. 2026].

# Obsah příloh

Zdrojové kódy implementace, včetně kompletní historie vývoje, jednotlivých verzí a vstupních dat, jsou také veřejně dostupné v repozitáři:
`https://github.com/markesjan/order-matching-engine`.

```
/
├── impl ..... zdrojové kódy párovacího jádra
│   ├── benchmark ..... výkonnostní mikrobenchmarky
│   ├── data ..... ukázková vstupní data
│   ├── include ..... hlavičkové soubory
│   ├── measurement ..... výsledky měření
│   ├── src ..... implementační soubory
│   └── test ..... jednotkové a integrační testy
├── text ..... zdrojové kódy textu práce
└── thesis.pdf ..... text práce ve formátu PDF
```